Module 2: Data and descriptive statistics

307312 Applied Statistics | University of Petra

Module contents

Click a topic below, or use the PDF reader bookmarks.
Every page has links to contents and the previous/next topic.

Overview 2

Module 2: Data Types and the Descriptive Statistics Toolbox 4

Module 2: Data Types and Toolbox — Practice 21

Module 2: Frequency Tables and Charts 24

Module 2: Frequency Tables and Charts — Practice 41

Module 2: Numerical Measures 44

Module 2: Numerical Measures — Practice 64

Module 2: Relationships Between Variables 67

Module 2: Relationships Between Variables — Practice 80

Module 2: Descriptive Statistics Review 82

Module 2: Data and descriptive statistics

Start with the variables, then choose the tools

Table of contents

Reduced teaching material 1
Learning sequence 1
Start here 1
Textbook examples used in this module 2
CSV data files 2
Development status 2

Reduced teaching material

Use the concise slides during class and the extended lessons below for full explanations, worked R code, and practice. Both follow the same lesson sequence.

- Reduced teaching slides (PDF)

Learning sequence

1. **Understand the data:** observational units, categorical and numerical variables, discrete and continuous quantities, and the four measurement levels.
2. **Meet the descriptive statistics toolbox:** tables, charts, and numerical measures.
3. **Apply the tools:** summarize distributions, compare groups, and interpret relationships in context.

Our reference is the 8th-edition textbook, Section 1.2 and Chapters 2–3. The discrete/continuous distinction from Section 5.1 is introduced early because it helps us choose summaries.

Start here

- Lesson 1: Data types and the descriptive statistics toolbox — definitions, original diagrams, a book figure, and worked R examples.
- Practice: Data types and toolbox — classification, tool selection, and short analyses.
- Lesson 2: Frequency tables and charts — counts, proportions, bar/pie charts, numerical classes, histogram shape and skewness, modality, unusual values, and cumulative frequencies.
- Practice: Frequency tables and charts — construct, check, and interpret the displays.
- Lesson 3: Numerical measures — center, variability, quartiles, boxplots, z-scores, and descriptive group comparisons.

- Practice: Numerical measures — calculations, sensitivity, and interpretation.
- Lesson 4: Relationships between variables — crosstabs, conditional percentages, scatterplots, covariance, correlation, and interpretation limits.
- Practice: Relationships between variables — denominator choice and paired-data analysis.
- Integrated descriptive statistics review — a short Quarto report using all four textbook datasets.

Read each lesson before attempting its practice. Instructor answers are kept separately.

Textbook examples used in this module

We use examples from our main 8th-edition book, choosing each for its teaching purpose:

- **Restaurant ratings and meal prices:** variable meaning, ordered categories, crosstabs, and group comparisons.
- **Soft-drink preferences:** categorical frequency tables, proportions, and bar charts in Lesson 2.
- **Audit durations:** numerical distributions, histograms, center, spread, and quartiles.
- **Electronics advertising and sales:** scatterplots and descriptive association.

Worked R analyses use prepared CSVs from the local textbook workbooks, with checks against the printed 8th-edition examples. The introductory lesson currently uses Restaurant, Audit, and Electronics. Lessons 2–4 develop these examples with reproducible R analyses.

CSV data files

Student analyses use CSV files. Download all four datasets and the data guide, or select an individual file:

- Restaurant ratings and prices — 300 restaurants.
- Audit durations — 20 audits.
- Electronics advertising and sales — 10 weeks.
- Soft-drink preferences — 50 observations.

The guide explains columns, units, category ordering, source files, and verification limits. The original Excel files are retained as source records.

Development status

The planned descriptive-statistics sequence is complete: Lessons 1–4, their practice, and an integrated review are available. Separate instructor answer guides cover all four lessons and the review. Proceed through the review before moving to probability and distributions.

Return to the course home.

Module 2: Data Types and the Descriptive Statistics Toolbox

Lesson 1 | Classify the data, then choose the summary

Table of contents

What is statistics? . . . 1
Our starting question . . . 2
First identify the observation and the variable . . . 3
Categorical and numerical data . . . 3
Discrete and continuous numerical variables . . . 4
Levels of measurement . . . 5
The descriptive statistics toolbox . . . 8
Tables: organize the information . . . 9
Charts: choose a display for the question . . . 10
Numerical measures: quantify the pattern . . . 12
Combine the tools to answer the business question . . . 14
Choose a tool: a short checklist . . . 16
Data for this lesson . . . 17
References and provenance . . . 17

Reduced teaching slides (PDF) · Module overview

This extended lesson provides the full explanation and reproducible examples behind the slides.

What is statistics?

Statistics is the science of collecting, organizing, presenting, analyzing, and interpreting data to answer questions and support informed decisions.



Figure 1: Conceptual illustration of statistical displays, supplied for this lesson.

Our starting question

A restaurant analyst asks: **How do meal prices differ between restaurants with different quality ratings?** Before choosing a formula or drawing a chart, we must understand what each variable means.

This case gives the lesson a concrete purpose. We will return to the same question as we classify variables, select tables and charts, and interpret numerical summaries.

يسأل محلّل يدرس بيانات المطاعم: **كيف تختلف أسعار الوجبات بين المطاعم ذات تقييمات الجودة المختلفة؟** قبل اختيار معادلة أو رسم بياني، علينا أن نفهم معنى كل متغيّر.

تمنحنا هذه الحالة سؤالًا عمليًا يوجّه التعلّم. سنعود إلى السؤال نفسه عند تصنيف المتغيّرات، واختيار الجداول والرسوم البيانية، وتفسير المقاييس العددية.



Figure 2: Restaurant case: one restaurant becomes one dataset record. Adapted instructor-supplied illustration; the values match the first checked CSV record.

This lesson follows a deliberate sequence:

1. Define categorical and numerical data.

2. Distinguish discrete and continuous numerical variables.
3. Identify nominal, ordinal, interval, and ratio measurement levels.
4. Choose from the descriptive statistics toolbox: **tables, charts, and numerical measures**.

By the end, you should be able to classify a variable, justify a suitable summary, and interpret the result without making claims beyond the data.

Book reference: Section 1.2 (printed pp.5–8); the discrete/continuous distinction in Section 5.1 (pp.219–220); Chapter 2 for tables and charts; Chapter 3 for numerical measures. We introduce the distinction from Chapter 5 now, without requiring probability distributions.

First identify the observation and the variable

An **element**, or observational unit, is the entity being studied. A **variable** is a characteristic recorded for that entity. An **observation** is the collection of recorded values for one unit.

In our restaurant dataset:

- The observational unit is **one restaurant**.
- The variables include an identifier, a quality rating, and a meal price.
- One row contains one restaurant's recorded values; one column contains one variable.

Download the Restaurant CSV. Run from the course RStudio project; each row is one restaurant.

```
# library() loads ggplot2 for charts and knitr for table formatting.
# read.csv() reads the checked Restaurant teaching CSV.
# factor()/c() retain the ordered ratings; levels fixes the category order.
library(ggplot2)
library(knitr)
restaurants <- read.csv("data/module02/restaurant_ratings.csv")
restaurants$quality_rating <- factor(restaurants$quality_rating,
  levels = c("Good", "Very Good", "Excellent"), ordered = TRUE)
```

Restaurant ID	Quality rating	Meal price (USD)
1	Good	18
2	Very Good	22
3	Good	28
4	Excellent	38
5	Very Good	33

The same-looking number can play different roles. The value `101` could identify a restaurant, count orders, or measure a price. Its statistical meaning comes from the variable definition and units, not from its appearance in a spreadsheet.

Categorical and numerical data

Categorical data identify a group, label, or ordered category. They tell us **which category** an observation belongs to.

Numerical data, also called **quantitative data**, represent a quantity. They tell us **how many** or **how much**. Numerical differences have a quantitative interpretation, although the measurement level determines which other operations are meaningful.

Variable	Example values	Data type	Why?
Payment method	Cash, card, e-wallet	Categorical	The values identify groups
Restaurant quality	Good, Very Good, Excellent	Categorical	The values identify ordered categories
Restaurant ID	101, 102, 103	Categorical identifier	The numbers are labels, not quantities
Orders received	0, 1, 2, 3, ...	Numerical	The values count orders
Waiting duration	2.4, 5.1, 8.7 minutes	Numerical	The values measure elapsed time
Meal price	18, 22, 28 USD	Numerical	The values measure an amount of money

> **i** A numerical code is not automatically a numerical variable
>
> Coding Cash = 1, Card = 2, and E-wallet = 3 does not make payment method quantitative. An average code of 2.1 does not mean an average payment method. Likewise, a quality scale coded 1, 2, 3 remains ordinal unless equal spacing has been justified.

Check your understanding: Is a telephone number a numerical quantity? Would subtracting two telephone numbers answer a meaningful measurement question?

Discrete and continuous numerical variables

Once a variable represents a quantity, ask which values are possible.

A **discrete variable** has a finite or countably infinite set of possible values. Values occur at separate points. Counts are common examples: a restaurant can receive 12 or 13 orders, but not 12.6 completed orders.

A **continuous variable** is modeled as able to take any value within an interval. Measurements such as waiting duration, weight, distance, and temperature are common examples. Between 2 and 3 minutes, values such as 2.1, 2.14, and 2.145 minutes are possible.

Discrete	Continuous
Number of customers arriving in an hour	Time between customer arrivals
Number of defective items in a batch	Weight of an item
Number of complaints in a day	Duration of a service call
Number of advertising commercials	Distance traveled

The distinction concerns possible values, not whether the displayed data contain decimal points. A discrete variable can take fractional steps, such as package sizes restricted to 0.5, 1.0, and 1.5 liters. Conversely, a continuous measurement may be rounded to a whole number.

Our audit times are recorded in whole days, but elapsed audit duration is an underlying continuous quantity. Money recorded to the nearest cent has discrete recording increments; business analyses often approximate monetary amounts using continuous models. State the recording precision and modeling choice when they matter.

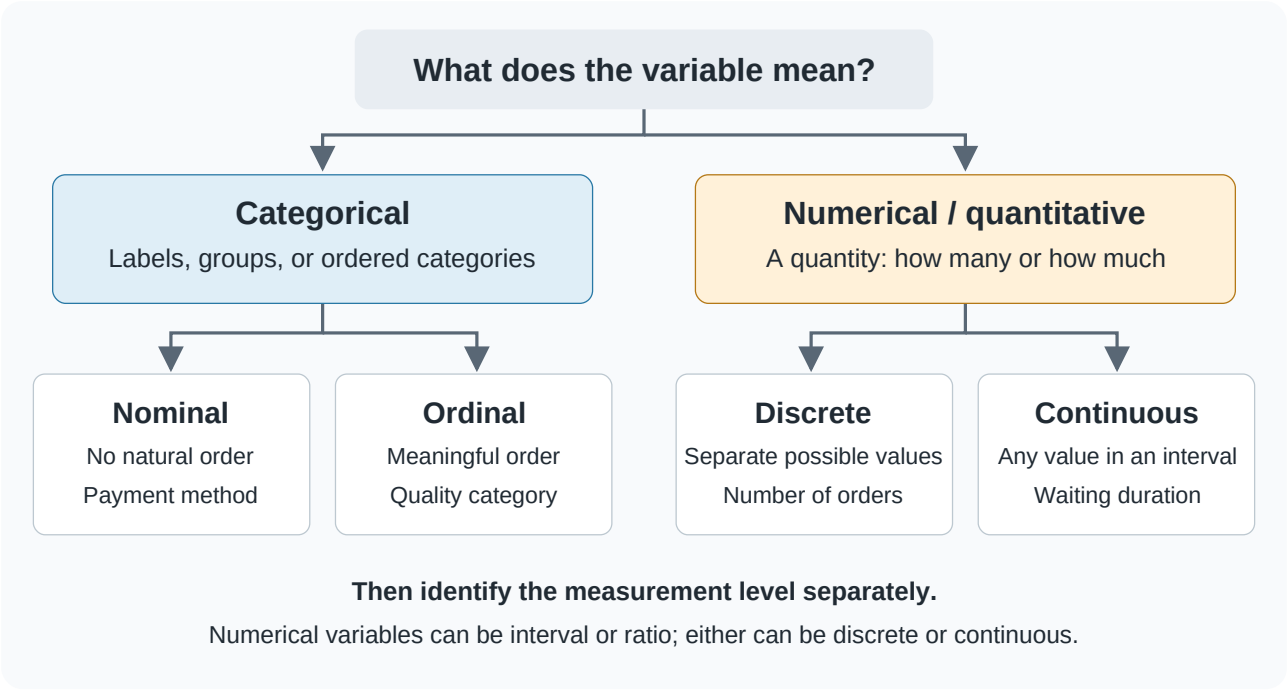


Figure 3: Original course diagram: classify the meaning of a variable before deciding how to summarize it.

Levels of measurement

A measurement level describes what comparisons and operations make sense. The book distinguishes **nominal, ordinal, interval, and ratio** levels.

Level	What the values tell us	Meaningful comparisons	Example
Nominal	Category membership	Same or different; category counts and proportions	Payment method, restaurant ID
Ordinal	Membership and meaningful order	Higher/lower; ordered frequencies, mode, and median category	Good, Very Good, Excellent
Interval	Order and equal measurement units	Differences and averages; ratios of raw values are not meaningful	Temperature in degrees Celsius
Ratio	Equal units and a meaningful zero indicating absence of the measured quantity	Differences and ratios, where defined	Duration, distance, weight, number of orders

Nominal: labels without a natural ranking

Cash is not inherently above or below card. Relabeling their category codes changes no substantive information. Use counts, percentages, and category comparisons.

Ordinal: order without established equal gaps

Excellent is above Very Good, and Very Good is above Good. That does not establish that the two gaps are equal. An ordered bar chart is meaningful; treating the category labels as equally spaced measurements is an additional assumption.

Interval: equal differences, an arbitrary zero

A change from 10°C to 20°C has the same size as a change from 20°C to 30°C. However, 20°C is not twice the temperature of 10°C: the zero is a conventional reference point. A mean in degrees Celsius is meaningful; a ratio of Celsius readings is not.

Ratio: equal differences and a meaningful zero

Zero minutes of waiting means no waiting time. A wait of 20 minutes is twice a wait of 10 minutes. A ratio such as 20/0 is still undefined: a meaningful zero does not permit division by zero.

Two classifications, two questions

Discrete/continuous: which values are possible?
Measurement level: which comparisons are meaningful?
For example, order count is **discrete and ratio**; waiting duration is **continuous and ratio**; Celsius temperature is usually **continuous and interval**. Calendar year recorded as a year number is **discrete and interval**. The two classifications are not interchangeable.

In R, storage type is a separate issue again. `numeric` does not prove that a variable is quantitative. We converted restaurant ID to character and stored the ordered quality categories as an ordered factor to make their intended roles clearer.

Summary: what each measurement level adds

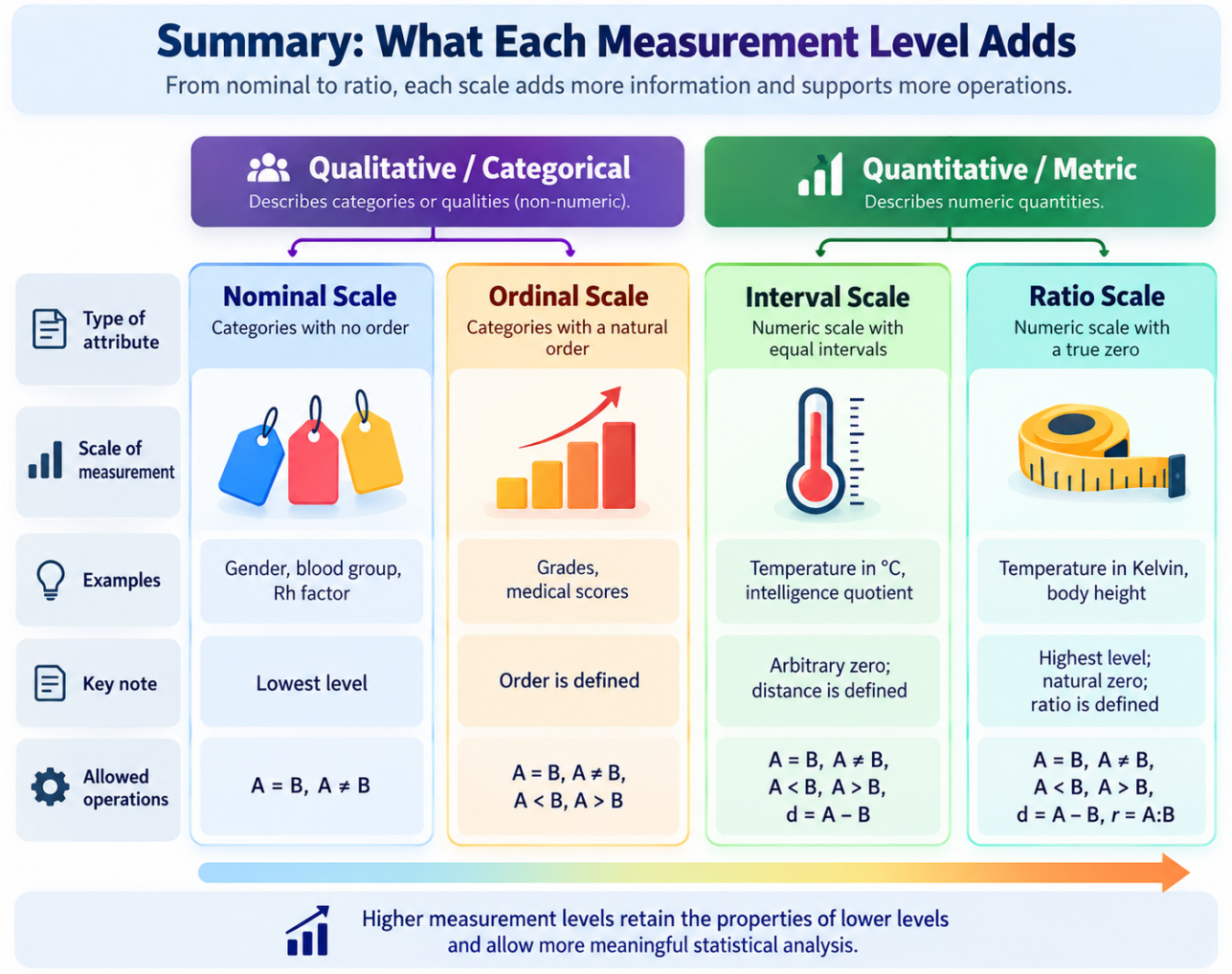


Figure 4: Measurement levels and their meaningful comparisons. Instructor-supplied summary diagram.

Open the diagram at full size.

Read from left to right: nominal data distinguish categories; ordinal data also support ranking; interval data also support meaningful differences; ratio data additionally support meaningful ratios because they have a meaningful zero. Each step adds measurement information and permits additional summaries and comparisons. For ratios, the denominator must still be nonzero.

This progression describes **which operations are meaningful**, rather than guaranteeing a better analysis. Categorical data also support substantial statistical analysis. The question, study design, and method assumptions still matter. Classify examples such as “grades” or “medical scores” from their actual definitions: ordered rating categories are ordinal, but not every numerical score automatically has that scale.

The descriptive statistics toolbox



Figure 5: Conceptual illustration of the descriptive statistics toolbox.

Descriptive statistics organize, display, and summarize the observations we have. They do not, by themselves, justify generalizing from a sample to every restaurant or explaining what caused a pattern.

One question, three complementary tools

Tables	Charts	Numerical measures
Organize and compare	See the pattern	Quantify the pattern
Counts and percentages	Shape and unusual values	Center and spread
Grouped frequencies	Group differences	Position and association

Interpret the evidence in the context of the question.

Figure 6: Original course illustration: tables, charts, and numerical measures describe complementary aspects of the same data.

Tool	Main purpose	Typical examples	Book reference
Tables	Organize observations into an interpretable summary	Frequency tables, relative frequencies, grouped distributions, crosstabs	Chapter 2 §§2.1–2.3
Charts	Reveal patterns and comparisons	Bar charts, histograms, dot plots, boxplots, scatterplots	Chapter 2 §§2.1–2.5; Chapter 3 §3.4
Numerical measures	Quantify important features	Mean, median, standard deviation, quartiles, correlation	Chapter 3 §§3.1–3.5

We normally combine tools. A mean can hide an unusual observation; a graph can reveal it. A graph can suggest a difference; a numerical summary tells us its size.

Tables: organize the information

Frequency and relative frequency

A **frequency** is the number of observations in a category or class. A **relative frequency** is that count divided by the total. Multiply a relative frequency by 100 to express it as a percentage.

For the 300 restaurants, keep the quality categories in their natural order.

```
# table() counts the ordered ratings; as.data.frame() creates a table.
# names()/c() label columns; nrow() gives the 300-restaurant percentage denominator.
# kable() displays the frequencies, proportions, and percentages; digits sets precision.
quality_table <- as.data.frame(table(restaurants$quality_rating))
names(quality_table) <- c("Quality", "Frequency")
quality_table$Relative_frequency <- quality_table$Frequency / nrow(restaurants)
quality_table$Percent <- 100 * quality_table$Relative_frequency
kable(quality_table, digits = c(0,0,2,0),
      col.names = c("Quality", "Frequency", "Relative frequency", "Percent"))
```

Quality	Frequency	Relative frequency	Percent
Good	84	0.28	28
Very Good	150	0.50	50
Excellent	66	0.22	22

Interpretation: 150 of the 300 restaurants, or 50%, are rated Very Good. This describes the observed restaurants; it does not establish the proportion among all restaurants in the city.

Grouped frequency tables

For numerical variables with many possible values, group observations into intervals. Classes must not overlap or leave unintended gaps.

An audit manager wants to understand how long audits take and how much their durations vary. We use 20 recorded audit durations, measured in days, to describe their distribution.

يريد مدير التدقيق معرفة المدة التي تستغرقها عمليات التدقيق ومدى تفاوت هذه المدد. نستخدم بيانات المدة المسجّلة لعشرين عملية تدقيق، مقيسة بالأيام، لوصف توزيعها.

Download the Audit CSV. Its 20 durations match the book's Table 2.4.

```r
# library() loads ggplot2 for charts and knitr for table formatting.
# read.csv() loads the checked Audit CSV; $ selects its audit_days column.
# x is the vector of 20 durations in days, not class midpoints.
library(ggplot2)
library(knitr)
audit <- read.csv("data/module02/audit_times.csv")
x <- audit$audit_days
```

```r
# names(): reads or assigns names to vector elements or table columns.
# c(): combines values into a vector, converting them to a common type when needed.
# stopifnot(): stops execution if any supplied condition is FALSE; successful checks
 ↪  produce no output.
# identical(): requires exact equality of values, types, and attributes.
# cut() groups days using boundaries 10, 15, ..., 35 from seq(10,35,5).
# right = FALSE includes the lower endpoint: [10,15), [15,20), and so on.
# table() counts each class; as.data.frame() makes the counts into columns.
# stopifnot() checks the expected counts; kable() displays a readable table.
classes <- cut(audit$audit_days, breaks = seq(10,35,5), right = FALSE)
audit_frequency <- as.data.frame(table(classes))
names(audit_frequency) <- c("Audit_days", "Frequency")
stopifnot(identical(audit_frequency$Frequency, c(4L,8L,5L,2L,1L)))
kable(audit_frequency, col.names = c("Audit duration class (days)", "Frequency"))
```

Audit duration class (days)	Frequency
[10,15)	4
[15,20)	8
[20,25)	5
[25,30)	2
[30,35)	1

Here `[15,20)` means **at least 15 but less than 20 days**. An audit lasting 20 days belongs in `[20,25)`. For these whole-day records, the intervals correspond to the book's 10–14, 15–19, and subsequent classes.

A **cumulative frequency** adds the counts up to a given ordered class. It is useful for questions such as "How many audits finished in less than 25 days?" A **crosstab** summarizes two categorical variables, or a category and a grouped numerical variable, in a two-way table.

Charts: choose a display for the question

Data and question	Suitable display	What to look for
One categorical variable: how common is each category?	Bar chart; sometimes a pie chart for a simple part-to-whole comparison	Counts or percentages; preserve order for ordinal categories
A numerical count with few possible values	Frequency bar chart or dot plot	Which counts occur and how often
One numerical variable with many values	Histogram or dot plot	Center, spread, shape, gaps, and unusual values
Numerical values across several groups	Side-by-side boxplots	Medians, variability, overlap, and possible outliers
Two numerical variables	Scatterplot	Direction, form, strength, and unusual observations
A quantity observed over time	Time-series line chart	Trend, seasonality, changes, and unusual periods

A bar chart separates categories; a histogram groups values along a numerical axis. Equal-width histogram bars can show counts directly. A pie chart can communicate a small number of proportions, but bar charts usually make precise comparisons easier. Avoid 3D decoration that distorts the comparison.

```
# ggplot()/aes() map categories and existing counts; geom_col() draws their heights.
# geom_text() labels counts; vjust moves labels and width controls bar width.
# scale_y_continuous()/c() set a zero baseline; labs() labels the axes and title.
ggplot(quality_table, aes(Quality, Frequency)) +
  geom_col(fill = "#2878A5", width = .65) +
  geom_text(aes(label = Frequency), vjust = -.4) +
  scale_y_continuous(limits = c(0,170)) +
  labs(title = "Very Good is the most common quality category",
       x = "Quality rating", y = "Number of restaurants")
```

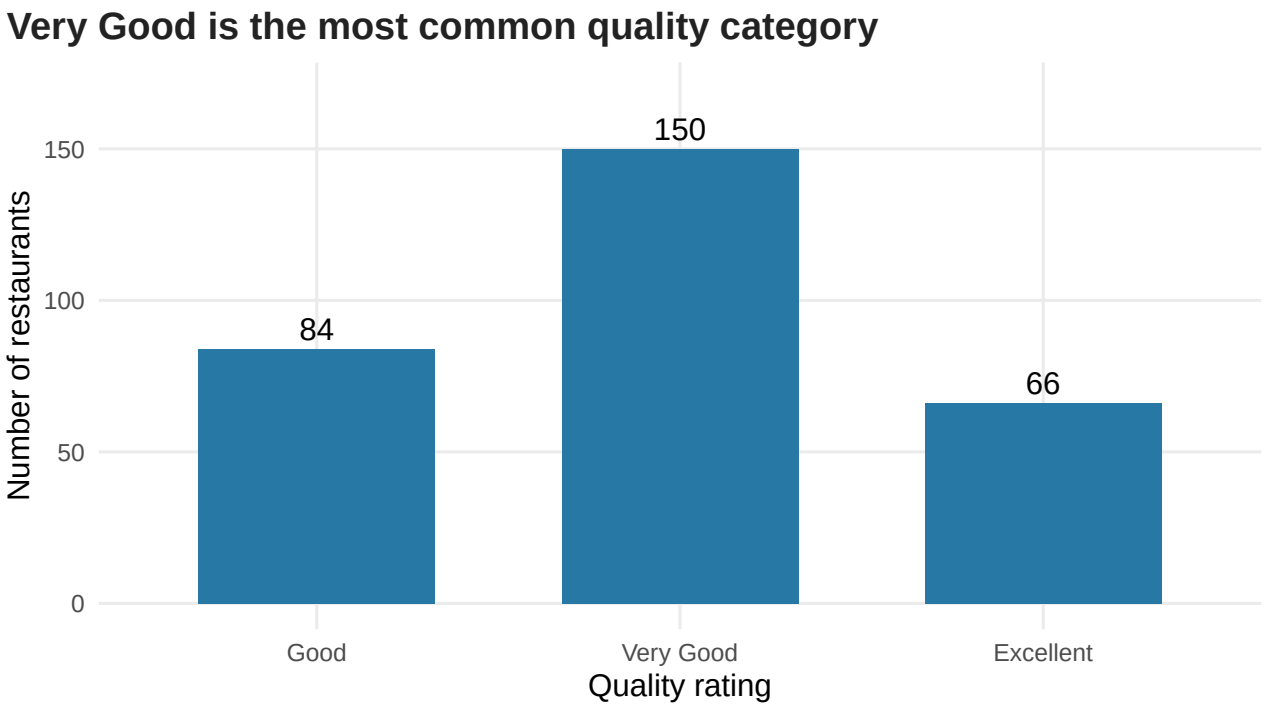


Figure 7: Original R chart from the checked Restaurant workbook. Counts agree with the book's Table 2.10 (printed p.66).

See the observations, then summarize their distribution

Each dot in the book's audit plot represents one audit; stacked dots represent repeated values.

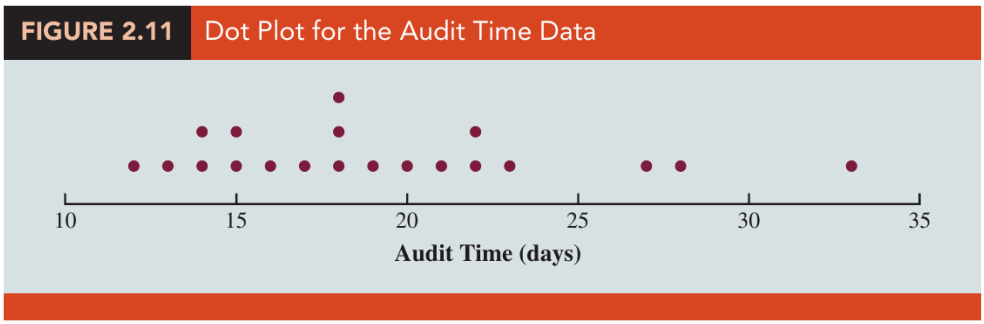


Figure 8: Book Figure 2.11: Dot Plot for the Audit Time Data. Anderson et al., Essentials of Modern Business Statistics with Microsoft Excel, 8th ed., printed p.52 (PDF p.84). Copyright 2020 Cengage Learning. Cropped excerpt from the local reference.

The histogram below uses the same checked observations and the same class boundaries as our grouped table.

```
# ggplot()/aes() map raw days; geom_histogram() counts them into seq() boundaries.
# closed = "left" includes lower endpoints; scale_x_continuous() marks boundaries.
# labs() labels the days and counts; fill/colour distinguish adjacent bars.
ggplot(audit, aes(audit_days)) +
  geom_histogram(breaks = seq(10,35,5), closed = "left",
                 fill = "#2878A5", colour = "white") +
  scale_x_continuous(breaks = seq(10,35,5)) +
  labs(title = "Most audits finish in the lower duration classes",
       x = "Audit duration (days)", y = "Number of audits")
```

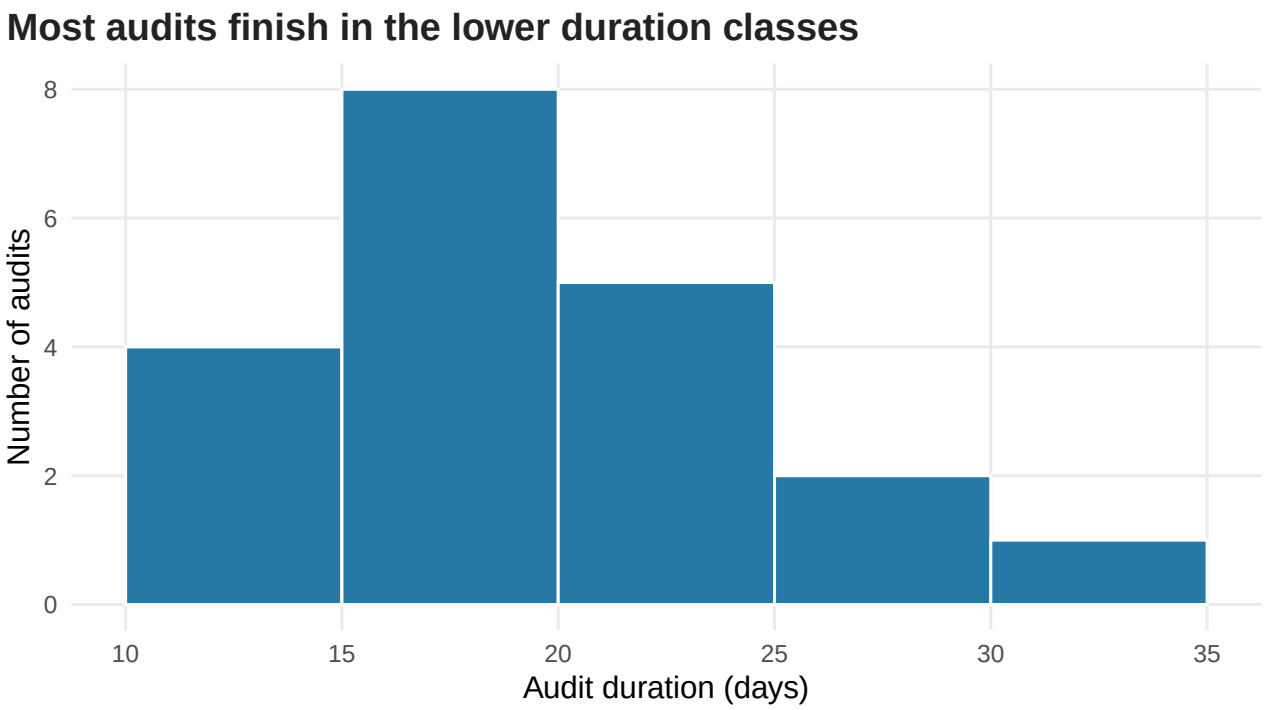


Figure 9: Original R histogram. The class frequencies are 4, 8, 5, 2, and 1, matching the book's audit example.

Do not remove the 33-day audit just because it is separated from the main cluster. First check whether it is valid and whether its circumstances differ from those of other audits.

Numerical measures: quantify the pattern

Numerical measures answer different questions. There is no single number that describes every important feature of a dataset.

Feature	Measures	Interpretation
Location / center	Mean, median, mode	What is typical or most frequent?
Variability / spread	Range, interquartile range, variance, standard deviation	How different are the observations from one another?
Position	Percentiles, quartiles, z-scores	Where does an observation sit in the distribution?

Feature	Measures	Interpretation
Shape and unusual values	Skewness, supported by plots and outlier checks	Is the distribution asymmetric? Which observations need investigation?
Association	Covariance and Pearson correlation	How do two numerical variables vary together, especially in a linear pattern?

Location

The **mean** adds the numerical observations and divides by their count: $\bar{x} = \sum x_i/n$. The **median** is the middle value after sorting, or the average of the two middle values when the numerical sample size is even. The **mode** is the most frequent value or category; a dataset can have more than one mode.

Mean and standard deviation are not meaningful summaries of arbitrary category codes. For ordinal categories, an ordered median category can be meaningful, but subtracting two category labels is not.

Spread and position

The **range** is maximum minus minimum. The **interquartile range (IQR)** is $Q_3 - Q_1$ and describes the width of the middle half of the observations. Quartiles are the 25th, 50th, and 75th percentiles.

The **sample variance** is $s^2 = \sum(x_i - \bar{x})^2/(n-1)$. The **sample standard deviation**, $s = \sqrt{s^2}$, expresses spread in the original measurement units. Variance has squared units. Population variance uses the population size in the denominator instead.

The **coefficient of variation**, $100s/\bar{x}$, expresses standard deviation relative to the mean. Reserve this comparison for suitable ratio-scale variables with a meaningful zero and a positive, nonzero mean; a Celsius-temperature CV is not meaningful.

A **z-score** expresses distance from the mean in standard-deviation units: $(x - \bar{x})/s$. It requires a positive standard deviation, and does not by itself establish that the distribution is normal.

```r
# c(): combines values into a vector, converting them to a common type when needed.
# data.frame(): builds a table whose named columns have compatible lengths.
# length(): counts elements in a vector or list.
# min(): returns the smallest supplied value.
# max(): returns the largest supplied value.
# diff(): subtracts each preceding value from the next by default; diff(range(x)) gives
 ↪  max minus min.
# range(): returns the minimum and maximum as a two-element vector.
# var(): computes sample variance using denominator n-1.
# kable(): formats a data frame or matrix as a readable report table; digits sets
 ↪  displayed precision.
# quantile(..., type = 6) calculates quartiles using our stated book convention.
# mean(), median(), sd(), and the quartiles summarize different distribution features.
x <- audit$audit_days
quartiles <- quantile(x, c(.25,.50,.75), type = 6, names = FALSE)
summary_table <- data.frame(
  Measure = c("Observations", "Mean (days)", "Median (days)",
              "Minimum (days)", "Maximum (days)", "Range (days)",
              "Q1 (days)", "Q3 (days)", "IQR (days)",
              "Sample variance (days squared)", "Sample SD (days)"),
  Value = c(length(x), mean(x), median(x), min(x), max(x), diff(range(x)),
            quartiles[1], quartiles[3], diff(quartiles[c(1,3)]), var(x), sd(x)))
```

```
kable(summary_table, digits = 2)
```

Measure	Value
Observations	20.00
Mean (days)	19.25
Median (days)	18.00
Minimum (days)	12.00
Maximum (days)	33.00
Range (days)	21.00
Q1 (days)	15.00
Q3 (days)	22.00
IQR (days)	7.00
Sample variance (days squared)	29.57
Sample SD (days)	5.44

The mean is **19.25 days** and the median is **18 days**. The standard deviation is approximately **5.44 days**. The middle half of the observations lies between **15 and 22 days**. Longer audits pull the mean above the median; the graph gives that difference context.

> **Matching the book's percentile convention**
>
> On printed pp.113–114, the book locates the pth percentile at $(n+1)p/100$ and interpolates between adjacent observations. R's `quantile(..., type = 6)` follows this convention for the interior percentiles used here. R's default is `type = 7`, so specify the method when reproducing the book's calculations. There are several legitimate percentile conventions.

```
# quantile() calculates the probabilities supplied by c() from audit-day vector x.
# type = 6 matches the book here; names = FALSE omits percentile labels.
quantile(x, c(.25, .50, .75), type = 6, names = FALSE)
```

```
[1] 15 18 22
```

Combine the tools to answer the business question

Return to the restaurants. Quality is ordinal and price is numerical, so an ordered table of group medians and spreads, together with side-by-side boxplots, is a useful descriptive starting point.

```
# names(): reads or assigns names to vector elements or table columns.
# quantile(): computes sample percentiles; probs gives probabilities and type selects the
 ↪  interpolation rule.
# c(): combines values into a vector, converting them to a common type when needed.
# data.frame(): builds a table whose named columns have compatible lengths.
# length(): counts elements in a vector or list.
# median(): returns the middle sorted value, averaging the middle pair for even sample
 ↪  sizes.
# kable(): formats a data frame or matrix as a readable report table; digits sets
 ↪  displayed precision.
# split() separates meal prices by rating while retaining the original values.
# lapply() summarizes each group; do.call(rbind, ...) combines the summary rows.
price_by_quality <- split(restaurants$meal_price_usd, restaurants$quality_rating)
group_summary <- do.call(rbind, lapply(names(price_by_quality), function(group) {
  prices <- price_by_quality[[group]]
  q <- quantile(prices, c(.25,.75), type = 6, names = FALSE)
  data.frame(Quality = group, Restaurants = length(prices),
             Median_USD = median(prices), Q1_USD = q[1], Q3_USD = q[2],
```

```
            IQR_USD = q[2]-q[1])
}))
kable(group_summary, digits = 2)
```

Quality	Restaurants	Median_USD	Q1_USD	Q3_USD	IQR_USD
Good	84	19.5	15.0	25	10.0
Very Good	150	25.0	20.0	32	12.0
Excellent	66	34.5	29.5	41	11.5

```
# ggplot(): starts a plot from data; additional layers are joined with +.
# aes(): maps variables to visual properties such as x, y, fill, color, or group.
# labs(): sets axis titles, legend titles, plot titles, and captions.
# geom_boxplot() compares the price distributions within quality categories.
# This overview uses ggplot2 defaults; Lesson 3 constructs explicit type-6 boxplots.
ggplot(restaurants, aes(quality_rating, meal_price_usd)) +
  geom_boxplot(fill = "#9DC7DE", colour = "#303030", width = .55) +
  labs(title = "Compare price distributions, not just averages",
       x = "Quality rating", y = "Meal price (USD)")
```

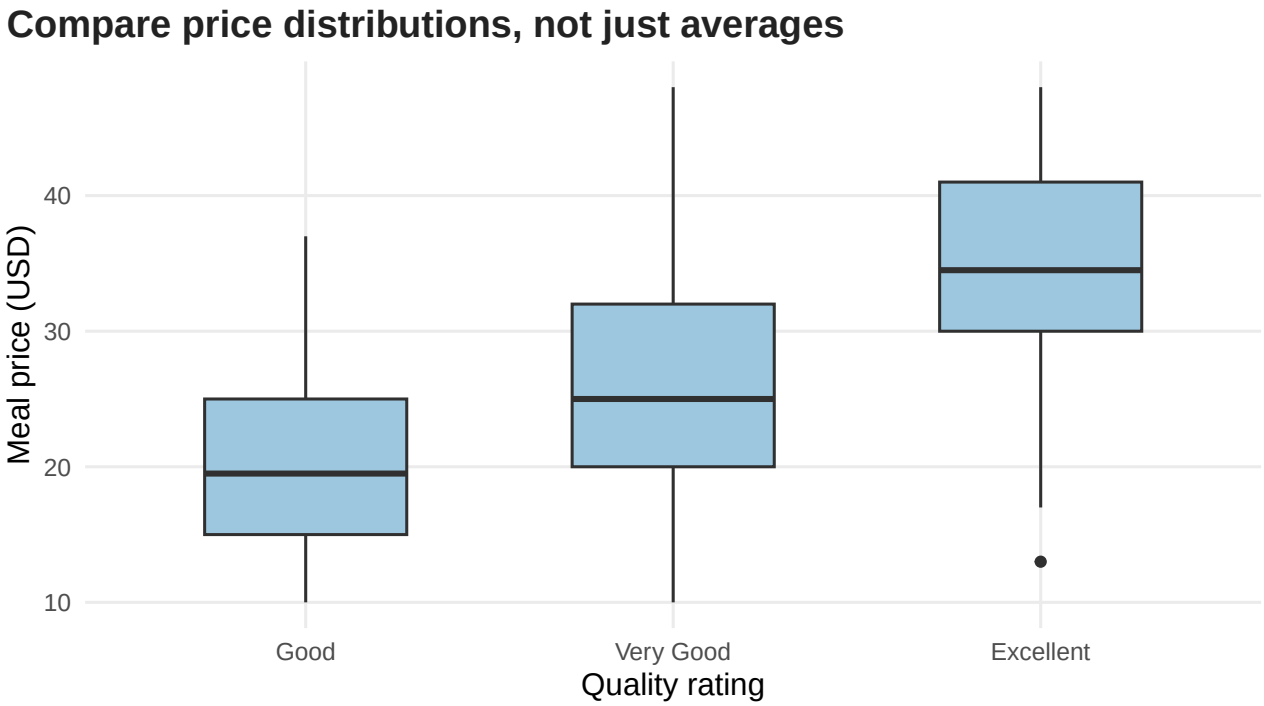


Figure 10: Original R boxplots from the checked Restaurant data. Dots outside whiskers are potential outliers to investigate. The numerical quartiles above use the book's type-6 convention; ggplot2 uses its own standard boxplot calculation, so small differences in hinges can occur.

Interpretation task: compare the group medians, the amount of overlap, and the variability. Describe an association, then name one reason why these observational data do not show that a higher rating causes a higher price.

For **two numerical variables**, start with a scatterplot. Correlation can then summarize a linear association. It does not establish causation, and a near-zero Pearson correlation does not rule out a nonlinear relationship.

An electronics retailer wants to examine whether weeks with more advertising commercials tend to have higher sales. We compare ten weeks of advertising counts and sales, measured in hundreds of US dollars. This describes an association; it does not show that advertising caused the sales differences.

يريد متجر إلكترونيات أن يفحص ما إذا كانت الأسابيع التي تُعرض فيها إعلانات أكثر تميل إلى تحقيق مبيعات أعلى. نقارن بيانات عشرة أسابيع تشمل عدد الإعلانات والمبيعات، المقيسة بمئات الدولارات الأمريكية. يصف هذا التحليل ارتباطًا بين المتغيّرين، ولا يثبت أن الإعلانات سبّبت الفروق في المبيعات.

```
# library() loads ggplot2 for charts.
# read.csv() loads the checked Electronics CSV, matched to the Stereo workbook.
# Keep each week's pair together; sales_hundreds_usd is measured in hundreds of USD.
library(ggplot2)
electronics <- read.csv("data/module02/electronics_sales.csv")
```

```
# ggplot(): starts a plot from data; additional layers are joined with +.
# scale_x_continuous(): controls numeric x-axis breaks, labels, limits, and spacing.
# geom_point() draws one paired commercials-sales observation per week.
# aes() maps the two numerical variables; labs() states sales in hundreds of USD.
ggplot(electronics, aes(commercials, sales_hundreds_usd)) +
  geom_point(size = 3, colour = "#2878A5") +
  scale_x_continuous(breaks = 1:5) +
  labs(title = "More commercials tend to accompany higher sales",
       x = "Number of commercials", y = "Weekly sales (hundreds of USD)")
```

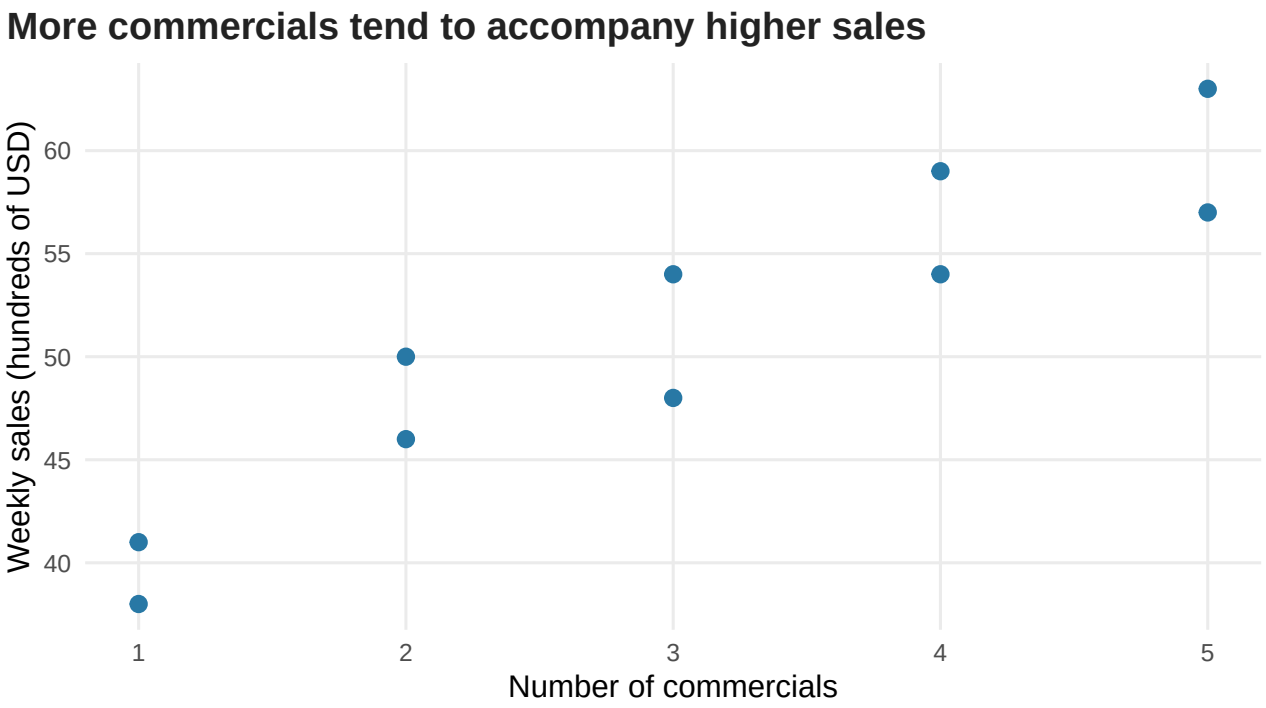


Figure 11: Original R scatterplot. The ten Electronics observations in book Table 2.14 (printed p.76) match the local Stereo workbook; sales are in hundreds of USD.

Choose a tool: a short checklist

1. **Define:** what does one row represent? What are the units and recording precision?
2. **Classify:** categorical or numerical? If numerical, discrete or continuous? What is the measurement level?
3. **Ask:** do we want frequencies, a distribution, a group comparison, a relationship, or a time pattern?
4. **Summarize:** choose an appropriate table, chart, and numerical measure where meaningful.
5. **Interpret:** explain the finding, its context, and its limitations.

Do not calculate an average just because a column contains numbers. Do not choose a chart just because the software offers it.

Open the student exercises or return to the Module 2 overview.

Data for this lesson

Download the CSV datasets and data guide. The restaurant, audit, and electronics examples above read these CSVs directly. Soft-drink data are used in Lesson 2: Frequency tables and charts.

References and provenance

- Anderson, D. R., Sweeney, D. J., Williams, T. A., Camm, J. D., Cochran, J. J., Fry, M. J., and Ohlmann, J. W. (2020). Essentials of Modern Business Statistics with Microsoft Excel, 8th edition. Cengage Learning. Definitions and organization: §1.2, pp.5–8; §5.1, pp.219–220; descriptive tools: Chapter 2, pp.35–100, and Chapter 3, pp.103–169. Wording, teaching questions, and unlabelled illustrations in this lesson are original course material.
- The Audit data match all 20 values in book Table 2.4 (p.47). The Restaurant data match the first 10 printed rows in Table 2.9 (p.65) and all 12 crosstab cells in Table 2.10 (p.66); the remaining individual rows are not printed for comparison. The Electronics/Stereo data match all ten rows in Table 2.14 (p.76).
- These are historical teaching datasets. Their role is to teach statistical reasoning, not describe the current restaurant or advertising market. The book figure and original R charts are identified separately in their captions.

Module 2: Data Types and Toolbox — Practice

Lesson 1 | Classify, choose, summarize, interpret

Table of contents

Before you begin 1
1. Classify the variables 1
2. Explain the mistakes 2
3. Choose the toolbox 2
4. Summarize restaurant quality 2
5. Summarize audit durations 2
What to submit 3

Before you begin

Read Lesson 1. Work individually on Questions 1–3, then use R for Questions 4–5. Give reasons, units, and interpretations alongside your answers.

1. Classify the variables

For each variable, identify **categorical or numerical**, its **measurement level**, and, if numerical, **discrete or continuous**. Explain your decision. Classify the underlying quantity; mention separately if recording precision changes what is stored.

Variable	Example or definition
Restaurant identifier	101, 102, 103: labels assigned to restaurants
Payment method	Cash, card, e-wallet
Quality rating	Good, Very Good, Excellent
Number of orders	Completed orders during one hour
Waiting duration	Time from joining a queue to receiving service, recorded to the nearest minute
Temperature	Air temperature in degrees Celsius
Calendar year	Year a restaurant opened, recorded as an integer
Distance traveled	Delivery distance in kilometers

2. Explain the mistakes

1. An analyst codes quality as Good = 1, Very Good = 2, Excellent = 3 and says: “Excellent is three times as good as Good.” Explain the problem.
2. An analyst says: “The audit times have no decimal places, so elapsed audit duration must be discrete.” Explain the distinction between the quantity and its recorded values.
3. A student says: “20°C is twice as hot as 10°C.” Why does the Celsius scale not support that conclusion?

3. Choose the toolbox

Choose a suitable **table**, **chart**, and, where appropriate, **numerical measure** for each question. Explain why each choice fits the variables.

1. Which quality category occurs most often among the 300 restaurants?
2. What is a typical audit duration, and how much do durations vary?
3. How do meal-price distributions differ across quality categories?
4. In the electronics data, how are weekly advertising counts associated with sales?

Do not infer causation from an observed association.

4. Summarize restaurant quality

Download the restaurant data, or open the course R project and use the path below.

```
# read.csv(): reads a comma-separated file into a data frame; its first row supplies
 ↪  column names by default.
# c(): combines values into a vector, converting them to a common type when needed.
# factor() gives categories explicit levels and an optional order.
# The level order controls grouping and display; ordered=TRUE records ordinal category
 ↪  order.
restaurants <- read.csv("data/module02/restaurant_ratings.csv")
restaurants$quality_rating <- factor(restaurants$quality_rating,
  levels = c("Good", "Very Good", "Excellent"), ordered = TRUE)
```

1. Make a frequency table with counts and percentages. Check both totals.
2. Draw an ordered bar chart with a clear title and labeled axes.
3. State the modal category and its percentage.
4. Explain why a histogram of the codes 1, 2, 3 would hide the categorical meaning.

5. Summarize audit durations

Download the audit data, or use the course project path.

```
# names(): reads or assigns names to vector elements or table columns.
# read.csv() imports a CSV into a data frame with named columns.
# The relative path starts at the course project folder; the source CSV is not modified.
audit <- read.csv("data/module02/audit_times.csv")
names(audit) # Identify the duration column before calculating summaries.
```

1. Construct a grouped frequency table using intervals **10–<15, 15–<20, 20–<25, 25–<30, 30–<35 days**. Check that every audit is counted once.
2. Draw a histogram using the same boundaries.
3. Calculate the mean, median, sample standard deviation, minimum, maximum, range, quartiles, and IQR. Use `quantile(x, c(.25, .75), type = 6)` for the book’s quartile convention.

4. Explain the mean and sample standard deviation in a sentence using days. Describe the middle half using the quartiles.
5. Compare the mean and median. What feature of the observed distribution helps explain their difference? Avoid claiming a population result from these 20 observations alone.

What to submit

A short Quarto report containing your classifications, reasons, tables, charts, and interpretations. Show enough R code to reproduce your calculations. Use the shared course style: pale lavender title panels, dark text, and restrained Petra accents. Choose readable analytical colors for your charts.

Module 2: Frequency Tables and Charts

Lesson 2 | Turn observations into a clear description

Table of contents

What we will learn . . . 1
Business case: which soft drinks were purchased? . . . 1
From observations to a frequency table . . . 2
Bar charts: compare categories . . . 3
Business case: how long do audits take? . . . 5
Group numerical values into classes . . . 6
Histograms: reveal the numerical distribution . . . 7
Cumulative frequencies: answer a threshold question . . . 15
Summary: choose and explain the display . . . 16
Data and references . . . 17

Reduced teaching slides (PDF) · Module overview

This extended lesson provides the full explanation and reproducible examples behind the slides.

What we will learn

Lesson 1 introduced the toolbox. Here we use its first two tools in detail: **tables organize the observations; charts help us see their pattern**.

By the end of this lesson, you should be able to:

- Construct frequency, relative-frequency, and percentage tables.
- Choose a useful order for categorical bars.
- Explain when a pie chart can represent a whole.
- Define numerical classes without gaps or overlaps.
- Read a histogram for shape, skewness, modality, center, spread, and unusual features.
- Construct and interpret a histogram and cumulative frequency table.
- Write a short business interpretation with an appropriate limit on the conclusion.

Reading: main 8th-edition book, §2.1, printed pp.37–42, and §2.2, pp.47–55. Our wording and R analyses are course material based on these textbook examples.

Business case: which soft drinks were purchased?

A manager wants a clear description of the brands selected in **50 soft-drink purchases**. Which brand was purchased most often, and what proportion of purchases did each brand represent? The book records purchases of five brands; one observation is **one purchase**, not necessarily one distinct customer.

يريد مدير أن يحصل على وصف واضح للعلامات التجارية المختارة في **خمسين عملية شراء لمشروبات غازية**. ما العلامة الأكثر شراءً، وما نسبة عمليات الشراء التي تمثلها كل علامة؟ يتضمن مثال الكتاب خمس علامات تجارية، وتمثل كل مشاهدة **عملية شراء واحدة**، وليس بالضرورة عميلًا مختلفًا.

Download the CSV. Use `brand_book_label` for the book's category names. The CSV also preserves `brand_original`: Coke Classic is mapped to Coca-Cola and Pepsi-Cola to Pepsi. This is a documented label alignment, not a change to the counts.

```
# library() loads ggplot2 for charts and knitr for table formatting.
# read.csv() loads purchases; factor()/c() retain the documented textbook labels.
# levels fixes category order; head() previews the first six recorded purchases.
library(ggplot2)
library(knitr)
soft <- read.csv("data/module02/soft_drinks.csv")
brand_levels <- c("Coca-Cola", "Diet Coke", "Dr. Pepper", "Pepsi", "Sprite")
soft$brand <- factor(soft$brand_book_label, levels = brand_levels)
head(soft["brand_book_label"])
```

```
  brand_book_label
1        Coca-Cola
2        Diet Coke
3            Pepsi
4        Diet Coke
5        Coca-Cola
6        Coca-Cola
```

From observations to a frequency table

A **frequency** is the number of observations assigned to a category or numerical class. A **frequency distribution** lists those categories or classes together with their counts.

For one categorical variable, each observation must belong to exactly one category. Check that categories are consistently named, missing values are handled explicitly, and no record is counted twice. An omitted category with zero observations can still be relevant to the question.

For a dataset of size n and a category frequency f:

$$\text{Relative frequency} = \frac{f}{n}, \qquad \text{Percentage} = 100\frac{f}{n}.$$

A relative frequency is a proportion, such as 0.38. Its percentage equivalent is 38%, not 0.38%.

```
# table()/as.data.frame() count purchases; names()/c() label the table columns.
# nrow() supplies the 50-purchase denominator; multiply proportions by 100 for percent.
brand_table <- as.data.frame(table(soft$brand))
names(brand_table) <- c("Brand", "Frequency")
brand_table$Relative_frequency <- brand_table$Frequency / nrow(soft)
brand_table$Percent <- 100 * brand_table$Relative_frequency
```

```
# names(): reads or assigns names to vector elements or table columns.
# c(): combines values into a vector, converting them to a common type when needed.
# stopifnot(): stops execution if any supplied condition is FALSE; successful checks
 ↪  produce no output.
# identical(): requires exact equality of values, types, and attributes.
# as.integer(): converts to integers by truncating numeric fractional parts; it does not
 ↪  round to nearest.
# sum(): adds vector entries; on logical values it counts TRUE entries.
# abs(): returns absolute values; a small tolerance in an equality check permits
 ↪  floating-point rounding.
# kable(): formats a data frame or matrix as a readable report table; digits sets
 ↪  displayed precision.
```

```r
# table() counts brand occurrences; as.data.frame() turns the counts into columns.
# Divide by nrow(soft) for proportions and multiply by 100 for percentages.
brand_table <- as.data.frame(table(soft$brand))
names(brand_table) <- c("Brand", "Frequency")
brand_table$Relative_frequency <- brand_table$Frequency / nrow(soft)
brand_table$Percent <- 100 * brand_table$Relative_frequency
stopifnot(identical(as.integer(brand_table$Frequency), c(19L, 8L, 5L, 13L, 5L)),
          sum(brand_table$Frequency) == 50,
          abs(sum(brand_table$Relative_frequency) - 1) < 1e-12)
kable(brand_table, digits = c(0, 0, 2, 0),
      col.names = c("Brand", "Frequency", "Relative frequency", "Percent (%)"))
```

Brand	Frequency	Relative frequency	Percent (%)
Coca-Cola	19	0.38	38
Diet Coke	8	0.16	16
Dr. Pepper	5	0.10	10
Pepsi	13	0.26	26
Sprite	5	0.10	10

The count total is **50**, the relative-frequency total is **1**, and the percentage total is **100%**. Display rounding can sometimes make percentages add to slightly more or less than 100%; calculate from the unrounded values.

Interpretation: Coca-Cola accounts for **19 of 50 purchases (38%)**, the largest observed share. This describes the sample of purchases; it does not establish a market share for all consumers or measure profit.

Check: Pepsi represents 13 purchases. Calculate its relative frequency and percentage before expanding the code.

Bar charts: compare categories

A **bar chart** places categories on one axis and their counts or percentages on the other. The bars are separated because categories are distinct. The length of each bar communicates the quantity, so the quantitative axis should start at zero.

Nominal brands have no inherent ranking. Alphabetical order is reasonable for lookup; descending frequency makes a popularity comparison easier. For **ordinal** categories, retain their meaningful order rather than sorting them by frequency.

```r
# order() ranks descending counts; as.character() breaks ties alphabetically.
# factor()/rev() set top-to-bottom bar order from the character category labels.
ranked <- brand_table[order(-brand_table$Frequency,
  as.character(brand_table$Brand)), ]
ranked$Brand <- factor(ranked$Brand, levels = rev(as.character(ranked$Brand)))
```

```r
# ggplot()/aes() map the ranked table; geom_col() uses its already computed counts.
# geom_text() adds counts; hjust positions them and width controls bar thickness.
# scale_x_continuous()/c()/seq() set count limits/ticks; labs() supplies labels.
ggplot(ranked, aes(Frequency, Brand)) +
  geom_col(fill = "#2878A5", width = .65) +
  geom_text(aes(label = Frequency), hjust = -.35) +
  scale_x_continuous(limits = c(0, 22), breaks = seq(0, 20, 5)) +
  labs(title = "Coca-Cola has the largest observed purchase count",
       x = "Purchases (out of 50)", y = NULL)
```

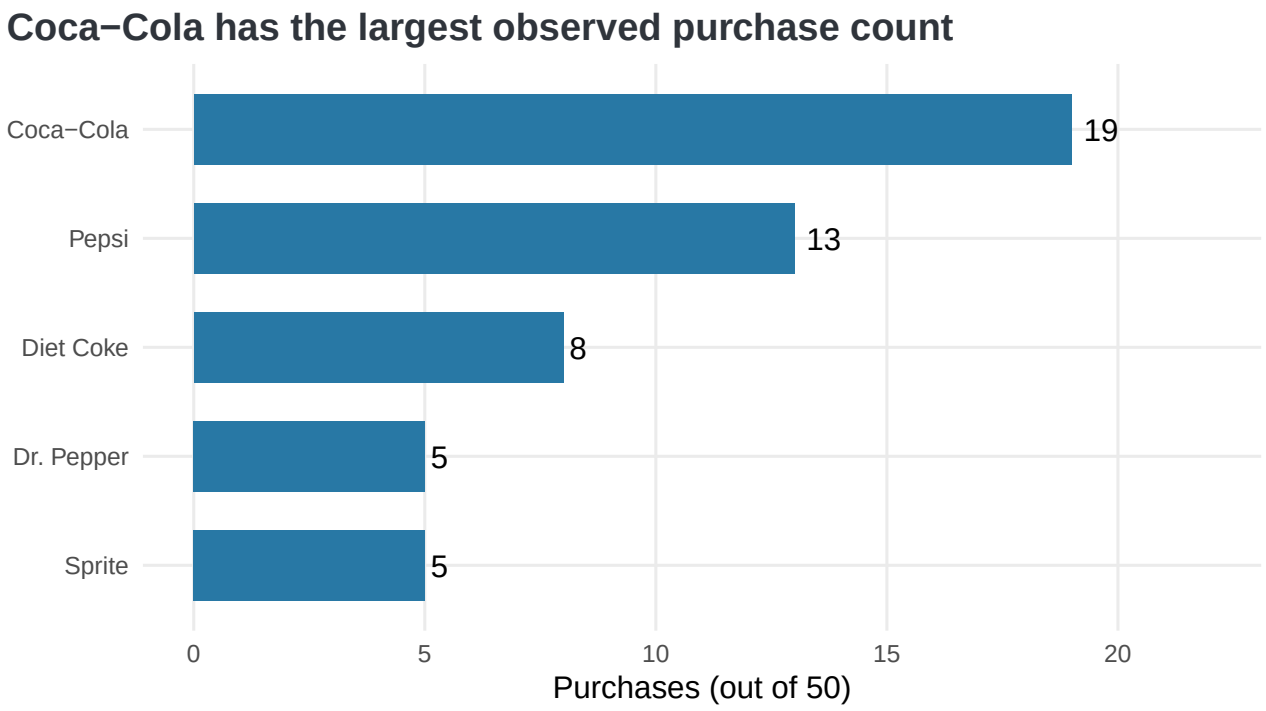


Figure 1: Original R chart from the checked SoftDrink CSV. Counts match book Table 2.2, printed p.38.

The same counts can be shown as percentages. Do not treat counts and percentages from this one sample as separate findings: one is a rescaling of the other.

Pie charts: show shares of one whole

A **pie chart** divides a circle into slices whose angles represent proportions of a whole. A 38% share has an angle of $0.38 \times 360^\circ = 136.8^\circ$.

Use it only when categories describe nonoverlapping parts of one clearly defined total. With many categories or similar shares, bars are easier to compare. Use a flat display and direct labels; 3D perspective can distort apparent slice sizes.

```
# paste0(): joins text with no separator; collapse can combine a vector into one string.
# ggplot(): starts a plot from data; additional layers are joined with +.
# aes(): maps variables to visual properties such as x, y, fill, color, or group.
# geom_col(): draws bars at supplied heights; it does not count records.
# geom_text(): draws labels from mapped data; position and hjust/vjust control placement.
# scale_fill_manual(): assigns explicit fill colors to categories; named values preserve
 ↪  category-color mappings.
# c(): combines values into a vector, converting them to a common type when needed.
# theme_void(): removes axes and most non-data decoration.
# theme(): customizes non-data plot elements such as legends, grids, and axis text.
# coord_polar(theta = "y") turns the stacked percentage bar into a circle.
# position_stack(vjust = .5) centers each label within its corresponding slice.
pie_data <- brand_table
pie_data$label <- paste0(pie_data$Brand, "\n", pie_data$Percent, "%")
ggplot(pie_data, aes(x = "", y = Percent, fill = Brand)) +
  geom_col(width = 1, colour = "white") +
  coord_polar(theta = "y") +
  geom_text(aes(label = label), position = position_stack(vjust = .5), size = 3.5) +
  scale_fill_manual(values = c("#99C1D9", "#D9E8F0", "#AAD5B7", "#F1DE99", "#E9B5B9")) +
  theme_void() + theme(legend.position = "none")
```

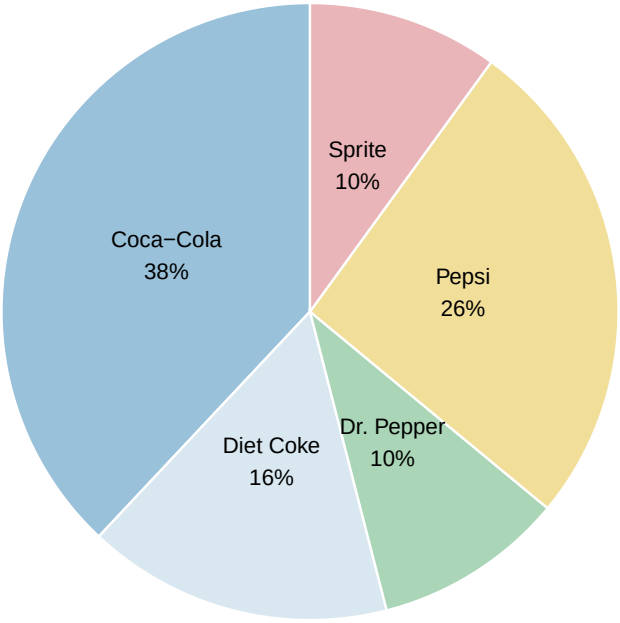


Figure 2: Original R chart: the same 50 purchases shown as percentages of one whole. Colors distinguish categories; they do not encode a ranking.

Discussion: Which display makes the tie between Sprite and Dr. Pepper easier to assess? Which is easier to read when brand names are long?

Percentages need a denominator

Always name the whole: here it is **50 recorded purchases**. If comparing datasets with different sample sizes, percentages can be more useful than raw counts, but differences in sampling and population coverage still matter.

Do not cumulatively add nominal categories as though they form a natural numerical progression. Adding the three most frequent brands can answer a stated “top three” question; the order is an analytical choice, not a property of the brand variable. Here the three largest shares total **80%**.

Business case: how long do audits take?

An accounting firm wants to describe the completion times of **20 year-end audits**, recorded in days. Which duration ranges occur most often, and how many audits finish before a chosen threshold? One observation is **one audit**. These data support a description of recorded durations, not a promise about a future client’s completion date.

يريد مكتب محاسبة وصف المدد اللازمة لإنجاز **عشرين عملية تدقيق لنهاية السنة**، والمسجّلة بالأيام. ما فئات المدة الأكثر تكرارًا، وكم عملية تدقيق انتهت قبل حد زمني محدد؟ تمثل كل مشاهدة **عملية تدقيق واحدة**. تصف هذه البيانات المدد المسجّلة، ولا تقدّم وعدًا بموعد إنجاز عملية تدقيق لعميل في المستقبل.

Download the Audit CSV. The variable `audit_days` contains all 20 values printed in book Table 2.4, p.47.

```r
# library() loads ggplot2 for charts and knitr for table formatting.
# read.csv() loads the checked Audit CSV; $ selects its audit_days column.
# x is the vector of 20 durations in days, not class midpoints.
library(ggplot2)
library(knitr)
audit <- read.csv("data/module02/audit_times.csv")
x <- audit$audit_days
```

```r
# c(): combines values into a vector, converting them to a common type when needed.
# seq_len(nrow(audit)) creates one row number for each recorded audit.
# data.frame() pairs row numbers with durations; kable() prints the table.
kable(data.frame(Audit = seq_len(nrow(audit)), Days = audit$audit_days),
      col.names = c("Audit (row number)", "Duration (days)"))
```

Audit (row number)	Duration (days)
1	12
2	15
3	20
4	22
5	14
6	14
7	15
8	27
9	21
10	18
11	19
12	18
13	22
14	33
15	16
16	18
17	17
18	23
19	28
20	13

Group numerical values into classes

Grouping replaces individual values with ranges. It makes patterns easier to see, but loses detail: a grouped table does not tell us every exact value inside a range.

Choose a number of classes, a convenient width, and boundaries that cover all observations. For Audit, the smallest value is 12 and the largest is 33 days. With five classes, an initial width estimate is $(33-12)/5 = 4.2$ days. Following the book, use a convenient **5-day width**, starting at 10 days.

Our intervals are **[10,15), [15,20), [20,25), [25,30), [30,35)**. A square bracket includes the boundary; a round bracket excludes it. Thus, 15 belongs to [15,20), while 20 belongs to [20,25). For these whole-day records, these groups reproduce the book's 10–14, 15–19, 20–24, 25–29, and 30–34 classes.

> **Class rules**
>
> Every value must fall into exactly one class. Avoid gaps, overlap, or an uncovered maximum. Retain empty classes when they are part of the chosen numerical scale. There is no universally best number of classes: check that your choice reveals rather than hides the distribution.

```r
# seq() supplies 5-day boundaries; cut() assigns each raw duration to a class.
# right = FALSE means [lower, upper); table() counts the resulting classes.
boundaries <- seq(10, 35, 5)
audit$class <- cut(audit$audit_days,
  breaks = boundaries, right = FALSE)
table(audit$class)
```

```
[10,15) [15,20) [20,25) [25,30) [30,35)
      4       8       5       2       1
```

```r
# as.data.frame(): converts an object to a rectangular data frame.
# names(): reads or assigns names to vector elements or table columns.
# c(): combines values into a vector, converting them to a common type when needed.
# nrow(): returns the number of rows in a table or matrix.
# stopifnot(): stops execution if any supplied condition is FALSE; successful checks
 ↪  produce no output.
# anyNA(): returns TRUE if any value is missing; !anyNA(...) requires no missing values.
# sum(): adds vector entries; on logical values it counts TRUE entries.
# identical(): requires exact equality of values, types, and attributes.
# as.integer(): converts to integers by truncating numeric fractional parts; it does not
 ↪  round to nearest.
# kable(): formats a data frame or matrix as a readable report table; digits sets
 ↪  displayed precision.
# seq(10,35,5) sets class boundaries; cut() assigns each audit to an interval.
# right = FALSE gives [lower, upper) classes; table() counts each class.
boundaries <- seq(10, 35, 5)
audit$class <- cut(audit$audit_days, breaks = boundaries, right = FALSE)
audit_table <- as.data.frame(table(audit$class))
names(audit_table) <- c("Days", "Frequency")
audit_table$Relative_frequency <- audit_table$Frequency / nrow(audit)
audit_table$Percent <- 100 * audit_table$Relative_frequency
stopifnot(!anyNA(audit$class), sum(audit_table$Frequency) == 20,
          identical(as.integer(audit_table$Frequency), c(4L, 8L, 5L, 2L, 1L)))
kable(audit_table, digits = c(0, 0, 2, 0),
      col.names = c("Duration class (days)", "Frequency", "Relative frequency", "Percent
       ↪  (%)"))
```

Duration class (days)	Frequency	Relative frequency	Percent (%)
[10,15)	4	0.20	20
[15,20)	8	0.40	40
[20,25)	5	0.25	25
[25,30)	2	0.10	10
[30,35)	1	0.05	5

The **15–<20-day class** is the most frequent: 8 audits, or 40%. This identifies a modal **class**, not an exact modal duration.

Histograms: reveal the numerical distribution

A **histogram** places numerical intervals on the horizontal axis. With equal-width intervals, bar heights can represent counts directly. The rectangles are adjacent because they represent neighboring intervals on a numerical scale.

```r
# ggplot()/aes() map raw audit days; geom_histogram() counts into boundaries.
# closed = "left" agrees with cut(..., right = FALSE), so 20 enters [20,25).
# scale_x_continuous()/scale_y_continuous() set ticks; labs() labels units and counts.
ggplot(audit, aes(audit_days)) +
  geom_histogram(breaks = boundaries, closed = "left",
```

```
                 fill = "#2878A5", colour = "white") +
scale_x_continuous(breaks = boundaries) +
scale_y_continuous(breaks = 0:8) +
labs(title = "Audit durations cluster below 25 days",
     x = "Audit duration (days)", y = "Audits")
```

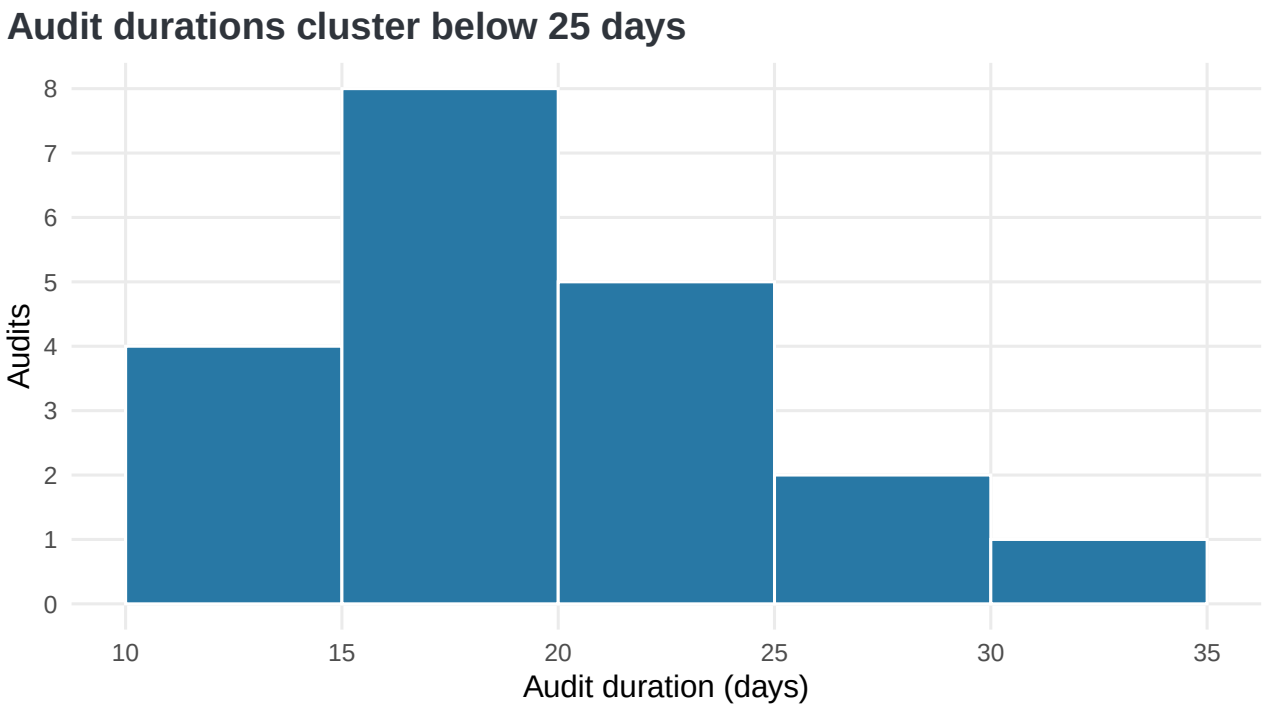


Figure 3: Original R histogram using the same five classes as the table; frequencies match book Table 2.5, printed p.49.

Read the **shape**, the concentration of values, the extent of the spread, and any gaps or isolated observations. Here, most observations lie below 25 days, with fewer audits at longer durations. Investigate the 33-day observation in context rather than deleting it because it is longer.

What does a distribution tell us?

A **distribution** describes how the observations are spread across possible values: which regions contain many observations, which contain few, and where gaps occur. A histogram groups that information into intervals. Its horizontal axis shows the measured values; its vertical axis shows counts, proportions, or densities as labeled. It does not show changes over time unless time itself is the measured variable.

Read a histogram through four questions:

Feature	Question to ask	What the histogram can suggest
Shape	Is the distribution balanced or skewed? How many peaks are apparent?	Symmetry, tail direction, and possible clusters
Center	Where is a typical region?	An approximate central location; exact means and medians require the data

Feature	Question to ask	What the histogram can suggest
Spread	How far do values extend and concentrate around the center?	Narrow or broad variation, interpreted with the horizontal scale
Unusual features	Are there gaps, isolated values, or small separated groups?	Observations or patterns to investigate

Two histograms can look equally wide while using different axis scales. Always read the values and units before comparing their spread.

Skewness: name the direction of the longer tail

A **roughly symmetric distribution** has similar shapes on either side of its center. **Right (positive) skewness** means the longer tail extends toward larger values; **left (negative) skewness** means it extends toward smaller values. Name the direction from the tail, not from the location of the tallest bars.

```
# seq_len(): returns integer positions 1 through n, safely returning an empty vector when
 ↪  n=0.
# c(): combines values into a vector, converting them to a common type when needed.
# data.frame(): builds a table whose named columns have compatible lengths.
# factor(): stores categorical values; levels fixes their permitted order and labels can
 ↪  rename their display.
# rep(): repeats values; times repeats a vector and each repeats individual elements.
# stopifnot(): stops execution if any supplied condition is FALSE; successful checks
 ↪  produce no output.
# all(): returns TRUE only when every supplied logical value is TRUE.
# ggplot(): starts a plot from data; additional layers are joined with +.
# aes(): maps variables to visual properties such as x, y, fill, color, or group.
# geom_histogram(): bins numeric observations; binwidth or breaks controls the intervals
 ↪  and boundary anchors them.
# seq(): creates an equally spaced sequence; by sets spacing and length.out sets the
 ↪  number of values.
# facet_wrap(): splits the plot into panels defined by a categorical formula.
# scale_x_continuous(): controls numeric x-axis breaks, labels, limits, and spacing.
# labs(): sets axis titles, legend titles, plot titles, and captions.
# theme(): customizes non-data plot elements such as legends, grids, and axis text.
# element_text(): sets text appearance; size uses points and angle rotates labels.
# qnorm() and qbeta() generate artificial values at evenly spaced probabilities.
# Reflecting the right-skewed values creates the left-skewed comparison; these are not
 ↪  observed cases.
p <- (seq_len(1200) - .5) / 1200
symmetric_example <- qnorm(p, mean = 50, sd = 12)
right_example <- 100 * qbeta(p, shape1 = 2, shape2 = 6)
shape_labels <- c("Roughly symmetric", "Right-skewed\nlonger right tail",
 ↪  "Left-skewed\nlonger left tail")
shape_examples <- data.frame(
  Value = c(symmetric_example, right_example, 100-right_example),
  Pattern = factor(rep(shape_labels, each = 1200), levels = shape_labels))
stopifnot(all(shape_examples$Value > 0 & shape_examples$Value < 100))
ggplot(shape_examples, aes(Value)) +
  geom_histogram(breaks = seq(0,100,5), closed = "left", fill = "#2878A5", colour =
   ↪  "white") +
  facet_wrap(~Pattern, nrow = 1) +
  scale_x_continuous(breaks = seq(0,100,25)) +
  labs(x = "Illustrative value (arbitrary units)", y = "Count") +
  theme(strip.text = element_text(size = 11))
```

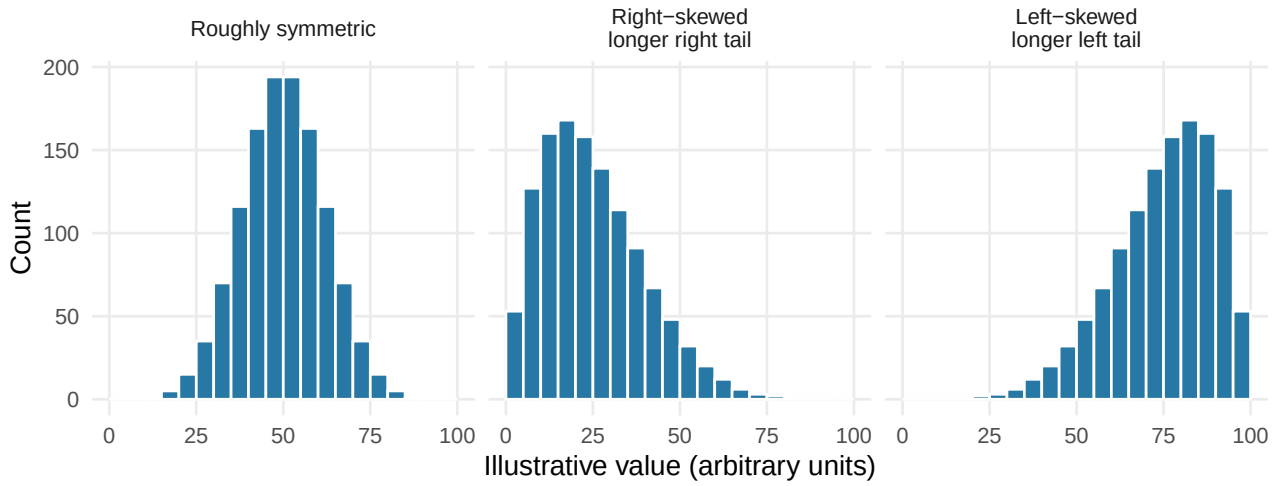


Figure 4: Original illustrations constructed from model quantiles, with 1,200 artificial values per panel. These are teaching patterns, not observed students or textbook records. Bin widths and axes are shared.

For example, a few unusually long service durations can produce a right tail; many high scores with a few low scores can produce a left tail. These are possible interpretations, not explanations established by the shape alone. In many familiar one-peaked skewed distributions, the mean is pulled toward the long tail; the plot alone does not prove a universal ordering of mean and median. A symmetric histogram does not by itself establish a normal distribution.

Modality: count prominent peaks, not every uneven bar

A **unimodal** distribution has one prominent peak; **bimodal** has two; **multimodal** has several. Here, a mode is a prominent region of the distribution, not necessarily the most frequent exact recorded value.

```
# length(): counts elements in a vector or list.
# seq_len(): returns integer positions 1 through n, safely returning an empty vector when
 ↪  n=0.
# unlist(): flattens a list into a vector, converting entries to a common type as needed.
# qnorm(): returns a normal lower-tail quantile; sd is standard deviation, not variance.
# c(): combines values into a vector, converting them to a common type when needed.
# data.frame(): builds a table whose named columns have compatible lengths.
# make_components(): is a locally defined helper building explicitly conceptual
 ↪  distribution examples, not observed data.
# factor(): stores categorical values; levels fixes their permitted order and labels can
 ↪  rename their display.
# rep(): repeats values; times repeats a vector and each repeats individual elements.
# stopifnot(): stops execution if any supplied condition is FALSE; successful checks
 ↪  produce no output.
# all(): returns TRUE only when every supplied logical value is TRUE.
# ggplot(): starts a plot from data; additional layers are joined with +.
# aes(): maps variables to visual properties such as x, y, fill, color, or group.
# geom_histogram(): bins numeric observations; binwidth or breaks controls the intervals
 ↪  and boundary anchors them.
# seq(): creates an equally spaced sequence; by sets spacing and length.out sets the
 ↪  number of values.
# scale_x_continuous(): controls numeric x-axis breaks, labels, limits, and spacing.
# labs(): sets axis titles, legend titles, plot titles, and captions.
# theme(): customizes non-data plot elements such as legends, grids, and axis text.
# element_text(): sets text appearance; size uses points and angle rotates labels.
# The helper combines artificial normal-component values with chosen centers.
# lapply() builds each component and facet_wrap() separates the modality examples.
```

```
make_components <- function(centers, spread) {
  count <- 1200 / length(centers)
  prob <- (seq_len(count)-.5)/count
  unlist(lapply(centers, function(center) qnorm(prob, center, spread)))
}
mode_labels <- c("Unimodal\none prominent peak", "Bimodal\ntwo prominent peaks",
 ↪  "Multimodal\nseveral prominent peaks")
mode_examples <- data.frame(
  Value = c(symmetric_example, make_components(c(30,70),5),
   ↪  make_components(c(20,50,80),3.5)),
  Pattern = factor(rep(mode_labels, each = 1200), levels = mode_labels))
stopifnot(all(mode_examples$Value > 0 & mode_examples$Value < 100))
ggplot(mode_examples, aes(Value)) +
  geom_histogram(breaks = seq(0,100,4), closed = "left", fill = "#2878A5", colour =
   ↪  "white") +
  facet_wrap(~Pattern, nrow = 1) +
  scale_x_continuous(breaks = seq(0,100,25)) +
  labs(x = "Illustrative value (arbitrary units)", y = "Count") +
  theme(strip.text = element_text(size = 11))
```

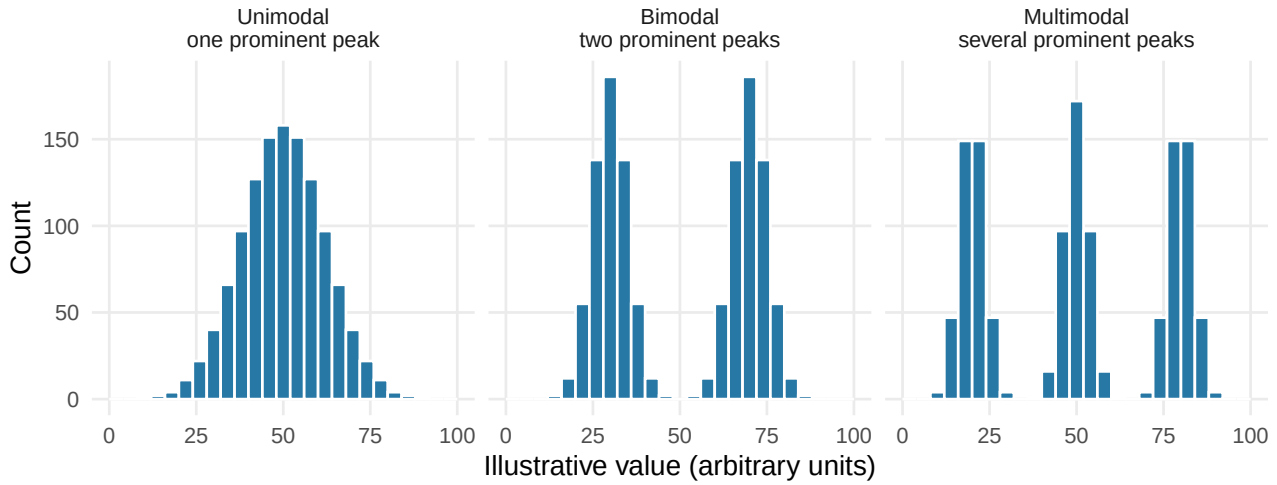


Figure 5: Original illustrations, each containing 1,200 artificial values. The multi-peak patterns combine constructed groups; such a mechanism cannot be inferred from an observed histogram alone.

Several peaks can suggest mixed groups or different operating conditions worth investigating. They do not prove that those groups exist. Sampling variation, rounding, bin width, and bin starting points can produce or obscure apparent peaks. Small fluctuations between adjacent bars are not automatically separate modes.

Additional comparison: recorded student ages and heights

The local teaching workbook contains **8,239 records** with age and height columns. We exported only those two measurements, without names or identifiers. Its population, collection method, and measurement-unit metadata are not documented locally, so the axes retain **recorded units**. This is a supplementary shape-reading example, not a verified university survey or a textbook dataset.

Download the two-column CSV · Read the source notes

```
# stopifnot(): stops execution if any supplied condition is FALSE; successful checks
 ↪  produce no output.
# nrow(): returns the number of rows in a table or matrix.
```

```
# anyNA(): returns TRUE if any value is missing; !anyNA(...) requires no missing values.
# identical(): requires exact equality of values, types, and attributes.
# names(): reads or assigns names to vector elements or table columns.
# c(): combines values into a vector, converting them to a common type when needed.
# rbind(): combines compatible vectors or tables by adding rows.
# data.frame(): builds a table whose named columns have compatible lengths.
# factor(): stores categorical values; levels fixes their permitted order and labels can
 ↪  rename their display.
# do.call(): calls a function using a list of arguments, e.g. rbind to combine a list of
 ↪  tables.
# lapply(): applies a function separately to each element and returns a list.
# levels(): reads or replaces the category labels and order of a factor.
# startsWith(): checks whether each text value begins with a specified prefix.
# seq(): creates an equally spaced sequence; by sets spacing and length.out sets the
 ↪  number of values.
# hist(): bins numeric values; breaks controls intervals and plot=FALSE returns counts
 ↪  without drawing.
# sum(): adds vector entries; on logical values it counts TRUE entries.
# head(): shows the first n entries or rows, six by default.
# tail(): shows the last n entries or rows, six by default.
# ggplot(): starts a plot from data; additional layers are joined with +.
# aes(): maps variables to visual properties such as x, y, fill, color, or group.
# geom_rect(): draws rectangles using xmin, xmax, ymin, and ymax.
# facet_wrap(): splits the plot into panels defined by a categorical formula.
# labs(): sets axis titles, legend titles, plot titles, and captions.
# theme(): customizes non-data plot elements such as legends, grids, and axis text.
# element_text(): sets text appearance; size uses points and angle rotates labels.
# read.csv() loads the two exported measurements; stopifnot() checks their structure.
# The panels use different bin widths and recorded units, so compare shapes rather than
 ↪  bar heights.
students_hist <- read.csv("data/module02/student_age_height.csv")
stopifnot(nrow(students_hist)==8239, !anyNA(students_hist),
          identical(names(students_hist),c("age","height")))
student_values <- rbind(
  data.frame(Variable="Age (recorded units)", Value=students_hist$age),
  data.frame(Variable="Height (recorded units)", Value=students_hist$height))
student_values$Variable <- factor(student_values$Variable,
  levels=c("Age (recorded units)","Height (recorded units)"))
student_bins <- do.call(rbind, lapply(levels(student_values$Variable), function(label) {
  vals <- student_values$Value[student_values$Variable==label]
  breaks <- if (startsWith(label,"Age")) seq(18,66,2) else seq(132,210,3)
  h <- hist(vals, breaks=breaks, right=FALSE, plot=FALSE)
  stopifnot(sum(h$counts)==8239)
  data.frame(Variable=label, Left=head(h$breaks,-1), Right=tail(h$breaks,-1),
   ↪  Count=h$counts)
}))
student_bins$Variable <- factor(student_bins$Variable,
 ↪  levels=levels(student_values$Variable))
ggplot(student_bins, aes(xmin=Left,xmax=Right,ymin=0,ymax=Count)) +
  geom_rect(fill="#2878A5",colour="white") +
  facet_wrap(~Variable,scales="free",nrow=1) +
  labs(x="Recorded measurement value",y="Records") +
  theme(strip.text=element_text(size=12))
```

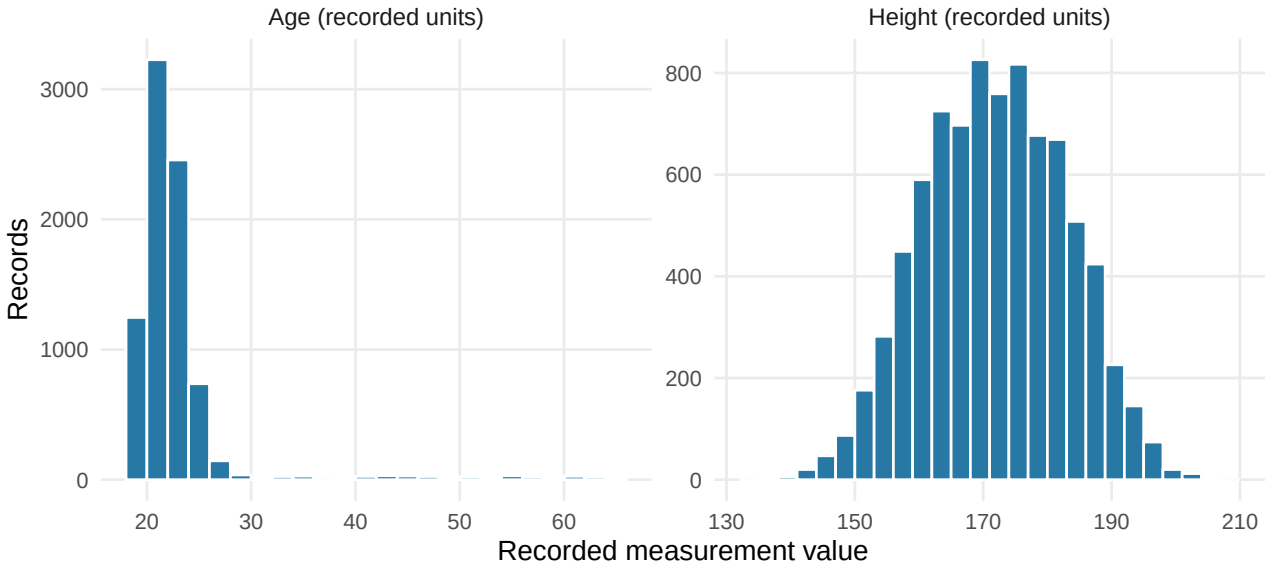


Figure 6: Original histograms from the local students worksheet (8,239 values per variable; no missing values in these two columns). Age bins have width 2 recorded units and height bins width 3. Each panel has its own axis scales; different scales and bin widths mean bar heights and visual widths should not be compared directly.

The age values show a long **right tail**: most are near the lower end, while relatively few extend to 64. The heights have a more balanced central concentration. Small bumps do not establish separate populations or exact normality. Higher ages or extreme heights are not errors merely because they are uncommon; source definitions and individual records would need checking.

Unusual observations and the effect of bins

A histogram may reveal isolated bars, gaps, or a small separated group. Call these **potential unusual observations** until you examine the raw values and context. There is no universal histogram-only cutoff for an outlier. Lesson 3 introduces a stated IQR screening rule, which is a separate procedure.

Compare the **same 20 Audit values** with three class widths. The dashed line marks the observed 33-day audit, not an outlier threshold.

```
# list(): stores named or unnamed objects that may have different types and lengths.
# seq(): creates an equally spaced sequence; by sets spacing and length.out sets the
 ↪  number of values.
# do.call(): calls a function using a list of arguments, e.g. rbind to combine a list of
 ↪  tables.
# names(): reads or assigns names to vector elements or table columns.
# stopifnot(): stops execution if any supplied condition is FALSE; successful checks
 ↪  produce no output.
# sum(): adds vector entries; on logical values it counts TRUE entries.
# data.frame(): builds a table whose named columns have compatible lengths.
# head(): shows the first n entries or rows, six by default.
# tail(): shows the last n entries or rows, six by default.
# factor(): stores categorical values; levels fixes their permitted order and labels can
 ↪  rename their display.
# ggplot(): starts a plot from data; additional layers are joined with +.
# aes(): maps variables to visual properties such as x, y, fill, color, or group.
# geom_rect(): draws rectangles using xmin, xmax, ymin, and ymax.
# geom_vline(): adds vertical references at xintercept; linetype selects solid or dashed
 ↪  lines.
# facet_wrap(): splits the plot into panels defined by a categorical formula.
```

```r
# scale_x_continuous(): controls numeric x-axis breaks, labels, limits, and spacing.
# c(): combines values into a vector, converting them to a common type when needed.
# scale_y_continuous(): controls numeric y-axis breaks, labels, limits, and spacing.
# labs(): sets axis titles, legend titles, plot titles, and captions.
# lapply() repeats the grouping for three bin-width specifications without changing audit
 ↪  values.
# hist(..., plot = FALSE) counts left-closed bins without drawing; the panels share a
 ↪  frequency scale.
audit_bin_specs <- list("1-day bins"=seq(10,40,1),"5-day bins"=seq(10,40,5),"10-day
 ↪  bins"=seq(10,40,10))
audit_bin_comparison <- do.call(rbind,lapply(names(audit_bin_specs),function(label){
  h <- hist(audit$audit_days,breaks=audit_bin_specs[[label]],right=FALSE,plot=FALSE)
  stopifnot(sum(h$counts)==20)
  data.frame(Bins=label,Left=head(h$breaks,-1),Right=tail(h$breaks,-1),Count=h$counts)
}))
audit_bin_comparison$Bins <-
 ↪  factor(audit_bin_comparison$Bins,levels=names(audit_bin_specs))
ggplot(audit_bin_comparison,aes(xmin=Left,xmax=Right,ymin=0,ymax=Count)) +
  geom_rect(fill="#2878A5",colour="white") +
  geom_vline(xintercept=33,linetype="dashed",colour="#596775",linewidth=.5) +
  facet_wrap(~Bins,nrow=1) +
  scale_x_continuous(breaks=c(10,20,30,40)) +
  scale_y_continuous(breaks=c(0,3,6,9,12)) +
  labs(x="Audit duration (days)",y="Audits")
```

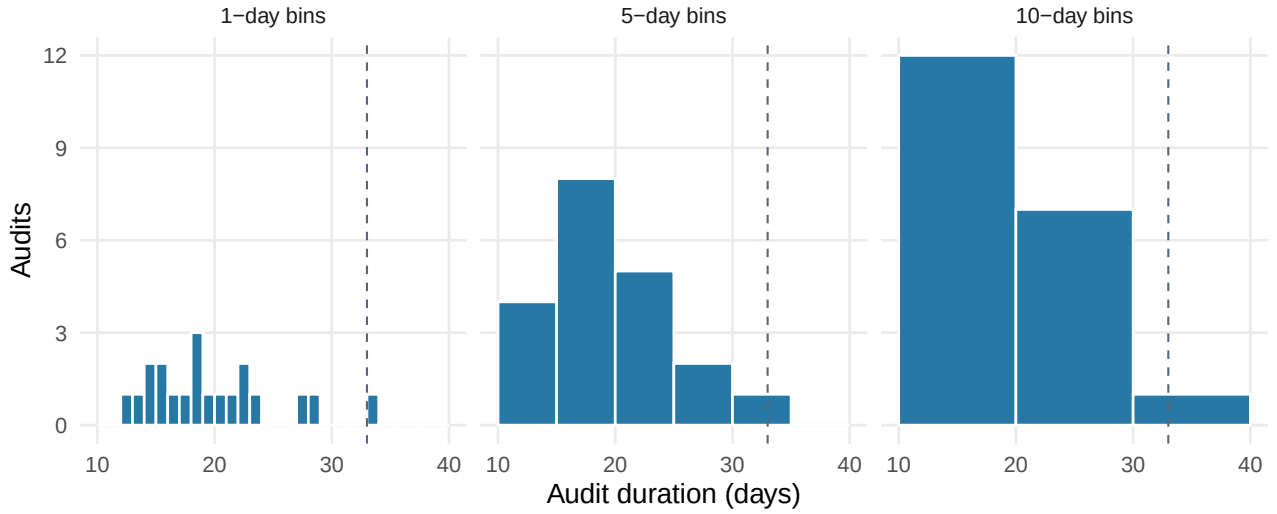


Figure 7: Original R comparison of the unchanged Audit data. Widths of 1, 5, and 10 days change the apparent detail and prominence of gaps. Every panel contains all 20 audits.

The data did not change. The display changed. Check more than one sensible bin width and starting point before making a claim about a small peak, gap, or unusual group. Count heights rise as bins combine observations; a taller bar in a wider-bin panel is not evidence of more audits overall.

Read the Audit histogram in four sentences

- **Shape:** most values are in the lower-duration classes, with a longer tail toward larger durations; the five-day grouping has one prominent peak.
- **Center:** a typical region appears around the high teens. Calculate from the CSV to obtain the exact mean (19.25 days) and median (18 days).
- **Spread:** the recorded durations run from 12 to 33 days. The histogram groups them; standard deviation and IQR quantify other aspects of spread in Lesson 3.

- **Unusual features:** the 33-day observation and the sparsely occupied upper end deserve context, not automatic deletion. Smaller bins and the dot plot show the individual values more clearly.

Keep the individual values in view

The book's dot plot complements the histogram: each dot represents one audit, preserving repeated and individual durations that grouping hides.

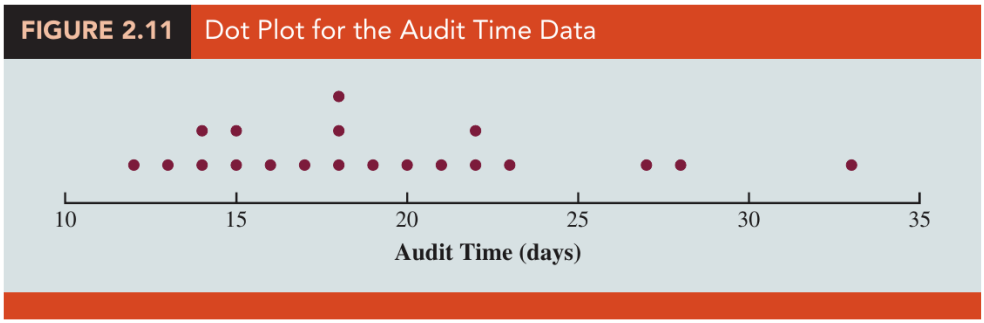


Figure 8: Book Figure 2.11: Audit time dot plot, printed p.52. Anderson et al., Essentials of Modern Business Statistics with Microsoft Excel, 8th ed. Copyright 2020 Cengage Learning.

Bar chart or histogram?

Feature	Bar chart	Histogram
Horizontal meaning	Distinct categories	Numerical intervals
Order	Choose a useful order; retain ordinal order	Keep numerical order
Separation	Gaps distinguish categories	Adjacent class rectangles
What width means	Usually decorative and consistent	Represents interval width
Primary question	How common is each category?	How are numerical values distributed?

If class widths differ, counts as heights can mislead: histogram **area**, rather than height alone, must represent frequency. Density height equals frequency divided by class width (or relative frequency divided by class width). We use equal widths here so count heights are appropriate.

Cumulative frequencies: answer a threshold question

A **cumulative frequency** adds the counts in successive numerical classes. It answers "how many observations fall below this boundary?" The denominator remains 20 audits.

```
# data.frame(): builds a table whose named columns have compatible lengths.
# paste(): joins text using sep between pieces; collapse combines a vector into one
 ↪  string.
# stopifnot(): stops execution if any supplied condition is FALSE; successful checks
 ↪  produce no output.
# identical(): requires exact equality of values, types, and attributes.
# as.integer(): converts to integers by truncating numeric fractional parts; it does not
 ↪  round to nearest.
```

```r
# c(): combines values into a vector, converting them to a common type when needed.
# sum(): adds vector entries; on logical values it counts TRUE entries.
# kable(): formats a data frame or matrix as a readable report table; digits sets
 ↪  displayed precision.
# cumsum() adds frequencies successively to count observations below each upper boundary.
# Divide those running totals by nrow(audit) and multiply by 100 for cumulative
 ↪  percentages.
cumulative <- data.frame(
  Threshold = paste("Less than", boundaries[-1], "days"),
  Audits = cumsum(audit_table$Frequency),
  Percent = 100 * cumsum(audit_table$Frequency) / nrow(audit))
stopifnot(identical(as.integer(cumulative$Audits), c(4L,12L,17L,19L,20L)),
          sum(audit$audit_days < 25) == 17)
kable(cumulative, digits = 0, col.names = c("Duration threshold", "Cumulative audits",
 ↪  "Cumulative percent (%)"))
```

Duration threshold	Cumulative audits	Cumulative percent (%)
Less than 15 days	4	20
Less than 20 days	12	60
Less than 25 days	17	85
Less than 30 days	19	95
Less than 35 days	20	100

Interpretation: 17 of the 20 recorded audits (**85%**) took less than 25 days. The remaining three (**15%**) took at least 25 days.

The grouped table cannot give an exact count below a threshold **inside** a class, such as 18 days. For that question, return to the CSV rather than assume values are evenly spread across the class. Similarly, “less than 20” and “20 or less” have different meanings when a 20-day observation is present.

Summary: choose and explain the display

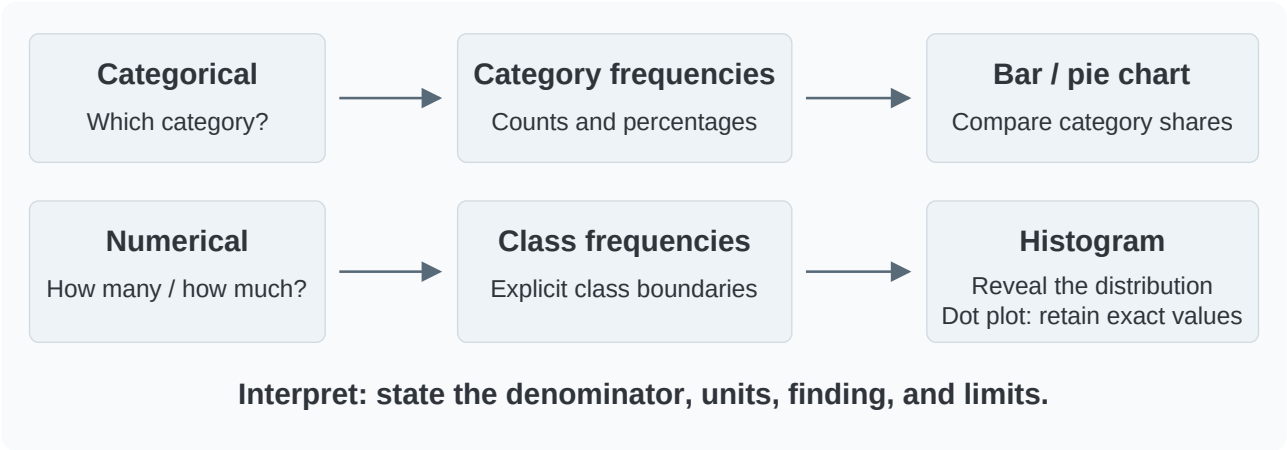


Figure 9: Original course summary: identify the variable, organize frequencies, choose a matching display, and interpret with a denominator and units.

Before handing a chart to a manager, check:

1. **Meaning:** the observation, variable, units, and denominator are clear.
2. **Construction:** categories or classes are suitable; counts reconcile to the data.
3. **Display:** labels, order, baseline, and widths support a fair comparison.

4. **Interpretation:** describe what the sample shows and avoid unsupported forecasts or causal claims.

Complete the Lesson 2 practice. Continue with Lesson 3: Numerical measures, using Audit to connect center and spread to the distribution.

Data and references

- CSV bundle and data guide. All calculations above read CSV files.
- Main source: Anderson et al. (2020), Essentials of Modern Business Statistics with Microsoft Excel, 8th ed., §§2.1–2.2. SoftDrink counts and proportions: Tables 2.2–2.3, p.38; Audit observations: Table 2.4, p.47; grouped frequencies: Tables 2.5–2.6, p.49; dot plot: Figure 2.11, p.52; cumulative distributions: pp.55–56.
- Artificial shape/modality plots are original teaching illustrations. Student age/height plots use a separate local teaching file with undocumented population and unit metadata; see the linked source notes. They are not attributed to the main textbook.
- SoftDrink category totals match after the documented label alignment; row order is not asserted to match the printed table. Audit matches all 20 printed observations. Original R charts are distinguished from the attributed book figure.

Module 2: Frequency Tables and Charts — Practice

Lesson 2 | Construct, check, interpret

Table of contents

Preparation 1
1. Reconstruct the soft-drink summary 1
2. Choose and defend a categorical chart 2
3. Group audit durations 2
4. Investigate grouping choices 2
5. Read the distribution 2
6. Write for a manager 3

Preparation

Read Lesson 2. Use the CSV bundle and data guide. Open the 20261 course project when using the paths below; adjust paths if working from a separate folder.

Submit a short Quarto report with reproducible R code, labeled tables and figures, and interpretations. Use the course style. Answer questions about the observed data before making recommendations.

```
# library(): loads a package so its functions can be used in this session.
# read.csv() imports a CSV into a data frame with named columns.
# The relative path starts at the course project folder; the source CSV is not modified.
library(ggplot2)
soft <- read.csv("data/module02/soft_drinks.csv")
audit <- read.csv("data/module02/audit_times.csv")
```

1. Reconstruct the soft-drink summary

Use `brand_book_label`, not the two brand columns stacked together.

1. State the observational unit and measurement level of the brand variable.
2. Make a table of counts, relative frequencies, and percentages for all five brands.
3. Check that the counts and proportions have the expected totals.
4. Identify the largest share, the tied categories, and the combined share of the three most frequent brands.
5. Explain why the two brand columns do not represent 100 separate purchases.

2. Choose and defend a categorical chart

1. Draw a horizontal bar chart ordered by descending frequency. Include readable labels and a zero baseline.
2. Draw a flat pie chart of the same percentages.
3. Explain which chart you would give a manager who wants to compare the two least frequent brands, and why.
4. Explain why the statement “Coca-Cola has 38% of the national market” is not supported by this dataset.
5. Suppose another sample contains 200 purchases. Explain why comparing raw counts alone could mislead; you do not need to invent or analyze a second dataset.

3. Group audit durations

1. Define five nonoverlapping intervals of width five days, starting at 10 and ending at 35. State which endpoints are included.
2. Make frequency and percentage tables using these intervals.
3. Draw a histogram with the same boundaries. Verify that its bar counts agree with your table.
4. Identify the modal class. Explain why it does not necessarily identify the most frequent exact duration.
5. Make a cumulative table. Report the count and percentage taking less than 25 days, and the count taking at least 25 days.
6. Use the raw CSV to compare the counts for **less than 20 days** and **20 days or less**. Explain the difference.

4. Investigate grouping choices

Use wider intervals **[10,20), [20,30), [30,40)**.

1. Recompute the frequencies and draw a second histogram.
2. Confirm that both histograms contain the same 20 audits.
3. Describe one detail that is less visible with wider bins.
4. Explain why a new set of bins does not mean that the underlying observations changed.
5. A colleague proposes unequal-width bins while keeping counts as heights. Explain why the area of each rectangle matters and what quantity should determine its height.

5. Read the distribution

1. For the Audit histogram, write one sentence for each feature: **shape, center, spread, and unusual features**. Distinguish approximate visual readings from exact CSV calculations.
2. Explain how you decide whether a histogram is right-skewed or left-skewed. Does the direction come from its tallest bar or its longer tail?
3. Compare the artificial unimodal, bimodal, and multimodal illustrations in the lesson. What might multiple peaks suggest? Give two reasons they do not prove that distinct populations exist.
4. Explain why a roughly symmetric histogram does not establish normality and why an isolated bar does not establish a data error.
5. Compare the 1-, 5-, and 10-day Audit histograms. Which features become harder to see in wider bins? Does the 33-day audit disappear from the dataset?
6. Optional: use the student age/height CSV to reproduce the comparison. Describe the age tail and the height concentration. Read the source notes and explain why the charts retain recorded units and do not support claims about all university students.

6. Write for a manager

Write two short paragraphs, each about 60–90 words:

- **Purchases:** summarize the leading brands and recommend a useful next information-gathering step before a purchasing decision. Distinguish an observed purchase share from profit or national demand.
- **Audit durations:** summarize the modal class and the share below 25 days. State what these observations do and do not justify when planning future work.

Do not claim that a descriptive chart establishes causation, guarantees completion times, or proves that an organization is inefficient.

Return to Lesson 2.

Module 2: Numerical Measures

Lesson 3 | Describe center, variability, and position

Table of contents

What we will learn . . . 1
Business case: describe a typical audit and its variability . . . 1
Mean, median, and mode: three meanings of typical . . . 2
Variability: how different are the durations? . . . 6
Percentiles and quartiles: locate values by rank . . . 11
How much does a long observation change the summary? . . . 14
Two additional measures: CV and z-scores . . . 15
Business case: compare restaurant price distributions . . . 16
Summary: match the measure to the question . . . 19
Sources and data . . . 19

Reduced teaching slides (PDF) · Module overview

This extended lesson provides the full explanation and reproducible examples behind the slides.

What we will learn

A table or chart reveals the distribution. Numerical measures describe selected features of that distribution precisely. In Lesson 2, most audits fell below 25 days. We now ask: **What is typical, how much do durations vary, and which values deserve attention?**

By the end, you should be able to:

- Calculate and interpret the mean, median, and mode.
- Discuss what the mean and range reveal, what they hide, and how extreme values affect them.
- Distinguish range, variance, standard deviation, and interquartile range.
- Calculate quartiles using a stated convention.
- Read a five-number summary and a boxplot.
- Explain a z-score and an outlier flag without overstating either.
- Compare groups with appropriate units and limits.

Reading: main 8th-edition book, §3.1 (location and percentiles), §3.2 (variability), selected ideas from §3.3 (z-scores), and §3.4 (five-number summaries and boxplots). The conceptual references are from the book; the R analyses below apply them to our checked Audit and Restaurant CSVs.

Business case: describe a typical audit and its variability

An accounting manager has completion times for **20 year-end audits**, recorded in days. The manager needs a concise description of a typical duration, the variation between audits, and any unusually long cases to investigate. One row is one audit. These historical observations inform a description; a completion commitment for a new client requires additional information.

لدى مدير في مكتب محاسبة بيانات المدة اللازمة لإنجاز **عشرين عملية تدقيق لنهاية السنة**، والمسجّلة بالأيام. يحتاج المدير إلى وصف موجز للمدة المعتادة، ومدى تفاوت المدد بين العمليات، وأي حالات طويلة بصورة غير معتادة تستحق الدراسة. يمثل كل صف عملية تدقيق واحدة. تساعد هذه المشاهدات التاريخية على الوصف، أما الالتزام بموعد إنجاز لعميل جديد فيتطلب معلومات إضافية.

Download the Audit CSV. We use the original recorded values, not histogram class midpoints. The 20 values match book Table 2.4, p.47.

```
# library() loads ggplot2 for charts and knitr for table formatting.
# read.csv() loads the checked Audit CSV; $ selects its audit_days column.
# x is the vector of 20 durations in days, not class midpoints.
library(ggplot2)
library(knitr)
audit <- read.csv("data/module02/audit_times.csv")
x <- audit$audit_days
```

Mean, median, and mode: three meanings of typical

Mean: balance all the numerical values

The **sample mean** adds the observations and divides by their number:

$$\bar{x} = \frac{\sum_{i=1}^{n} x_i}{n}.$$

For these audits, the total is **385 days across 20 audits**, so $\bar{x} = 385/20 = 19.25$ days. This is the average of recorded durations, not a statement that every audit took about 19 days.

The mean **compresses many numerical observations into one useful summary**. It gives an initial insight into the level of the data, preserves the original units, and connects directly to the total: mean × number of observations = total.

Every value contributes to the mean. Long durations can pull it upward; this sensitivity can be useful when total workload matters. However, the mean does not show how observations are distributed, and an extreme value can make it a poor description of where most observations lie. Its usefulness depends on the question being asked.

```
# mean() averages the 20 raw audit-day values stored in x.
# The result is in days; use the complete vector, not histogram class midpoints.
mean(x)
```

```
[1] 19.25
```

The R result is **19.25 days**. The mean times 20 recovers the recorded total of 385 days. ### Median: locate the middle after sorting

The **median** is the middle ordered value. With an odd number of observations, take the central observation; with an even number of numerical observations, average the two central values.

```
# kable(): formats a data frame or matrix as a readable report table; digits sets
 ↪  displayed precision.
# c(): combines values into a vector, converting them to a common type when needed.
# sort() orders audit days; seq_along() numbers their positions from 1 to n.
# data.frame() pairs positions with values so we can identify the median observations.
ordered_audits <- data.frame(Position = seq_along(x), Days = sort(x))
kable(ordered_audits, col.names = c("Sorted position", "Audit duration (days)"))
```

Sorted position	Audit duration (days)
1	12
2	13
3	14
4	14
5	15
6	15
7	16
8	17
9	18
10	18
11	18
12	19
13	20
14	21
15	22
16	22
17	23
18	27
19	28
20	33

There are 20 observations. Positions 10 and 11 are both 18, so the median is $(18 + 18)/2 = 18$ days. It describes the center by rank and is less sensitive than the mean to a very large observation. Ties mean that "half below and half above" should not be interpreted as exactly half strictly below and above the median.

```
# median() locates the center by rank after sorting the numerical values.
# With 20 durations, it averages the two middle observations.
median(x)
```

```
[1] 18
```

The R result is **18 days**, agreeing with the two middle ordered observations. ### Mode: identify the most frequent value

The **mode** is the value or category occurring most frequently. Here **18 days** occurs three times, more than any other exact duration. A dataset can have several modes or no single most frequent value.

R's `mode()` describes an object's storage mode; it does **not** calculate the statistical mode. Count values instead.

```
# table() counts exact days; max() identifies the largest count.
# names() retrieves all matching labels; as.numeric() converts them back to days.
value_counts <- table(x)
mode_days <- as.numeric(names(value_counts)[value_counts == max(value_counts)])
mode_days
```

```
[1] 18
```

```
# max(): returns the largest supplied value.
# data.frame(): builds a table whose named columns have compatible lengths.
# c(): combines values into a vector, converting them to a common type when needed.
# mean(): computes an average; logical TRUE/FALSE values contribute 1/0, giving a
 ↪  proportion.
# median(): returns the middle sorted value, averaging the middle pair for even sample
 ↪  sizes.
# stopifnot(): stops execution if any supplied condition is FALSE; successful checks
 ↪  produce no output.
```

```
# identical(): requires exact equality of values, types, and attributes.
# kable(): formats a data frame or matrix as a readable report table; digits sets
 ↪  displayed precision.
# table() counts exact durations; the largest count identifies the statistical mode.
# as.numeric(names(...)) converts the matching table labels back to numerical days.
value_counts <- table(x)
audit_modes <- as.numeric(names(value_counts)[value_counts == max(value_counts)])
center <- data.frame(Measure = c("Mean", "Median", "Mode"),
                     Days = c(mean(x), median(x), audit_modes))
stopifnot(mean(x) == 19.25, median(x) == 18,
          identical(audit_modes, 18), max(value_counts) == 3)
kable(center, digits = 2)
```

Measure	Days
Mean	19.25
Median	18.00
Mode	18.00

```
# data.frame(): builds a table whose named columns have compatible lengths.
# sort(): sorts values; decreasing=TRUE reverses the default ascending order.
# c(): combines values into a vector, converting them to a common type when needed.
# mean(): computes an average; logical TRUE/FALSE values contribute 1/0, giving a
 ↪  proportion.
# median(): returns the middle sorted value, averaging the middle pair for even sample
 ↪  sizes.
# ggplot(): starts a plot from data; additional layers are joined with +.
# aes(): maps variables to visual properties such as x, y, fill, color, or group.
# geom_point(): draws a point for each row at its mapped x and y values.
# scale_y_continuous(): controls numeric y-axis breaks, labels, limits, and spacing.
# labs(): sets axis titles, legend titles, plot titles, and captions.
# theme(): customizes non-data plot elements such as legends, grids, and axis text.
# ave(..., FUN = seq_along) stacks repeated durations so every observation is visible.
# geom_vline() marks the calculated mean and median using distinct line styles.
dots <- data.frame(Days = sort(x))
dots$Stack <- ave(dots$Days, dots$Days, FUN = seq_along)
centers <- data.frame(Measure = c("Mean: 19.25 days", "Median: 18 days"), Days =
 ↪  c(mean(x), median(x)))
ggplot(dots, aes(Days, Stack)) +
  geom_point(size = 3, colour = "#2878A5") +
  geom_vline(data = centers, aes(xintercept = Days, linetype = Measure),
             colour = "#596775", linewidth = .7) +
  scale_y_continuous(breaks = 1:3) +
  labs(title = "The mean and median describe different centers",
       x = "Audit duration (days)", y = "Stacked observations", linetype = NULL) +
  theme(legend.position = "bottom")
```

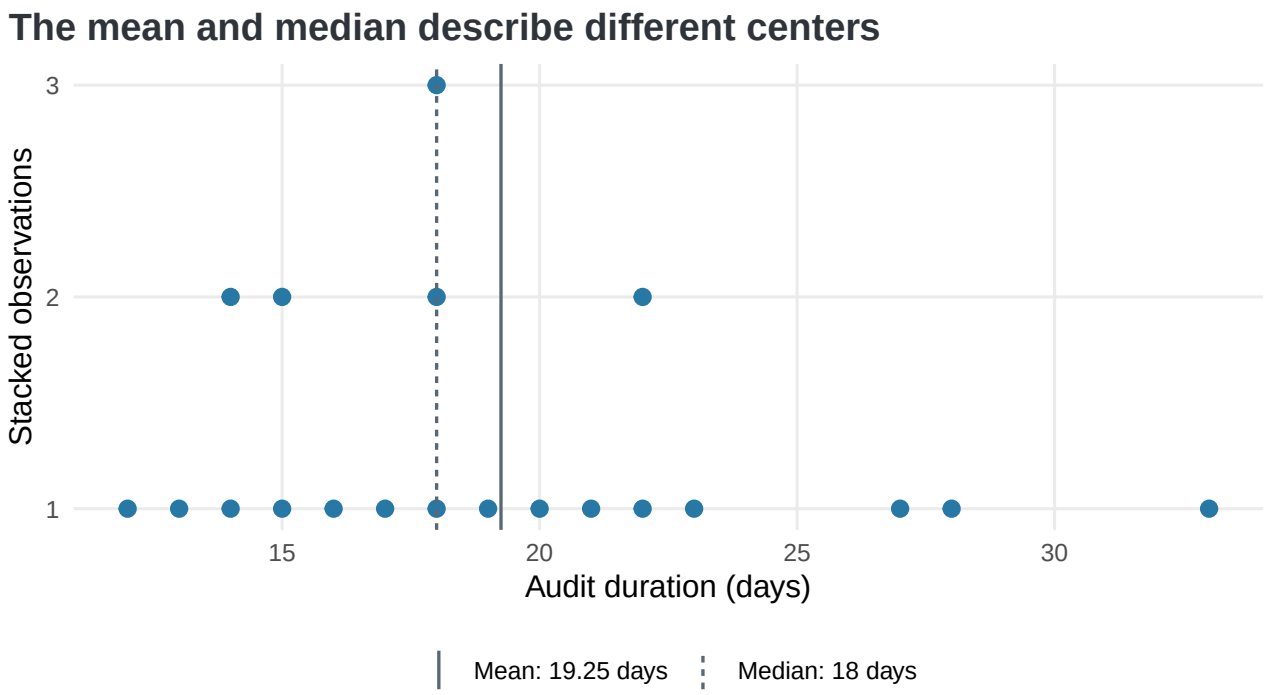


Figure 1: Original R display of the checked Audit observations. Each dot is one audit; line styles distinguish the mean and median.

Interpretation: the mean exceeds the median in these observations. The longer durations pull the mean upward. Inspect the distribution as well: the ordering of mean and median alone is not a universal test of skewness.

When the mean conceals the typical value

The instructor-provided descriptive statistics PDF (p.51; original deck slide 38) uses seven dollar amounts: **1, 2, 3, 3, 3, 1, 100**. Six lie between USD 1 and USD 3, while one is USD 100. These are teaching values, with no business setting specified.

```
# kable(): formats a data frame or matrix as a readable report table; digits sets
 ↪  displayed precision.
# data.frame(): builds a table whose named columns have compatible lengths.
# c(): combines values into a vector, converting them to a common type when needed.
# stopifnot(): stops execution if any supplied condition is FALSE; successful checks
 ↪  produce no output.
# abs(): returns absolute values; a small tolerance in an equality check permits
 ↪  floating-point rounding.
# subset() selects the extreme-value teaching example from the supplementary CSV.
# sum(), mean(), and median() show how the large value changes the reported center.
amounts <- subset(limitations, example == "extreme_value")$value
kable(data.frame(Measure = c("Total (USD)", "Mean (USD)", "Median (USD)", "Mode (USD)"),
                 Value = c(sum(amounts), mean(amounts), median(amounts), 3)), digits = 2)
```

Measure	Value
Total (USD)	113.00
Mean (USD)	16.14
Median (USD)	3.00
Mode (USD)	3.00

```
stopifnot(sum(amounts) == 113, median(amounts) == 3, abs(mean(amounts)-113/7) < 1e-10)
```

The mean, **USD 16.14**, correctly divides the total by seven. Yet it is above six of the seven observations, so it poorly describes the typical amount in this example. The median, **USD 3**, better describes the middle. The mean remains useful when the total matters. A valid large value should remain in the data even when it makes the mean unrepresentative of most observations.

Here the mode is also **USD 3**, but it answers a different question: which amount occurs most often? The median describes the middle by rank, while the mode describes frequency. Neither supplies the total or describes the spread. We choose and interpret these summaries according to their meaning; they are not automatic replacements for the mean.

Download the teaching comparisons as CSV. The source notes distinguish examples from your instructor's slides and original course illustrations.

Variability: how different are the durations?

The mean is useful, but it is not enough on its own to describe data. Even a correctly calculated, representative mean tells us nothing by itself about how far apart the observations are. The instructor-provided PDF (pp.60–61; original deck slides 44–45) gives five sets of three values. **Every set has mean 200**, despite very different variability. The numbers are conceptual teaching values in arbitrary units, not audit durations.

```
# subset(): keeps entries or rows satisfying a condition; select can choose columns.
# do.call(): calls a function using a list of arguments, e.g. rbind to combine a list of
 ↪  tables.
# names(): reads or assigns names to vector elements or table columns.
# data.frame(): builds a table whose named columns have compatible lengths.
# mean(): computes an average; logical TRUE/FALSE values contribute 1/0, giving a
 ↪  proportion.
# stopifnot(): stops execution if any supplied condition is FALSE; successful checks
 ↪  produce no output.
# all(): returns TRUE only when every supplied logical value is TRUE.
# kable(): formats a data frame or matrix as a readable report table; digits sets
 ↪  displayed precision.
# split() separates the instructor examples by set; lapply() summarizes each set.
# paste(..., collapse = ", ") lists its values, and rbind combines the summary rows.
same_mean <- split(subset(limitations, example == "same_mean")$value,
                   subset(limitations, example == "same_mean")$series)
same_mean_table <- do.call(rbind, lapply(names(same_mean), function(g) {
  v <- same_mean[[g]]
  data.frame(Set = g, Values = paste(v, collapse = ", "), Mean = mean(v))
}))
stopifnot(all(same_mean_table$Mean == 200))
kable(same_mean_table, row.names = FALSE)
```

Set	Values	Mean
S1	100, 200, 300	200
S2	200, 200, 200	200
S3	0, 600, 0	200
S4	-600, 600, 600	200
S5	-2000, 2500, 100	200

S2 has no variation: every observation is 200. S5 contains widely separated values, from -2000 to 2500. Reporting only "mean = 200" conceals this distinction. The mean has correctly summarized their common average, but it has compressed away information about their differences. **Describe center and dispersion together**, using a plot to see the distribution as well.

Range: the full observed span

The **range** is maximum minus minimum: $33 - 12 = 21$ days. It uses only two observations, so it can change markedly when a single extreme value changes. It does not describe how the values are distributed between those endpoints.

For the five sets above, the ranges are **200, 0, 600, 1200, and 4500**, respectively. The range immediately reveals that their observed spans differ. It is useful when the full observed span matters, but it is not a description of the spread of most observations.

```
# range() returns the two endpoints, not their difference.
# diff() subtracts successive endpoints: maximum minus minimum, in days.
range(x)
```

```
[1] 12 33
```

```
diff(range(x))
```

```
[1] 21
```

R returns endpoints **12 and 33 days**, then the range **21 days**. ### The range is sensitive to an extreme value

Return to the seven dollar amounts: **1, 2, 3, 3, 3, 1, 100**. Their range is **USD 99**, although six of the seven amounts occupy the narrow interval from USD 1 to USD 3. Both statements are correct: the full span is wide, and most observations are close together.

```
# min() and max() find the endpoints; diff(range()) subtracts the minimum from the
 ↪  maximum.
# data.frame() constructs the summary row, and kable() displays the dollar amounts to two
 ↪  decimals.
# stopifnot() stops if the calculated range fails to match the slide's USD 99 result.
kable(data.frame(Minimum_USD = min(amounts), Maximum_USD = max(amounts),
                 Range_USD = diff(range(amounts))), digits = 2)
```

Minimum_USD	Maximum_USD	Range_USD
1	100	99

```
stopifnot(diff(range(amounts)) == 99)
```

An extreme value directly changes the range when it changes an endpoint. For example, **if USD 100 were USD 1000 in a hypothetical version**, the range would rise from USD 99 to USD 999 while the other six observations stayed exactly the same. This is a sensitivity illustration, not a correction to the original values.

The range therefore gives limited insight into the spread of the main cluster when an endpoint is extreme. Report it for what it measures, alongside the distribution and other suitable summaries. Measures using all observations can describe internal spread, but they can also be sensitive to extremes.

The range misses the values between the endpoints

Consider a second, **original course illustration** in arbitrary units:

- Set A: **0, 5, 5, 5, 10**.
- Set B: **0, 0, 5, 10, 10**.

Both have **mean 5 and range 10**. In A, three observations sit at the center. In B, four sit at the extremes. The range cannot distinguish these patterns because their endpoints agree.

```
# subset(): keeps entries or rows satisfying a condition; select can choose columns.
# ggplot(): starts a plot from data; additional layers are joined with +.
# aes(): maps variables to visual properties such as x, y, fill, color, or group.
# geom_vline(): adds vertical references at xintercept; linetype selects solid or dashed
 ↪  lines.
# geom_point(): draws a point for each row at its mapped x and y values.
# scale_x_continuous(): controls numeric x-axis breaks, labels, limits, and spacing.
# c(): combines values into a vector, converting them to a common type when needed.
# scale_y_continuous(): controls numeric y-axis breaks, labels, limits, and spacing.
# labs(): sets axis titles, legend titles, plot titles, and captions.
# ave() gives repeated values stacking positions within each set.
# facet_wrap() separates A and B while the dashed line marks their common mean.
same_range <- subset(limitations, example == "same_range")
same_range$Stack <- ave(same_range$value, same_range$series, same_range$value, FUN =
 ↪  seq_along)
ggplot(same_range, aes(value, Stack)) +
  geom_vline(xintercept = 5, linetype = "dashed", colour = "#7B858D") +
  geom_point(size = 4, colour = "#2878A5") +
  facet_wrap(~series, ncol = 1) +
  scale_x_continuous(breaks = c(0, 5, 10), limits = c(-1, 11)) +
  scale_y_continuous(breaks = 1:3, limits = c(.5, 3.5)) +
  labs(title = "Equal ranges can conceal different internal spread",
       x = "Value (arbitrary units)", y = "Stacked observations")
```

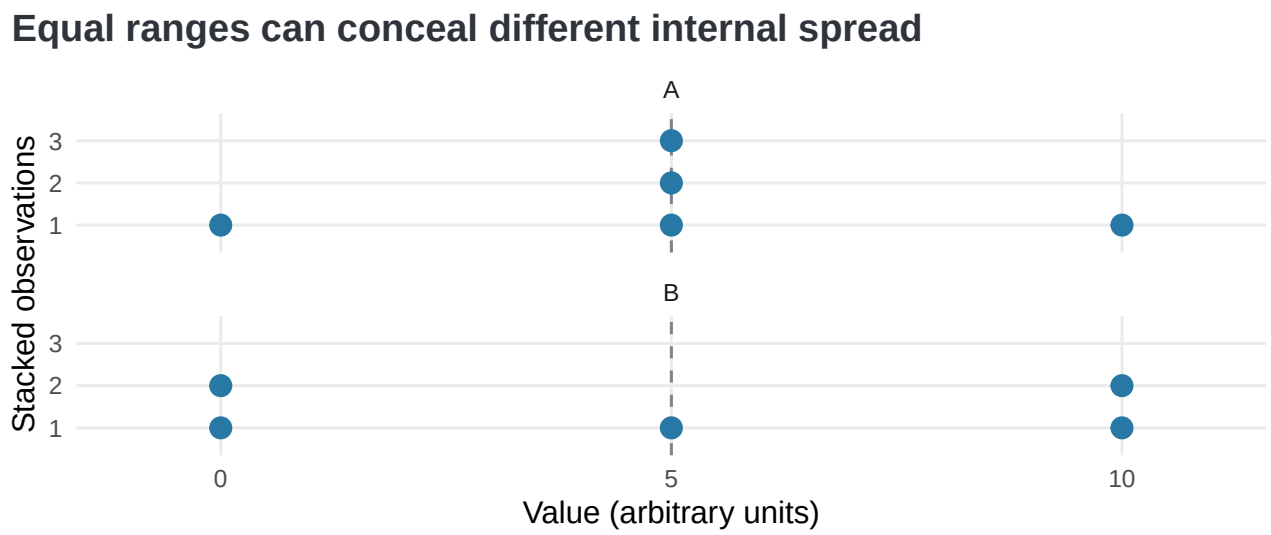


Figure 2: Original course illustration. Each dot is one observation; stacked dots indicate repeated values. Both sets share the same mean and range.

Mean absolute deviation: every distance contributes

To describe internal spread, measure each observation's distance from the mean. Simply averaging **signed deviations** fails because they sum to zero. One solution is to average their absolute values:

$$\text{Mean absolute deviation about the mean} = \frac{1}{n}\sum_{i=1}^{n}|x_i - \bar{x}|.$$

For A, the distances are **5, 0, 0, 0, 5**, averaging **2 units**. For B, they are **5, 5, 0, 5, 5**, averaging **4 units**. This measure distinguishes their internal spread and retains the original units.

In R, use `mean(abs(v - mean(v)))`. R's `mad()` calculates a scaled **median** absolute deviation by default, which is a different measure.

The next approach squares distances. Absolute deviations grow linearly with distance, while squared deviations give increasingly large contributions to distant observations. This choice serves a different purpose; it does not make variance or standard deviation universally better or resistant to extremes.

```
# mean(x) supplies the reference center; abs() takes every deviation's magnitude.
# The outer mean() averages all absolute distances, retaining days as the unit.
mean(abs(x - mean(x)))
```

```
[1] 4.2
```

This applies the same definition to the textbook Audit observations. It differs from R's `mad()`, which defaults to a scaled median absolute deviation. ### Sample variance: average squared variation, with an adjustment

For a sample of size $n > 1$, the sample variance is

$$s^2 = \frac{\sum_{i=1}^{n}(x_i - \bar{x})^2}{n-1}.$$

The deviations describe distances from the mean. Squaring prevents negative and positive deviations from cancelling. The denominator is $n-1$ because estimating the mean uses one degree of freedom. Under independent sampling from a common population with finite variance, this adjustment makes s^2 an unbiased estimator of the population variance.

For a complete population, its descriptive variance uses its population mean and denominator N. Whether data constitute a population or a sample depends on the question, not whether the file contains "all the rows available." Here we treat the 20 audits as a sample, and R's `var()` follows the $n-1$ convention.

```
# var() calculates sample variance using denominator n - 1.
# Its unit is days squared; all 20 original observations contribute.
var(x)
```

```
[1] 29.56579
```

R returns approximately **29.57 days squared**. ### Standard deviation: spread in the original units

The **sample standard deviation** is $s = \sqrt{s^2}$. Variance has squared units; standard deviation has the original units. For Audit, variance is approximately **29.57 days²** and standard deviation is **5.44 days**.

Standard deviation summarizes spread around the mean. It is not the standard error, a maximum possible deviation, or a guarantee that every observation lies within one standard deviation of the mean. No normal-distribution assumption is required merely to calculate it.

```
# sd() is the square root of sample variance, using the n - 1 convention.
# It returns spread in the original unit, days; it is not the standard error.
sd(x)
```

```
[1] 5.437443
```

R returns approximately **5.44 days**.

```
# sum(): adds vector entries; on logical values it counts TRUE entries.
# data.frame(): builds a table whose named columns have compatible lengths.
# c(): combines values into a vector, converting them to a common type when needed.
# diff(): subtracts each preceding value from the next by default; diff(range(x)) gives
 ↪  max minus min.
# range(): returns the minimum and maximum as a two-element vector.
# stopifnot(): stops execution if any supplied condition is FALSE; successful checks
 ↪  produce no output.
# abs(): returns absolute values; a small tolerance in an equality check permits
 ↪  floating-point rounding.
# length(): counts elements in a vector or list.
# kable(): formats a data frame or matrix as a readable report table; digits sets
 ↪  displayed precision.
# Subtract mean(x), square the deviations, and sum them to measure total squared
 ↪  variation.
# var() divides by n-1; sd() returns its square root in the original units.
squared_deviations <- sum((x - mean(x))^2)
variability <- data.frame(
  Measure = c("Range (days)", "Sum of squared deviations (days squared)",
              "Sample variance (days squared)", "Sample standard deviation (days)"),
  Value = c(diff(range(x)), squared_deviations, var(x), sd(x)))
stopifnot(abs(squared_deviations / (length(x)-1) - var(x)) < 1e-10,
          abs(sd(x)^2 - var(x)) < 1e-10)
kable(variability, digits = 2)
```

Measure	Value
Range (days)	21.00
Sum of squared deviations (days squared)	561.75
Sample variance (days squared)	29.57
Sample standard deviation (days)	5.44

Returning to sets A and B, both measures that use all observations detect the difference the range missed:

```
# do.call(): calls a function using a list of arguments, e.g. rbind to combine a list of
 ↪  tables.
# lapply(): applies a function separately to each element and returns a list.
# data.frame(): builds a table whose named columns have compatible lengths.
# length(): counts elements in a vector or list.
# stopifnot(): stops execution if any supplied condition is FALSE; successful checks
 ↪  produce no output.
# all(): returns TRUE only when every supplied logical value is TRUE.
# c(): combines values into a vector, converting them to a common type when needed.
# sqrt(): returns the nonnegative square root of a nonnegative number.
# kable(): formats a data frame or matrix as a readable report table; digits sets
 ↪  displayed precision.
# split() processes sets A and B separately; diff(range(v)) calculates maximum minus
 ↪  minimum.
# mean(abs(v-mean(v))) averages absolute distances; sd(v) calculates sample SD.
spread_comparison <- do.call(rbind, lapply(split(same_range$value, same_range$series),
 ↪  function(v) {
  data.frame(N = length(v), Mean = mean(v), Range = diff(range(v)),
             Mean_absolute_deviation = mean(abs(v-mean(v))), Sample_SD = sd(v))
}))
stopifnot(all(spread_comparison$Mean == 5), all(spread_comparison$Range == 10),
          all(spread_comparison$Mean_absolute_deviation == c(2,4)),
          abs(spread_comparison$Sample_SD[1]-sqrt(12.5)) < 1e-10,
          spread_comparison$Sample_SD[2] == 5)
```

```
kable(spread_comparison, digits = 2,
      col.names = c("n", "Mean", "Range", "Mean absolute deviation", "Sample SD"))
```

	n	Mean	Range	Mean absolute deviation	Sample SD
A	5	5	10	2	3.54
B	5	5	10	4	5.00

What has been addressed? Measures using all values describe internal variation that range alone misses. **What remains?** Mean absolute deviation about the mean and standard deviation still respond to extreme values. We next introduce quartiles and IQR to describe the central portion of the distribution.

Percentiles and quartiles: locate values by rank

A **percentile** locates a value within an ordered distribution. Quartiles are the 25th, 50th, and 75th percentiles: Q_1, Q_2 (the median), and Q_3. They describe approximate portions of the distribution; ties and interpolation matter in small samples.

There are several accepted sample-percentile conventions. On printed pp.113–114, the book uses the position

$$L_p = (n+1)\frac{p}{100},$$

with interpolation between adjacent sorted observations when the position lies between them. We use **R's type = 6** to follow that interior-percentile convention for these quartiles.

For 20 audits:

- $L_{25} = 21 \times 0.25 = 5.25$: one-quarter of the way from the 5th value (15) to the 6th (15), giving $Q_1 = 15$.
- $L_{50} = 10.5$: halfway from the 10th value (18) to the 11th (18), giving $Q_2 = 18$.
- $L_{75} = 15.75$: three-quarters of the way from the 15th value (22) to the 16th (22), giving $Q_3 = 22$.

```
# quantile() computes probabilities supplied by c(); x contains raw audit days.
# type = 6 follows the stated book convention; R's default is type = 7.
quartiles <- quantile(x, c(.25,.50,.75), type = 6)
quartiles
```

```
25% 50% 75%
 15  18  22
```

```
# c(): combines values into a vector, converting them to a common type when needed.
# kable(): formats a data frame or matrix as a readable report table; digits sets
 ↪  displayed precision.
# data.frame(): builds a table whose named columns have compatible lengths.
# stopifnot(): stops execution if any supplied condition is FALSE; successful checks
 ↪  produce no output.
# identical(): requires exact equality of values, types, and attributes.
# quantile() finds the requested percentiles; type = 6 follows our book convention.
# unname() removes percentile labels for the display; the numerical quartiles are
 ↪  unchanged.
kable(data.frame(Quartile = c("Q1: 25th percentile", "Q2: median", "Q3: 75th percentile"),
                 Days = unname(quartiles)))
```

Quartile	Days
Q1: 25th percentile	15
Q2: median	18
Q3: 75th percentile	22

```
stopifnot(identical(unname(quartiles), c(15,18,22)))
```

> **State the software convention**
>
> R's default is `type = 7`. Specify `type = 6` for our book-aligned calculations. `IQR(x)` and ordinary ggplot2 boxplots also use their own defaults, so calculate the IQR from the stated quartiles and use the explicit boxplot statistics below. The methods happen to give the same Audit quartiles; that does not make them interchangeable for every dataset. Do not extrapolate the book's interior-position calculation beyond the available ordered observations.

Interquartile range: the width of the middle half

The **interquartile range** is $IQR = Q_3 - Q_1 = 22 - 15 = 7$ days. The quartiles describe the middle half of the distribution, approximately 15 to 22 days here. With ties, the number of records inside an inclusive quartile interval need not equal exactly half the sample.

Median and IQR emphasize the central part of the distribution. Mean and standard deviation use the magnitudes of all observations. Choose them according to the question and the observed distribution, not from a rule that one pair is always superior.

```
# c() selects Q1/Q3 from our type-6 quartiles; diff() subtracts Q1 from Q3.
# unname() removes the resulting label, leaving the IQR in days.
unname(diff(quartiles[c(1, 3)]))
```

```
[1] 7
```

R returns **7 days**. Using our stored quartiles keeps the convention explicit. ## Five-number summaries and boxplots

A **five-number summary** contains the minimum, Q_1, median, Q_3, and maximum.

```
# min(): returns the smallest supplied value.
# median(): returns the middle sorted value, averaging the middle pair for even sample
 ↪  sizes.
# max(): returns the largest supplied value.
# kable(): formats a data frame or matrix as a readable report table; digits sets
 ↪  displayed precision.
# data.frame(): builds a table whose named columns have compatible lengths.
# c() combines the minimum, quartiles, median, and maximum into a named vector.
# unname() supplies its values to the table without duplicating the vector labels.
five <- c(Minimum = min(x), Q1 = quartiles[1], Median = median(x),
          Q3 = quartiles[3], Maximum = max(x))
kable(data.frame(Measure = c("Minimum", "Q1", "Median", "Q3", "Maximum"),
                 Days = unname(five)))
```

Measure	Days
Minimum	12
Q1	15
Median	18
Q3	22
Maximum	33

A **boxplot** shows the box from Q_1 to Q_3, with a line at the median. Under the book's outlier-screening rule, calculate:

$$\text{Lower fence} = Q_1 - 1.5IQR, \qquad \text{Upper fence} = Q_3 + 1.5IQR.$$

The **whiskers end at observed values inside the fences**, not at the fences themselves. Observations beyond the fences are drawn separately. Thus, a whisker endpoint is not always the overall minimum or maximum.

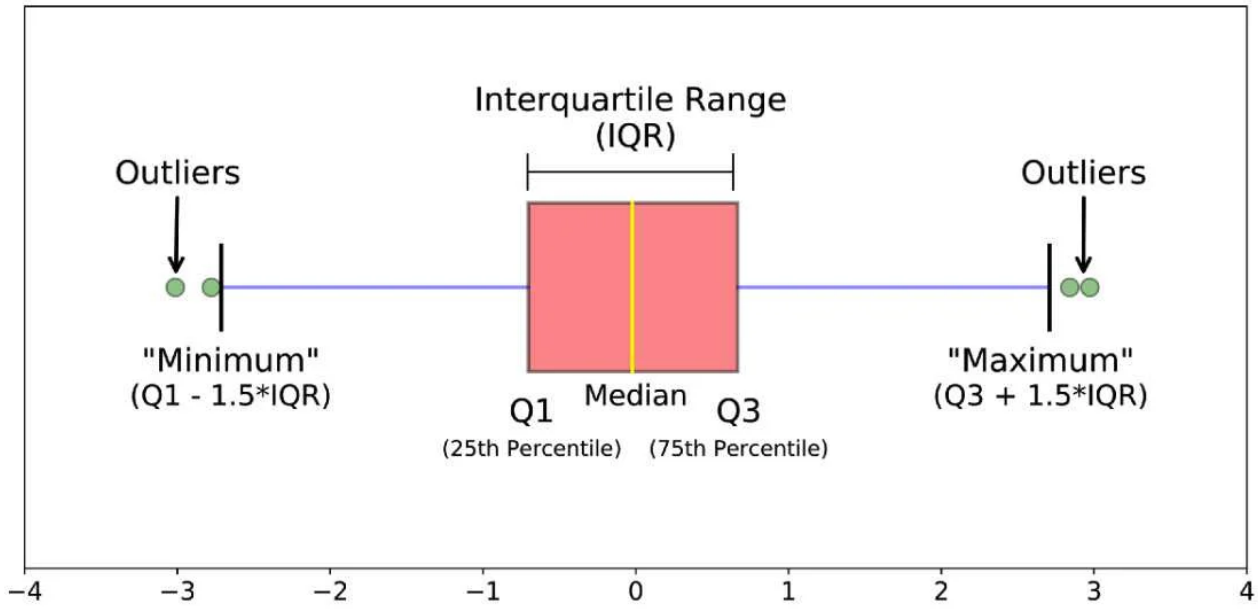


Figure 3: Anatomy of a boxplot: quartiles, median, IQR, whiskers, and separately plotted observations. Supplied schematic; the fence labels are clarified below.

i Clarifying the illustration's labels

The expressions labeled "Minimum" and "Maximum" in this schematic are the **lower and upper fences**, not generally the dataset's minimum and maximum. For our boxplots, whiskers stop at the most extreme **observed values within** those fences. A point beyond a fence is flagged for investigation.

Open the illustration at full size.

For Audit, the fences are **4.5 and 32.5 days**. The whiskers end at **12 and 28 days**. The 33-day audit is flagged for investigation, while remaining part of the dataset and its five-number summary.

```
# stopifnot(): stops execution if any supplied condition is FALSE; successful checks
 ↪  produce no output.
# identical(): requires exact equality of values, types, and attributes.
# as.integer(): converts to integers by truncating numeric fractional parts; it does not
 ↪  round to nearest.
```

```r
# ggplot(): starts a plot from data; additional layers are joined with +.
# aes(): maps variables to visual properties such as x, y, fill, color, or group.
# geom_boxplot(): draws quartiles, whiskers, and flagged points; a flagged value is not
 ↪  automatically an error.
# geom_point(): draws a point for each row at its mapped x and y values.
# data.frame(): builds a table whose named columns have compatible lengths.
# geom_hline(): adds horizontal references at yintercept.
# labs(): sets axis titles, legend titles, plot titles, and captions.
# box_record() is our helper for type-6 quartiles, fences, and observed whisker endpoints.
# stat = "identity" draws those supplied statistics instead of recomputing ggplot2
 ↪  defaults.
audit_box <- box_record(x, "Audit")
audit_flagged <- x[x < audit_box$Lower_fence | x > audit_box$Upper_fence]
stopifnot(identical(as.integer(audit_flagged), 33L), audit_box$ymin == 12, audit_box$ymax
 ↪  == 28)
ggplot(audit_box, aes(x = Group, ymin = ymin, lower = Q1, middle = Median,
                      upper = Q3, ymax = ymax)) +
  geom_boxplot(stat = "identity", width = .35, fill = "#B6D2E2", colour = "#394B58") +
  geom_point(data = data.frame(Group = "Audit", Days = audit_flagged),
             aes(x = Group, y = Days), inherit.aes = FALSE,
             size = 3, colour = "#B47816") +
  geom_hline(yintercept = audit_box$Upper_fence, linetype = "dashed", colour = "#7B858D")
   ↪  +
  labs(title = "A flag is a prompt to investigate", x = NULL, y = "Audit duration (days)")
```

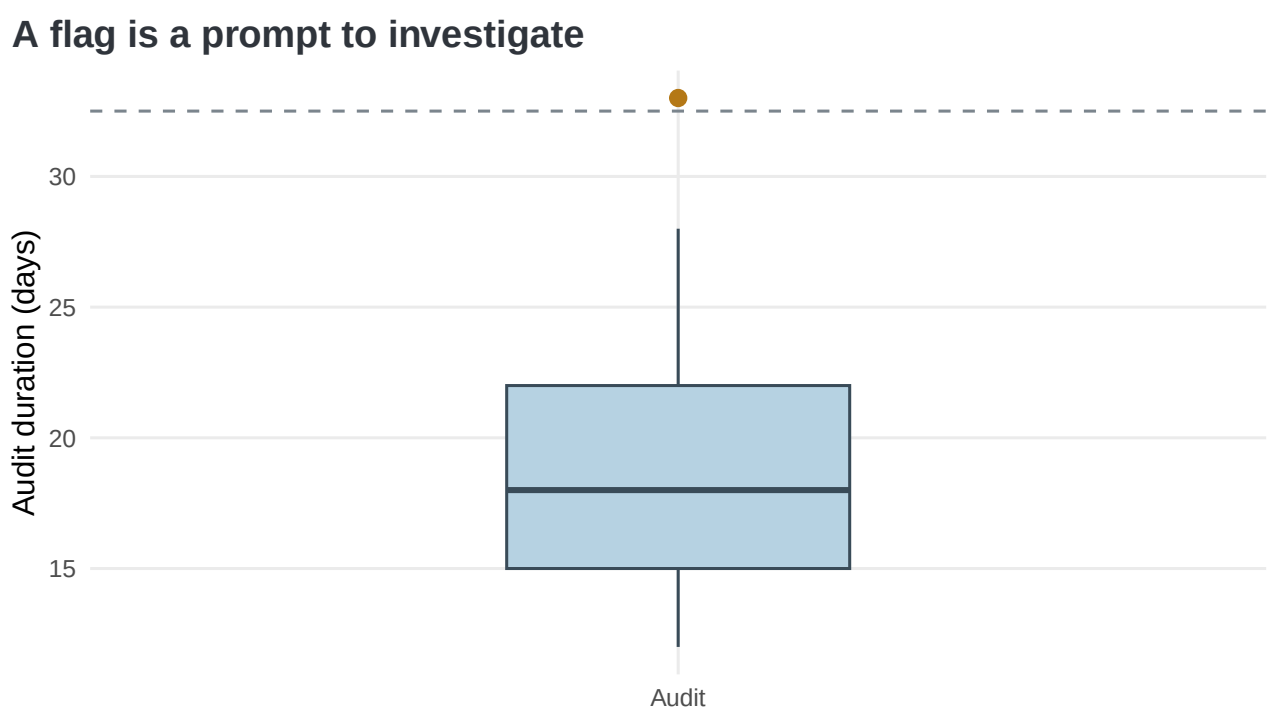


Figure 4: Original R boxplot using explicit type-6 quartiles and 1.5-IQR fences. The separately drawn point is the 33-day audit; it has not been removed.

A flag does not establish a recording error, poor performance, or a reason to delete a valid audit. Check the source record, the audit's scope, and the circumstances. A boxplot also hides detail; return to the dot plot or histogram when looking for clusters or multiple peaks.

How much does a long observation change the summary?

Consider a **hypothetical sensitivity exercise**: replace the 33-day value with 60 days in a separate copy. This is an illustration, not a correction to the book's data.

```r
# data.frame(): builds a table whose named columns have compatible lengths.
# c(): combines values into a vector, converting them to a common type when needed.
# mean(): computes an average; logical TRUE/FALSE values contribute 1/0, giving a
 ↪  proportion.
# median(): returns the middle sorted value, averaging the middle pair for even sample
 ↪  sizes.
# sd(): computes sample standard deviation using denominator n-1.
# diff(): subtracts each preceding value from the next by default; diff(range(x)) gives
 ↪  max minus min.
# q6(): is a locally defined helper computing type-6 sample quartiles for textbook
 ↪  consistency.
# kable(): formats a data frame or matrix as a readable report table; digits sets
 ↪  displayed precision.
# stopifnot(): stops execution if any supplied condition is FALSE; successful checks
 ↪  produce no output.
# Create a separate copy, then which.max() locates the largest duration to replace.
# Recalculate both sets of summaries while preserving the original audit vector x.
hypothetical <- x
hypothetical[which.max(hypothetical)] <- 60
sensitivity <- data.frame(
  Dataset = c("Original observations", "Hypothetical: 33 replaced by 60"),
  Mean_days = c(mean(x), mean(hypothetical)),
  Median_days = c(median(x), median(hypothetical)),
  SD_days = c(sd(x), sd(hypothetical)),
  IQR_days = c(diff(q6(x, c(.25,.75))), diff(q6(hypothetical, c(.25,.75)))))
kable(sensitivity, digits = 2,
      col.names = c("Dataset", "Mean (days)", "Median (days)", "SD (days)", "IQR (days)"))
```

Dataset	Mean (days)	Median (days)	SD (days)	IQR (days)
Original observations	19.25	18	5.44	7
Hypothetical: 33 replaced by 60	20.60	18	10.25	7

```r
stopifnot(mean(x) == 19.25, max(x) == 33, mean(hypothetical) == 20.6,
          median(hypothetical) == 18)
```

The mean rises to **20.60 days** and the standard deviation to **10.25 days**. The median and IQR are unchanged in this particular exercise. This demonstrates resistance to one tail change, not immunity to every possible change in the data.

Two additional measures: CV and z-scores

Coefficient of variation

The **coefficient of variation** expresses standard deviation relative to the mean:

$$CV = 100\frac{s}{\bar{x}}.$$

For Audit, it is approximately **28.25%**. It can help compare relative variability for suitable positive ratio-scale quantities, but it is unstable when the mean is near zero and is not meaningful for arbitrary-zero scales such as Celsius. Comparing CVs alone does not establish that one business process is better.

```r
# sd()/mean() compare sample spread with the average; multiply by 100 for percent.
# Audit duration has a meaningful zero and a positive mean; this use of CV is suitable.
100 * sd(x) / mean(x)
```

```
[1] 28.24646
```

Z-score

A **sample z-score** measures how far a value is from the sample mean in standard-deviation units:

$$z_i = \frac{x_i - \bar{x}}{s}, \qquad s > 0.$$

The 33-day audit has $z \approx 2.53$: it is about 2.53 sample standard deviations above the mean. This calculation does not require a normal distribution, but it does not automatically supply a probability. The 1.5-IQR boxplot rule and a $|z| > 3$ screening rule can flag different observations; neither is an automatic deletion rule.

```
# mean()/sd() provide the reference center and sample spread in days.
# Subtract and divide to locate the 33-day audit in SD units; this is not a normality test.
(33 - mean(x)) / sd(x)
```

```
[1] 2.528762
```

```
# kable(): formats a data frame or matrix as a readable report table; digits sets
 ↪  displayed precision.
# data.frame(): builds a table whose named columns have compatible lengths.
# c(): combines values into a vector, converting them to a common type when needed.
# 100*sd(x)/mean(x) expresses sample spread as a percentage of the mean (CV).
# (33-mean(x))/sd(x) places the 33-day audit in sample-standard-deviation units.
kable(data.frame(Measure = c("Coefficient of variation (%)", "Z-score of the 33-day
 ↪  audit"),
                 Value = c(100 * sd(x) / mean(x), (33 - mean(x)) / sd(x))), digits = 2)
```

Measure	Value
Coefficient of variation (%)	28.25
Z-score of the 33-day audit	2.53

Business case: compare restaurant price distributions

Return to the **300 restaurants**. An analyst wants to compare meal prices across the ordered quality categories Good, Very Good, and Excellent. Are typical prices different, and how variable are prices within each group? One observation remains one restaurant; prices are in US dollars. These observational comparisons describe an association and do not isolate a causal effect of a quality rating.

نعود إلى بيانات **ثلاثمئة مطعم**. يريد محلّل مقارنة أسعار الوجبات بين فئات الجودة المرتبة: جيد، وجيد جدًا، وممتاز. هل تختلف الأسعار المعتادة، وما مدى تفاوت الأسعار داخل كل فئة؟ تظل كل مشاهدة ممثلةً لمطعم واحد، والأسعار مقيسة بالدولار الأمريكي. تصف هذه المقارنات ارتباطًا في بيانات رصدية، ولا تعزل أثرًا سببيًا لتقييم الجودة.

Download the Restaurant CSV. Report each group size so a reader can see that the groups differ in size.

```r
# names(): reads or assigns names to vector elements or table columns.
# c(): combines values into a vector, converting them to a common type when needed.
# data.frame(): builds a table whose named columns have compatible lengths.
# length(): counts elements in a vector or list.
# mean(): computes an average; logical TRUE/FALSE values contribute 1/0, giving a
 ↪  proportion.
# median(): returns the middle sorted value, averaging the middle pair for even sample
 ↪  sizes.
# sd(): computes sample standard deviation using denominator n-1.
# diff(): subtracts each preceding value from the next by default; diff(range(x)) gives
 ↪  max minus min.
# stopifnot(): stops execution if any supplied condition is FALSE; successful checks
 ↪  produce no output.
# sum(): adds vector entries; on logical values it counts TRUE entries.
# identical(): requires exact equality of values, types, and attributes.
# kable(): formats a data frame or matrix as a readable report table; digits sets
 ↪  displayed precision.
# split() separates prices by ordered rating; q6() is our type-6 quantile helper.
# lapply() calculates each group summary and do.call(rbind, ...) joins the rows.
groups <- split(restaurants$meal_price_usd, restaurants$quality_rating)
group_summary <- do.call(rbind, lapply(names(groups), function(g) {
  v <- groups[[g]]; q <- q6(v, c(.25,.75))
  data.frame(Quality = g, N = length(v), Mean = mean(v), Median = median(v),
             SD = sd(v), Q1 = q[1], Q3 = q[2], IQR = diff(q))
}))
stopifnot(sum(group_summary$N) == 300,
          identical(group_summary$N, c(84L,150L,66L)))
kable(group_summary[c("Quality","N","Mean","Median","SD")], digits = 2,
      col.names = c("Quality", "n", "Mean (USD)", "Median (USD)", "SD (USD)"))
```

Quality	n	Mean (USD)	Median (USD)	SD (USD)
Good	84	19.73	19.5	6.13
Very Good	150	25.63	25.0	8.30
Excellent	66	34.35	34.5	8.29

```r
kable(group_summary[c("Quality","Q1","Q3","IQR")], digits = 2,
      col.names = c("Quality", "Q1 (USD)", "Q3 (USD)", "IQR (USD)"))
```

Quality	Q1 (USD)	Q3 (USD)	IQR (USD)
Good	15.0	25	10.0
Very Good	20.0	32	12.0
Excellent	29.5	41	11.5

```r
# do.call(): calls a function using a list of arguments, e.g. rbind to combine a list of
 ↪  tables.
# lapply(): applies a function separately to each element and returns a list.
# names(): reads or assigns names to vector elements or table columns.
# factor(): stores categorical values; levels fixes their permitted order and labels can
 ↪  rename their display.
# data.frame(): builds a table whose named columns have compatible lengths.
# rep(): repeats values; times repeats a vector and each repeats individual elements.
# length(): counts elements in a vector or list.
# ggplot(): starts a plot from data; additional layers are joined with +.
# aes(): maps variables to visual properties such as x, y, fill, color, or group.
# geom_point(): draws a point for each row at its mapped x and y values.
# labs(): sets axis titles, legend titles, plot titles, and captions.
# box_record() supplies matching quartiles and whisker endpoints for every quality group.
# geom_boxplot(stat = "identity") draws them; separate points mark values beyond the
 ↪  fences.
```

```r
restaurant_boxes <- do.call(rbind, lapply(names(groups), function(g)
 ↪  box_record(groups[[g]], g)))
restaurant_boxes$Group <- factor(restaurant_boxes$Group, levels = quality_levels, ordered
 ↪  = TRUE)
flagged <- do.call(rbind, lapply(names(groups), function(g) {
  r <- box_record(groups[[g]], g); v <- groups[[g]]
  out <- v[v < r$Lower_fence | v > r$Upper_fence]
  data.frame(Group = rep(g, length(out)), Price = out)
}))
flagged$Group <- factor(flagged$Group, levels = quality_levels, ordered = TRUE)
ggplot(restaurant_boxes, aes(x = Group, ymin = ymin, lower = Q1, middle = Median,
                             upper = Q3, ymax = ymax)) +
  geom_boxplot(stat = "identity", width = .5, fill = "#B6D2E2", colour = "#394B58") +
  geom_point(data = flagged, aes(x = Group, y = Price), inherit.aes = FALSE,
             colour = "#B47816", size = 2.4) +
  labs(title = "Compare center and spread together", x = "Quality rating", y = "Meal price
   ↪  (USD)")
```

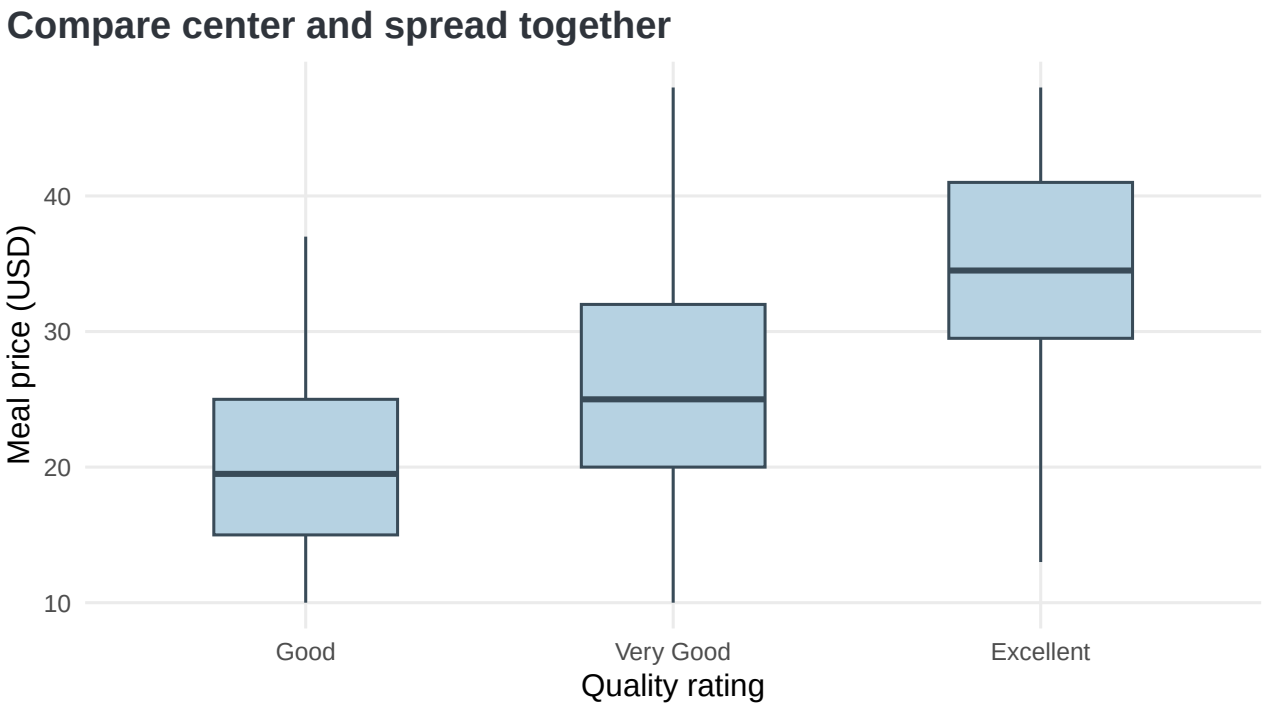


Figure 5: Original R boxplots. Quartiles and fences are explicitly calculated with the same type-6 convention as the summary table. Separate points are flagged observations.

Median prices are **USD 19.50, 25.00, and 34.50** in increasing quality order. The distributions overlap, so the category does not determine an individual restaurant's price. In these data, Good has the smallest IQR (**USD 10**); Very Good has the largest (**USD 12**). "Higher typical price" and "greater price variability" are different questions.

Summary: match the measure to the question

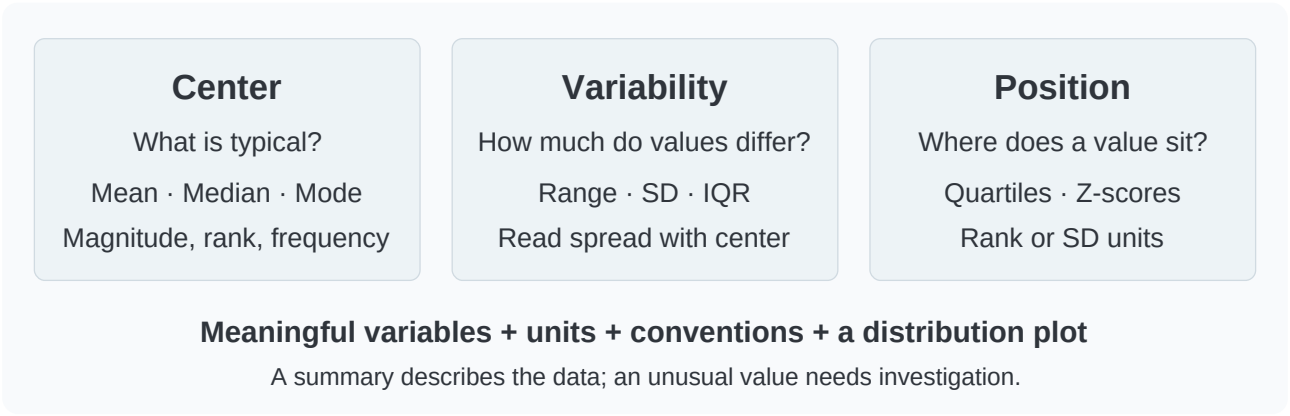


Figure 6: Original course summary of center, variability, and position.

Question	Useful starting measures	Read alongside
What is the numerical average?	Mean	Distribution and sample SD
Where is the middle by rank?	Median	Quartiles, IQR, and shape
Which value or category occurs most?	Mode	Frequencies and any ties
How widely do values vary?	Range, mean absolute deviation, SD, IQR	Their different sensitivities and units
Where does a value sit?	Percentile or z-score	The convention and distribution
Do groups differ descriptively?	Group sizes, centers, and spreads	Aligned plots and study context

Limitation	Useful companion	Remaining limitation
The mean can be pulled by extremes	Median and a distribution plot	The median alone hides spread
Center alone hides variability	A spread measure and plot	Different spread measures describe different features
Range uses only the endpoints	Mean absolute deviation or SD	These still respond to extremes
SD is sensitive to extremes	IQR alongside the median	IQR describes the middle half and leaves tail detail out

Do not compute arithmetic summaries of arbitrary category codes. Do not treat an outlier flag as an explanation. Preserve valid observations, report units and conventions, and connect each number to the business question.

Complete Lesson 3 practice. Continue to Lesson 4: Relationships between variables for crosstabs, conditional percentages, scatterplots, covariance, and correlation.

Sources and data

- CSV bundle and data guide.

- Anderson et al. (2020), Essentials of Modern Business Statistics with Microsoft Excel, 8th ed.: §3.1, pp.105–115; §3.2, pp.121–126; z-scores in §3.3, p.131; five-number summaries and boxplots, §3.4, pp.138–140.
- Audit values match all 20 printed observations in Table 2.4. Restaurant matches the first ten printed rows and all twelve crosstab cells in Tables 2.9–2.10; remaining individual rows are not printed for comparison. The numerical summaries and figures here are original course calculations from these checked CSVs, not claims that each output is printed in Chapter 3.
- Instructor-provided 01-descriptive-statistics.pdf, pp.51 and 60–63 (original Part 1 Descriptive Statistics deck slides 38 and 44–47): extreme dollar value, equal means with different spreads, range sensitivity, and absolute and squared deviations. Adapted explanations and tables; the A/B equal-range comparison is an original course illustration. Example data and provenance.

Module 2: Numerical Measures — Practice

Lesson 3 | Calculate, compare, and interpret

Table of contents

Preparation 1
1. Describe the center 1
2. Quantify variability 2
3. Quartiles and a boxplot 2
4. Sensitivity and relative measures 2
5. Return to the restaurant question 2
6. Explain the limitations of a summary 3
Reporting check 3

Preparation

Read Lesson 3. Use the Audit and Restaurant files in the CSV bundle. Submit a concise Quarto report with reproducible calculations, units, and interpretations.

Use the book's **type-6 percentile convention** throughout. Preserve the original CSV values. If an exercise changes an observation, create a separate hypothetical copy and label it clearly.

```
# library(): loads a package so its functions can be used in this session.
# read.csv() imports a CSV into a data frame with named columns.
# The relative path starts at the course project folder; the source CSV is not modified.
library(ggplot2)
audit <- read.csv("data/module02/audit_times.csv")
restaurants <- read.csv("data/module02/restaurant_ratings.csv")
x <- audit$audit_days
```

1. Describe the center

1. Calculate the number of audits, total recorded duration, mean, and median.
2. Show which sorted positions determine the median.
3. Identify the statistical mode using value frequencies. Explain why `mode(x)` does not answer that question in R.
4. Write a sentence explaining the mean and median to an audit manager. Explain why neither number guarantees a future completion date.

2. Quantify variability

1. Calculate minimum, maximum, and range.
2. Calculate the sum of squared deviations from the sample mean, then divide by `n - 1` to obtain the sample variance. Check against `var(x)`.
3. Take its square root and check against `sd(x)`.
4. Give the units of the range, variance, and standard deviation.
5. Explain why dividing by n would produce a different descriptive quantity and why having all 20 CSV rows does not automatically make them the full population of interest.

3. Quartiles and a boxplot

1. Calculate Q_1, median, and Q_3 with `quantile(x, c(.25,.5,.75), type = 6)`.
2. Calculate IQR and the five-number summary.
3. Calculate both 1.5-IQR fences and identify any flagged values.
4. Identify the actual whisker endpoints. Explain how they differ from the fences and the overall maximum.
5. Draw a boxplot with those quartiles and whiskers. Label its units and state the convention. If using a software default, check that its quartiles match your stated method for these data.
6. State what you would investigate before deciding whether a flagged observation is erroneous.

4. Sensitivity and relative measures

1. Replace only the largest audit duration with **60 days in a separate copy**.
2. Compare mean, median, sample SD, range, and IQR before and after the hypothetical change.
3. Explain which measures changed most and why. Do not treat the hypothetical dataset as a corrected source file.
4. Calculate the CV for the original audits and interpret it.
5. Calculate the sample z-score of the original 33-day audit. Explain why it may be flagged by the IQR rule while having a z-score smaller than 3.
6. Explain why a CV for Celsius temperature is not meaningful in the same way as a CV for elapsed duration.

5. Return to the restaurant question

For each quality category, calculate group size, mean, median, sample SD, Q_1, Q_3, and IQR. Preserve the order Good, Very Good, Excellent.

1. Which group has the highest median price? Which has the smallest IQR?
2. Calculate the difference between the Excellent and Good median prices, in USD. Describe it as a difference in group medians, not as an individual restaurant's price change.
3. Compare the groups with side-by-side boxplots, using the same quartile convention as the table.
4. Write a short interpretation of center, spread, and overlap. Explain why these data do not prove that changing a rating causes the price to change.

6. Explain the limitations of a summary

Use the teaching-comparisons CSV, separate from the textbook bundle.

1. For the seven dollar values (`example == "extreme_value"`), calculate the mean and median. Which better describes the middle of the observations? When would the mean still be useful? Explain what information is lost when these seven observations are compressed into a single mean. Does the mode answer the same question as the median?
2. For S1–S5 (`example == "same_mean"`), verify that every mean is 200 and calculate each range. Explain what the mean alone conceals.
3. For A and B (`example == "same_range"`), calculate mean, range, mean absolute deviation about the mean, and sample SD. Explain how two sets with identical means and ranges can differ internally.
4. For the seven dollar amounts, calculate the range and explain why USD 99 does not describe the spread of most amounts. If only USD 100 became USD 1000 in a hypothetical copy, what would the range be? What would remain unchanged about the other six amounts?
5. Explain why replacing range with SD does not remove sensitivity to extreme values. What additional information do median, IQR, and a plot provide?

Reporting check

Every table and chart should make the units and group sizes clear. Use English for statistical explanations. Arabic narration is reserved for introducing business cases; it is not required for your calculation answers.

Return to Lesson 3.

Module 2: Relationships Between Variables

Lesson 4 | Compare proportions and describe association

Table of contents

What we will learn . . . 1
Choose tools for the variable pair . . . 1
Business case: quality ratings and meal-price bands . . . 2
Crosstabs: count combinations . . . 2
Business case: commercials and weekly electronics sales . . . 6
Scatterplots: inspect the paired pattern . . . 7
Covariance: do deviations move together? . . . 8
Correlation: standardize the linear association . . . 9
Why a correlation coefficient needs its plot . . . 11
Summary: a defensible relationship description . . . 12
Sources and data . . . 13

Reduced teaching slides (PDF) · Module overview

This extended lesson provides the full explanation and reproducible examples behind the slides.

What we will learn

Lessons 1–3 describe variables individually and compare numerical distributions across groups. We now ask how **two variables relate within the same observations**.

By the end, you should be able to:

- Construct a crosstab and distinguish joint, marginal, and conditional percentages.
- Choose a denominator that answers the stated question.
- Read a scatterplot for direction, form, strength, and unusual observations.
- Calculate sample covariance and Pearson correlation, with their units and limitations.
- Describe association without claiming causation.

Reading: main 8th-edition textbook, §2.3, pp.65–69 (crosstabs and percentages), §2.4, pp.75–80 (scatterplots and comparative charts), and §3.5, pp.144–152 (covariance and correlation). These are selected descriptive topics, rather than every procedure in those sections.

Choose tools for the variable pair

Variable pair	Useful display	Numerical summary
Two categorical variables	Crosstab, grouped bars, or percentage bars	Counts and conditional percentages
Categorical and numerical	Side-by-side boxplots or aligned distribution plots	Group sizes, centers, and spreads
Two numerical variables	Scatterplot	Covariance and correlation when appropriate

A numerical variable can be grouped into classes for a crosstab. This simplifies comparisons but loses within-class detail. Keep the original values for numerical analysis. The Restaurant boxplots in Lesson 3 and the price-band tables below answer related questions at different resolutions.

Business case: quality ratings and meal-price bands

An analyst studies **300 Los Angeles restaurants** from the book's historical teaching dataset. Each restaurant has an ordered quality rating and a meal price in US dollars. The analyst asks: **Within each quality category, how are restaurants distributed across price bands?** A second question reverses the conditioning: **Within a price band, how are quality ratings distributed?** These questions use the same records but different denominators.

يدرس محلّل بيانات **ثلاثمئة مطعم في لوس أنجلوس** من بيانات الكتاب التعليمية التاريخية. لكل مطعم تقييم جودة مرتب وسعر وجبة بالدولار الأمريكي. يسأل المحلّل: **داخل كل فئة من فئات الجودة، كيف تتوزع المطاعم على فئات الأسعار؟** ثم يعكس السؤال: **داخل فئة سعرية محددة، كيف تتوزع تقييمات الجودة؟** يستخدم السؤالان السجلات نفسها، لكن مقام حساب النسبة يختلف بينهما.

Download Restaurant CSV. One row is one restaurant. The four bands are **[10,20), [20,30), [30,40), [40,50)** USD: include the lower boundary and exclude the upper one. A price of USD 20 belongs to the second band. For these whole-dollar observations, these correspond to the book's USD 10–19, 20–29, 30–39, and 40–49 classes.

Crosstabs: count combinations

A **crosstabulation** counts observations classified by two variables. Each internal cell describes a combination. **Marginal totals** describe each variable separately.

```
# library() loads ggplot2 for charts and knitr for table formatting.
# read.csv() reads the checked Restaurant teaching CSV.
# factor()/c() retain the ordered ratings; levels fixes the category order.
library(ggplot2)
library(knitr)
restaurants <- read.csv("data/module02/restaurant_ratings.csv")
restaurants$quality_rating <- factor(restaurants$quality_rating,
  levels = c("Good", "Very Good", "Excellent"), ordered = TRUE)
```

```
# c() supplies band labels; seq()/cut() form the corresponding 10-USD intervals.
# right = FALSE means [lower, upper); labels changes display text, not boundaries.
# table() counts quality/band pairs; addmargins() adds row and column totals.
price_labels <- c("10–<20", "20–<30", "30–<40", "40–<50")
restaurants$price_band <- cut(restaurants$meal_price_usd,
  breaks = seq(10, 50, 10), right = FALSE, labels = price_labels)
counts <- table(Quality = restaurants$quality_rating,
  Price = restaurants$price_band)
addmargins(counts)
```

```
           Price
Quality     10–<20 20–<30 30–<40 40–<50 Sum
  Good          42     40      2      0  84
  Very Good     34     64     46      6 150
  Excellent      2     14     28     22  66
  Sum           78    118     76     28 300
```

```r
# addmargins() adds row totals, column totals, and the grand total to the crosstab.
# kable() displays counts, not percentages; each internal cell counts one combination.
# stopifnot()/all() check every cell against the book counts; sum() checks the total.
stopifnot(all(counts == expected_counts), sum(counts) == 300)
kable(addmargins(counts), caption = "Restaurants by quality and meal-price band (USD).
↪  Sum denotes totals.")
```

Table 1: Restaurants by quality and meal-price band (USD). Sum denotes totals.

	10–<20	20–<30	30–<40	40–<50	Sum
Good	42	40	2	0	84
Very Good	34	64	46	6	150
Excellent	2	14	28	22	66
Sum	78	118	76	28	300

The cell **64** represents Very Good restaurants priced from USD 20 up to, but below, USD 30. The Very Good row total is **150**, the second price-band column total is **118**, and the grand total is **300**. A restaurant contributes to exactly one internal cell.

Joint, marginal, and conditional percentages

The same count can answer different questions:

Question	Denominator	Calculation	Interpretation
What share of all restaurants are Very Good **and** in the second price band?	All 300 restaurants	$64/300 \times 100 = 21.33\%$	Joint percentage
What share of all restaurants are Very Good?	All 300 restaurants	$150/300 \times 100 = 50\%$	Marginal percentage
Among Very Good restaurants, what share are in the second price band?	The 150 Very Good restaurants	$64/150 \times 100 = 42.67\%$	Conditional row percentage
Among restaurants in the second price band, what share are Very Good?	The 118 restaurants in that band	$64/118 \times 100 = 54.24\%$	Conditional column percentage

Say “among whom?” before calculating a percentage. A percentage without its reference group can answer the wrong question even when the arithmetic is correct.

Row percentages: price distribution within quality

```
# prop.table() divides each cell by the indicated margin total.
# margin = 1 uses each quality row; multiply by 100 to give row percentages.
row_pct <- 100 * prop.table(counts, margin = 1)
row_pct
```

```
           Price
Quality         10–<20    20–<30    30–<40    40–<50
  Good       50.000000 47.619048  2.380952  0.000000
  Very Good  22.666667 42.666667 30.666667  4.000000
  Excellent   3.030303 21.212121 42.424242 33.333333
```

```
# kable(): formats a data frame or matrix as a readable report table; digits sets
 ↪  displayed precision.
# rowSums(): sums each row separately.
# row_pct was calculated with prop.table(counts, margin = 1): divide by each row total.
# cbind() adds the percentage totals; digits = 1 rounds only the displayed values.
kable(cbind(row_pct, Total = rowSums(row_pct)), digits = 1,
      caption = "Within each quality category: percentage in each price band. Each row
       ↪  totals 100% before rounding.")
```

Table 2: Within each quality category: percentage in each price band. Each row totals 100% before rounding.

	10–<20	20–<30	30–<40	40–<50	Total
Good	50.0	47.6	2.4	0.0	100
Very Good	22.7	42.7	30.7	4.0	100
Excellent	3.0	21.2	42.4	33.3	100

For Good restaurants, **50.0%** fall in the lowest band. For Excellent restaurants, **75.8%** cost at least USD 30: $(28 + 22)/66 \times 100$. The comparable share for Good is **2.4%**: $(2 + 0)/84 \times 100$. These distributions differ substantially in this sample.

The groups contain 84, 150, and 66 restaurants. Conditional percentages make their distributions comparable despite unequal group sizes. Counts should still accompany the comparison.

```
# factor(): stores categorical values; levels fixes their permitted order and labels can
 ↪  rename their display.
# ifelse(): chooses a value element by element according to a logical condition.
# sprintf(): formats text using placeholders such as %.2f for two decimal places.
# paste0(): joins text with no separator; collapse can combine a vector into one string.
# rowSums(): sums each row separately.
# ggplot(): starts a plot from data; additional layers are joined with +.
# aes(): maps variables to visual properties such as x, y, fill, color, or group.
# geom_col(): draws bars at supplied heights; it does not count records.
# geom_text(): draws labels from mapped data; position and hjust/vjust control placement.
# scale_fill_manual(): assigns explicit fill colors to categories; named values preserve
 ↪  category-color mappings.
# c(): combines values into a vector, converting them to a common type when needed.
# scale_x_discrete(): controls category order, labels, and spacing on the x-axis.
# scale_y_continuous(): controls numeric y-axis breaks, labels, limits, and spacing.
# seq(): creates an equally spaced sequence; by sets spacing and length.out sets the
 ↪  number of values.
# labs(): sets axis titles, legend titles, plot titles, and captions.
# theme(): customizes non-data plot elements such as legends, grids, and axis text.
# as.data.frame() reshapes the percentage table into one row per quality-price
 ↪  combination.
# position_stack() places segments and labels within each 100% quality-group bar.
```

```r
bar_data <- as.data.frame(row_pct, responseName = "Percent")
bar_data$Quality <- factor(bar_data$Quality, levels = quality_levels)
bar_data$Price <- factor(bar_data$Price, levels = price_labels)
bar_data$Label <- ifelse(bar_data$Percent >= 8, sprintf("%.1f%%",bar_data$Percent), "")
quality_axis <- paste0(quality_levels, "\n(n = ", rowSums(counts), ")")
ggplot(bar_data, aes(Quality, Percent, fill = Price)) +
  geom_col(position = position_stack(reverse = TRUE), width = .62,
           colour = "white", linewidth = .4) +
  geom_text(aes(label = Label), position = position_stack(vjust = .5, reverse = TRUE),
            colour = "#22313C", size = 3.8) +
  scale_fill_manual(values = c("#DCE8EF", "#ACC9DA", "#E8CFA5", "#C9D8BD"), drop = FALSE)
   ↪  +
  scale_x_discrete(labels = quality_axis) +
  scale_y_continuous(breaks = seq(0,100,25), limits = c(0,100)) +
  labs(title = "Price distributions differ across quality categories",
       x = "Quality rating", y = "Within-group percentage", fill = "Meal price (USD)") +
  theme(legend.position = "bottom")
```

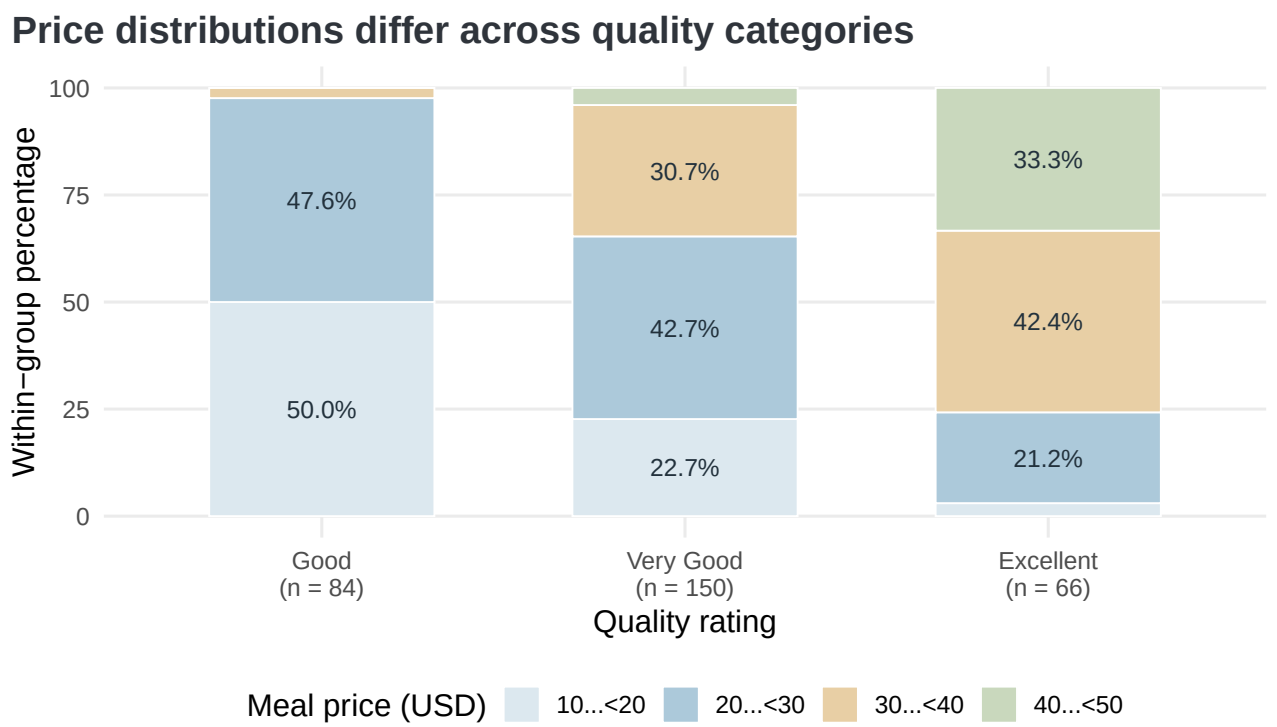


Figure 1: Original R chart from the checked Restaurant CSV. Every bar represents 100% of its quality group. Segment labels are percentages; n gives group size.

Small segments are readable in the table even when too narrow for a label. Middle segments in stacked bars have different baselines, so use the table for precise comparisons.

Column percentages: quality distribution within price

```r
# prop.table() expresses cells relative to a specified margin total.
# margin = 2 uses each price-band column; multiply by 100 for percentages.
column_pct <- 100 * prop.table(counts, margin = 2)
column_pct
```

```
            Price
```

```
Quality        10–<20    20–<30    30–<40    40–<50
  Good      53.846154 33.898305  2.631579  0.000000
  Very Good 43.589744 54.237288 60.526316 21.428571
  Excellent  2.564103 11.864407 36.842105 78.571429
```

```
# kable(): formats a data frame or matrix as a readable report table; digits sets
 ↪  displayed precision.
# colSums(): sums each column separately.
# column_pct uses prop.table(counts, margin = 2): divide by each column total.
# rbind() adds a totals row so each price-band distribution can be checked against 100%.
kable(rbind(column_pct, Total = colSums(column_pct)), digits = 1,
      caption = "Within each price band: percentage in each quality category. Each column
       ↪  totals 100% before rounding.")
```

Table 3: Within each price band: percentage in each quality category. Each column totals 100% before rounding.

	10–<20	20–<30	30–<40	40–<50
Good	53.8	33.9	2.6	0.0
Very Good	43.6	54.2	60.5	21.4
Excellent	2.6	11.9	36.8	78.6
Total	100.0	100.0	100.0	100.0

Among the **28 restaurants in the highest band**, **22 are Excellent**, or **78.6%**. Among the **66 Excellent restaurants**, **22 are in the highest band**, or **33.3%**. Reversing the conditioning changes the denominator and the answer.

```
# stopifnot(): stops execution if any supplied condition is FALSE; successful checks
 ↪  produce no output.
# abs(): returns absolute values; a small tolerance in an equality check permits
 ↪  floating-point rounding.
# Index counts and percentages by quality row and price-band column.
# stopifnot() stops execution if the count or either conditioning denominator is
 ↪  incorrect.
stopifnot(counts["Very Good",2] == 64,
          abs(row_pct["Very Good",2]-100*64/150) < 1e-10,
          abs(column_pct["Very Good",2]-100*64/118) < 1e-10)
```

> **i** Calculate from counts, then round
>
> Our percentages come directly from the checked counts. Round only for display. Independently rounded cells may not total exactly 100%. The book's Table 2.11 prints some cells differently (for example 33.4% for 22/66); our calculated value rounds to 33.3%. We retain the observations and reproducible arithmetic.

Crosstabs show an association between quality and price bands here. They do not show that raising a price improves quality. Grouping boundaries can affect the display, and combining subgroups can hide or reverse patterns. The book develops this aggregation issue as Simpson's paradox in §2.3. A full subgroup analysis would require relevant subgroup data, which this Restaurant file does not contain.

Business case: commercials and weekly electronics sales

A San Francisco electronics store records **10 weeks** of television-commercial counts and sales. One row pairs the commercials and sales **from the same week**. Sales are recorded in **hundreds**

of US dollars: a value of 50 means USD 5,000. The question is: **Do weeks with more commercials tend to have higher sales?** These historical observations can describe an association, but they do not isolate the effect of advertising.

يسجّل متجر إلكترونيات في سان فرانسيسكو عدد الإعلانات التلفزيونية والمبيعات خلال **عشرة أسابيع**. يجمع كل صف عدد الإعلانات والمبيعات **للأسبوع نفسه**. سُجّلت المبيعات **بمئات الدولارات الأمريكية**؛ فالقيمة 50 تعني خمسة آلاف دولار. والسؤال هو: **هل تميل الأسابيع التي تضم إعلانات أكثر إلى تسجيل مبيعات أعلى؟** تصف هذه المشاهدات التاريخية ارتباطًا، لكنها لا تعزل أثر الإعلان في المبيعات.

Download Electronics CSV. The original workbook is named Stereo.xls; all ten observations match the book's Electronics table. `week` identifies the observation, while `commercials` and `sales_hundreds_usd` form our variable pair.

```
# library() loads ggplot2 for charts.
# read.csv() loads ten paired weeks from the checked Electronics teaching CSV.
# The sales_hundreds_usd column is in hundreds of USD; keep the weekly pairs intact.
library(ggplot2)
electronics <- read.csv("data/module02/electronics_sales.csv")
```

```
# c(): combines values into a vector, converting them to a common type when needed.
# kable() displays each week together with its commercials and sales observations.
# col.names changes the visible headings, including the sales unit, without changing the
 ↪  data.
kable(electronics, col.names = c("Week", "Commercials", "Sales (hundreds of USD)"))
```

Week	Commercials	Sales (hundreds of USD)
1	2	50
2	5	57
3	1	41
4	3	54
5	4	54
6	1	38
7	5	63
8	3	48
9	4	59
10	2	46

Scatterplots: inspect the paired pattern

A **scatterplot** places one numerical variable on each axis and one point for each paired observation. Keep each week's pair together. Sorting the two columns independently destroys the observed relationship.

```
# ggplot()/aes() map paired variables; geom_point() draws weeks and geom_text() labels
 ↪  them.
# nudge_y moves labels; scale_x_continuous() marks integer commercial counts.
# scale_y_continuous()/expansion()/c() add proportional space; labs() labels sales units.
ggplot(electronics, aes(commercials, sales_hundreds_usd)) +
  geom_point(size = 3.2, colour = "#2878A5") +
  geom_text(aes(label = week), nudge_y = 1, size = 3.5, colour = "#56616D") +
  scale_x_continuous(breaks = 1:5) +
  scale_y_continuous(expand = expansion(mult = c(.06,.12))) +
  labs(title = "More commercials accompany higher weekly sales",
       x = "Number of television commercials",
       y = "Weekly sales (hundreds of USD)")
```

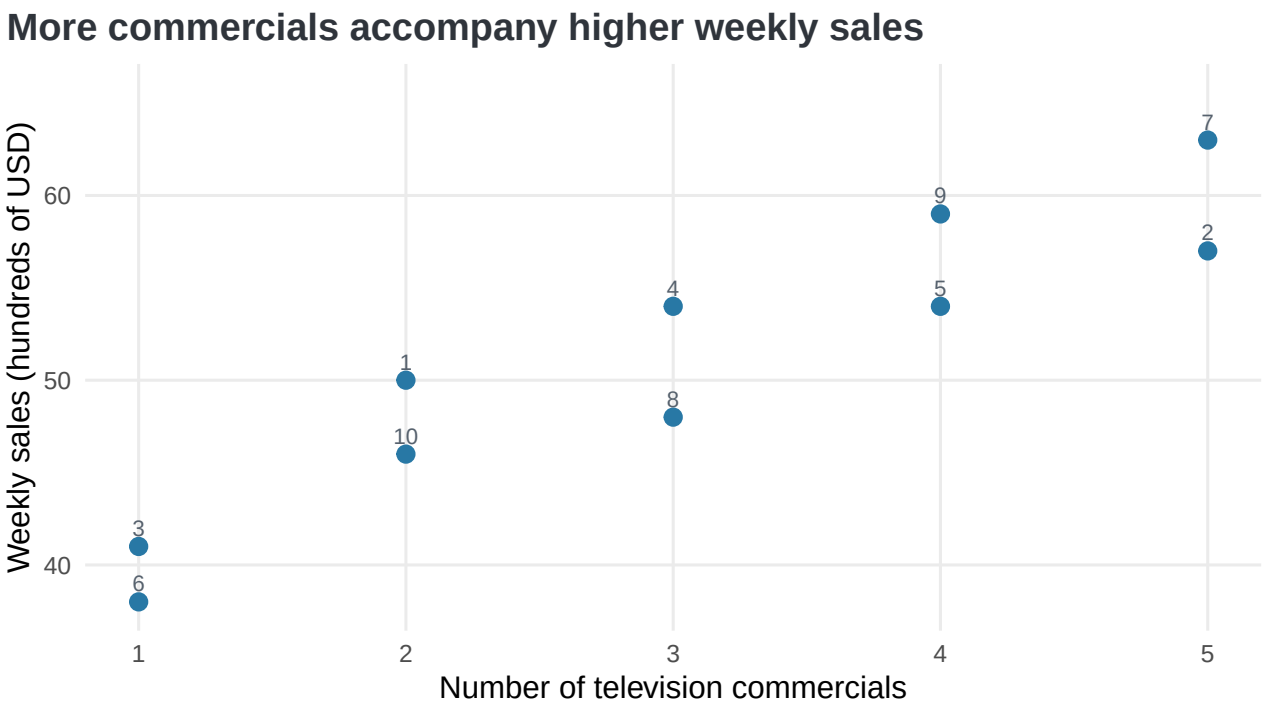


Figure 2: Original R scatterplot of the ten checked Electronics observations. Labels identify weeks. No fitted regression model is required for this descriptive display.

Read four features before calculating a coefficient:

- **Direction:** as commercials increase, sales tend to increase. This is a positive association.
- **Form:** the pattern is approximately straight rather than clearly curved.
- **Strength:** points cluster fairly closely around an upward linear pattern, with variation at each commercial count.
- **Unusual observations:** look for isolated points and values far from the overall pattern. The plot alone cannot establish whether a record is erroneous.

The count is on the horizontal axis because it is the proposed explanatory variable for this question. That placement does not establish cause and effect. We leave connecting lines out because our immediate question concerns the paired relationship, rather than a chronological sales path.

Covariance: do deviations move together?

The **sample covariance** is

$$s_{xy} = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{n - 1}.$$

When both observations are above their means, or both below, their product of deviations is positive. Opposite signs give a negative product. Summing these products describes whether deviations tend to move together.

```
# $ selects paired columns; x is commercials and y is sales in hundreds of USD.
# cov() uses the sample denominator n - 1; preserve each week's pairing.
x <- electronics$commercials
y <- electronics$sales_hundreds_usd
cov(x, y)
```

```
[1] 11
```

```r
# data.frame(): builds a table whose named columns have compatible lengths.
# mean(): computes an average; logical TRUE/FALSE values contribute 1/0, giving a
 ↪  proportion.
# kable(): formats a data frame or matrix as a readable report table; digits sets
 ↪  displayed precision.
# stopifnot(): stops execution if any supplied condition is FALSE; successful checks
 ↪  produce no output.
# Subtract each sample mean and multiply the paired deviations week by week.
# sum() combines the cross-products; cov() divides their sum by n-1 for sample covariance.
x <- electronics$commercials
y <- electronics$sales_hundreds_usd
products <- data.frame(Week = electronics$week, x = x, y = y,
                       x_deviation = x-mean(x), y_deviation = y-mean(y),
                       Product = (x-mean(x))*(y-mean(y)))
kable(products, digits = 1)
```

Week	x	y	x_deviation	y_deviation	Product
1	2	50	-1	-1	1
2	5	57	2	6	12
3	1	41	-2	-10	20
4	3	54	0	3	0
5	4	54	1	3	3
6	1	38	-2	-13	26
7	5	63	2	12	24
8	3	48	0	-3	0
9	4	59	1	8	8
10	2	46	-1	-5	5

```r
stopifnot(mean(x) == 3, mean(y) == 51, sum(products$Product) == 99,
          cov(x,y) == 11)
```

Here $\bar{x} = 3$, $\bar{y} = 51$, and the sum of products is 99. Thus $s_{xy} = 99/9 = 11$, in **commercials × hundreds of USD**. Positive covariance agrees with the upward scatterplot.

Covariance depends on measurement units. Its sign helps describe direction, but its magnitude alone cannot establish that an association is strong. For a complete population, the population covariance uses population means and denominator N. R's `cov()` uses the sample denominator $n - 1$.

Correlation: standardize the linear association

The **Pearson sample correlation** divides covariance by the sample standard deviations:

$$r = \frac{s_{xy}}{s_x s_y}.$$

It requires paired numerical observations with nonzero variation in both variables. It is **unitless**, ranges from **−1 to +1**, and describes **linear** association:

Value	Meaning
$r = +1$	All points lie on a straight line with positive slope
$r = -1$	All points lie on a straight line with negative slope

Value	Meaning
r near zero	Little linear association; a nonlinear relationship may remain
Larger $\lvert r \rvert$	Stronger linear association, interpreted with the plot and context

```
# cor() defaults to Pearson's linear correlation for the paired vectors x and y.
# The result is unitless; both vectors require nonzero variation.
cor(x, y)
```

```
[1] 0.9304906
```

```
# kable(): formats a data frame or matrix as a readable report table; digits sets
 ↪  displayed precision.
# data.frame(): builds a table whose named columns have compatible lengths.
# c(): combines values into a vector, converting them to a common type when needed.
# stopifnot(): stops execution if any supplied condition is FALSE; successful checks
 ↪  produce no output.
# abs(): returns absolute values; a small tolerance in an equality check permits
 ↪  floating-point rounding.
# cor() calculates Pearson linear correlation from the paired observations.
# Dividing cov(x,y) by sd(x)*sd(y) checks the same unitless calculation explicitly.
r <- cor(x,y)
kable(data.frame(Quantity = c("Sample SD: commercials", "Sample SD: sales (hundreds of
 ↪  USD)",
                              "Sample covariance", "Pearson correlation"),
                 Value = c(sd(x),sd(y),cov(x,y),r)), digits = 4)
```

Quantity	Value
Sample SD: commercials	1.4907
Sample SD: sales (hundreds of USD)	7.9303
Sample covariance	11.0000
Pearson correlation	0.9305

```
stopifnot(abs(r-cov(x,y)/(sd(x)*sd(y))) < 1e-10,
          abs(r-0.9304906) < 1e-7)
```

Interpretation: across these ten weeks, commercials and sales have a strong positive linear association, $r \approx 0.9305$, consistent with the book's rounded value 0.93. This does not mean "sales increase by 93%," and correlation does not give a change in sales per additional commercial. Estimating such a change requires a model, developed later in regression.

Changing units: covariance changes, correlation stays

```
# data.frame(): builds a table whose named columns have compatible lengths.
# c(): combines values into a vector, converting them to a common type when needed.
# kable(): formats a data frame or matrix as a readable report table; digits sets
 ↪  displayed precision.
# stopifnot(): stops execution if any supplied condition is FALSE; successful checks
 ↪  produce no output.
# abs(): returns absolute values; a small tolerance in an equality check permits
 ↪  floating-point rounding.
# Multiply recorded sales by 100 to convert hundreds of USD into USD.
# cov() scales by the same factor; cor() is unchanged under this positive unit conversion.
```

```r
y_usd <- 100*y
units <- data.frame(Sales_unit = c("Hundreds of USD", "USD"),
                    Sample_covariance = c(cov(x,y),cov(x,y_usd)),
                    Correlation = c(cor(x,y),cor(x,y_usd)))
kable(units, digits = 4)
```

Sales_unit	Sample_covariance	Correlation
Hundreds of USD	11	0.9305
USD	1100	0.9305

```r
stopifnot(abs(cov(x,y_usd)-100*cov(x,y)) < 1e-10,
          abs(cor(x,y_usd)-cor(x,y)) < 1e-10)
```

Converting sales to USD changes covariance from **11 to 1100**, but the correlation stays **0.9305**. The underlying relationship has not changed. Positive changes of measurement units and shifts of origin leave Pearson correlation unchanged. Correlation also treats x and y symmetrically: `cor(x,y)` equals `cor(y,x)`.

Why a correlation coefficient needs its plot

The following are **artificial teaching patterns**, not Electronics data. The curved pattern uses $y = x^2$ for $x = -4, -3, \ldots, 4$.

```r
# data.frame(): builds a table whose named columns have compatible lengths.
# factor(): stores categorical values; levels fixes their permitted order and labels can
 ↪  rename their display.
# c(): combines values into a vector, converting them to a common type when needed.
# stopifnot(): stops execution if any supplied condition is FALSE; successful checks
 ↪  produce no output.
# abs(): returns absolute values; a small tolerance in an equality check permits
 ↪  floating-point rounding.
# ggplot(): starts a plot from data; additional layers are joined with +.
# aes(): maps variables to visual properties such as x, y, fill, color, or group.
# geom_point(): draws a point for each row at its mapped x and y values.
# labs(): sets axis titles, legend titles, plot titles, and captions.
# rbind() combines three explicitly artificial paired patterns: y=x, y=-x, and y=x^2.
# facet_wrap() separates the plots; cor() verifies that a curve can have zero linear
 ↪  correlation.
u <- -4:4
patterns <- rbind(data.frame(Pattern = "Positive line: r = 1", x = u, y = u),
                  data.frame(Pattern = "Negative line: r = -1", x = u, y = -u),
                  data.frame(Pattern = "Curve: r = 0", x = u, y = u^2))
patterns$Pattern <- factor(patterns$Pattern,
  levels = c("Positive line: r = 1", "Negative line: r = -1", "Curve: r = 0"))
stopifnot(cor(u,u) == 1, cor(u,-u) == -1, abs(cor(u,u^2)) < 1e-10)
ggplot(patterns, aes(x,y)) +
  geom_point(size = 2.5, colour = "#2878A5") +
  facet_wrap(~Pattern, nrow = 1, scales = "free_y") +
  labs(title = "Zero correlation can conceal a clear relationship",
       x = "x (arbitrary units)", y = "y (arbitrary units)")
```

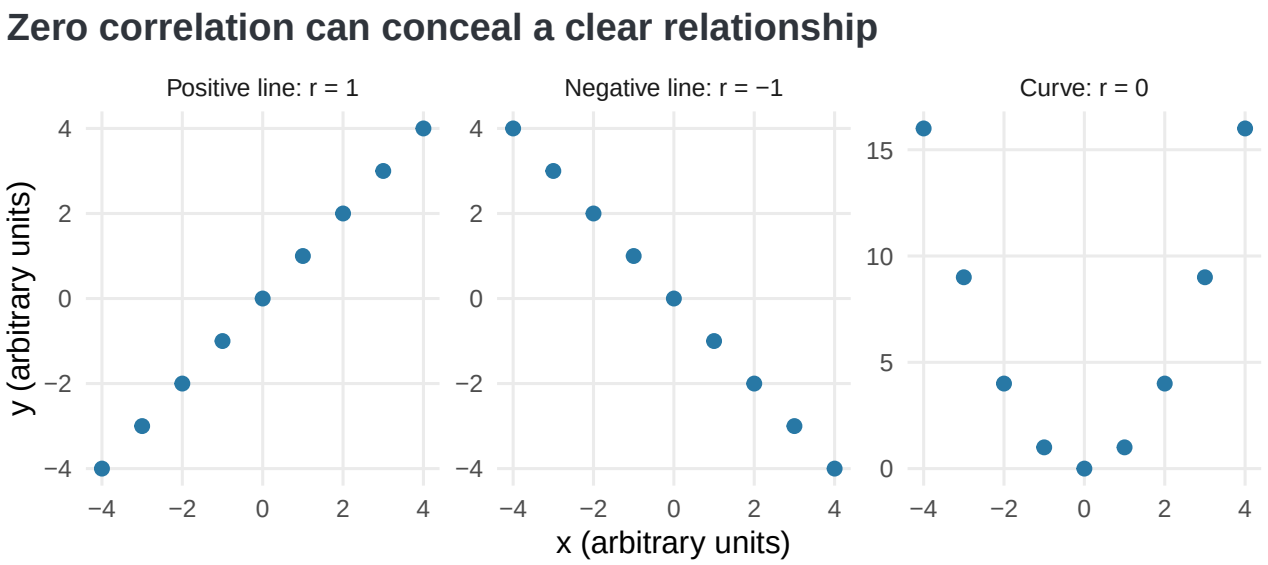


Figure 3: Original artificial examples. Pearson correlation captures straight-line direction but can be zero despite a clear curved relationship. Panel axes use arbitrary units.

Keep these limitations in view:

- **Nonlinearity:** $r = 0$ does not establish that variables are unrelated or independent.
- **Influential observations:** an isolated observation can substantially change correlation. Investigate it and compare results transparently; preserve valid records.
- **Restricted range and mixed groups:** a narrow observed range or a mixture of subgroups can change the apparent association. Inspect context and subgroup information when available.
- **Causation:** seasonal demand, promotions, or other unmeasured factors might affect both advertising and sales. These are possible explanations to investigate, not recorded facts about this store.
- **Scope:** ten observed weeks do not establish a stable relationship for all future weeks. Repeated weeks may also be dependent; calculation alone does not justify inferential claims.

Computing Pearson correlation requires no normality assumption by itself. Hypothesis tests and confidence intervals for association require additional assumptions and belong to later modules.

Summary: a defensible relationship description

Task	What to report	Essential check
Compare categorical distributions	Counts, group sizes, and conditional percentages	State the conditioning group and denominator
Compare numerical distributions by group	Group centers, spreads, and aligned plots	Use meaningful units and the same conventions
Describe two numerical variables	Scatterplot and, when suitable, covariance and correlation	Keep pairs intact; check shape and influential observations
Explain the result	An association in the observed context	Separate descriptive evidence from causal claims

For Restaurant, conditional price distributions differ across ratings. For Electronics, weekly commercials and sales show a strong positive linear association. Neither result establishes a causal effect.

Complete Lesson 4 practice, then complete the integrated module review.

Sources and data

- Anderson et al. (2020), Essentials of Modern Business Statistics with Microsoft Excel, 8th ed.: §2.3, pp.65–69, Tables 2.9–2.11; §2.4, pp.75–80, Table 2.14; §3.5, pp.144–152, Table 3.6 and the covariance/correlation example.
- Restaurant CSV: first ten displayed records and all twelve cells of Table 2.10 match; remaining individual records are not printed for comparison.
- Electronics CSV: all ten paired observations match Table 2.14 and the repeated example in Table 3.6. R calculations reproduce covariance 11 and correlation rounded to 0.93.
- All charts and numerical tables here are original R outputs from these CSVs, except the explicitly artificial correlation-pattern illustration. Data guide.

Module 2: Relationships Between Variables — Practice

Lesson 4 | State the denominator and interpret the pattern

Table of contents

Preparation 1
1. Construct and check the crosstab 1
2. Choose the denominator 1
3. Inspect paired numerical observations 2
4. Calculate and interpret association 2
5. Write a responsible conclusion 2

Preparation

Read Lesson 4. Use Restaurant and Electronics from the checked CSV bundle. Work in the `20261` project and submit a short Quarto report with reproducible code and interpretations.

```
# read.csv() imports a CSV into a data frame with named columns.
# The relative path starts at the course project folder; the source CSV is not modified.
restaurants <- read.csv("data/module02/restaurant_ratings.csv")
electronics <- read.csv("data/module02/electronics_sales.csv")
```

1. Construct and check the crosstab

1. Identify the observational unit and classify the two Restaurant variables.
2. Group prices into [10,20), [20,30), [30,40), [40,50) USD. Explain where a price of exactly USD 30 belongs.
3. Construct the quality-by-price-band crosstab, retaining the order Good, Very Good, Excellent. Include marginal totals and verify that the grand total is 300.
4. Interpret the cell containing 46 and its row and column totals.
5. Explain what information price grouping loses compared with the original numerical values.

2. Choose the denominator

Calculate each percentage from counts. For every answer, state the numerator, denominator, and reference group.

1. All restaurants: Very Good **and** priced USD 30 up to, but below, USD 40.
2. Among Very Good restaurants: the proportion in that band.
3. Among restaurants in that band: the proportion rated Very Good.

4. Among Excellent restaurants: the proportion priced at least USD 30. Compare it with the corresponding proportion among Good restaurants.
5. Among restaurants in the highest band: the proportion rated Excellent. Explain why this differs from the proportion in the highest band among Excellent restaurants.
6. Draw percentage bars comparing price distributions within quality categories. Show group sizes and identify which dimension totals 100%.

3. Inspect paired numerical observations

1. Identify the observational unit and explain the sales units in Electronics. Convert the first week's sales to USD.
2. Draw a scatterplot of commercials against sales. Describe direction, form, strength, and any observations that might deserve investigation.
3. Explain why sorting the two columns independently invalidates the analysis.
4. Explain why putting commercials on the horizontal axis does not establish causation.

4. Calculate and interpret association

1. Calculate both sample means and the sum of cross-products of deviations. Use them to calculate sample covariance and check with `cov()`.
2. State covariance's units and explain its sign.
3. Calculate Pearson correlation using covariance and the two sample SDs. Check with `cor()` and interpret it in context.
4. Convert sales to USD. Recalculate covariance and correlation and explain the difference in behavior.
5. Evaluate: "A correlation of 0.93 means each extra commercial raises sales by 93%."
6. Create `u <- -4:4` and `v <- u^2`. Draw the scatterplot and calculate correlation. Explain what the example shows about zero correlation.

5. Write a responsible conclusion

In three or four sentences, summarize the Electronics result, give one plausible alternative explanation to investigate, and explain what the ten observations do not establish. Label the alternative explanation as a possibility, rather than a fact in the file. Keep this conclusion descriptive; no hypothesis test or regression model is required.

Return to Lesson 4. Then complete the integrated review.

Module 2: Descriptive Statistics Review

Bring the question, data, tools, and interpretation together

Table of contents

Purpose and submission 1
1. Define the variables and choose tools 1
2. Summarize one categorical distribution 2
3. Describe a numerical distribution 2
4. Compare groups and percentages 2
5. Describe numerical association 2
Review checklist 2

Reduced teaching slides (PDF) · Module overview

This extended lesson provides the full explanation and reproducible examples behind the slides.

Purpose and submission

Prepare a **short Quarto report** using the checked textbook CSVs. This review brings Lessons 1–4 together. You should demonstrate that you can choose and interpret a tool, rather than simply produce every available statistic.

Include one concise section for each task below, with readable tables, labeled plots, and reproducible calculations. Read the data guide and use the CSV bundle. Keep source observations unchanged. Use type-6 quartiles consistently when requested.

These are the same business cases introduced in the lessons. Refer back to their English and Arabic introductions for context. Statistical explanations and answers remain in English.

1. Define the variables and choose tools

For Restaurant quality rating and meal price, Audit duration, SoftDrink brand preference, and Electronics commercials and sales:

1. Identify the observational unit, variable type, and measurement level. Distinguish numerical counts from measured quantities where relevant.
2. Choose a suitable table, chart, or numerical measure for each task below. Justify your choice in one sentence.
3. Explain why an average restaurant ID or average brand code would not answer a meaningful quantitative question.

2. Summarize one categorical distribution

Using **SoftDrink**:

1. Construct a frequency and percentage table using `brand_book_label`.
2. Draw a bar chart and identify the most frequent category.
3. State what one record represents. Explain why percentages here describe the recorded purchases rather than all consumers in a market.

3. Describe a numerical distribution

Using **Audit**:

1. Draw a histogram and a dot plot. Briefly discuss shape, center, spread, and unusual features. Explain why 20 observations and the histogram's bin width limit fine claims about modality.
2. Report n, mean, median, range, sample SD, type-6 quartiles, and IQR, with units.
3. Identify the 1.5-IQR fences and any flagged observations. Explain why a flag does not automatically justify deletion.
4. Recommend a center-and-spread pair for a typical-duration description, while explaining what useful information the other pair contributes.

4. Compare groups and percentages

Using **Restaurant**:

1. Show a compact table of group sizes, median prices, and type-6 IQRs. Compare groups using aligned boxplots with the same quartile convention.
2. Construct a crosstab with the four price bands from Lesson 4.
3. Compare the proportion priced at least USD 30 **within** each quality category. State the three denominators.
4. Explain why the crosstab and boxplots complement each other, and why these descriptive differences do not establish a causal effect of quality ratings on price.

5. Describe numerical association

Using **Electronics**:

1. Draw a scatterplot with clear sales units.
2. Report sample covariance and Pearson correlation. Explain why covariance changes when sales are converted to USD but correlation does not.
3. Write a concise conclusion supported by both the plot and the coefficient. Include a limitation concerning causal interpretation or generalization.

Review checklist

Criterion	Evidence in your report
Correct variable meaning	Observational units, labels, units, and classifications
Suitable tool selection	A reason for each summary or plot
Reproducible calculations	CSV input, correct denominators, stated quartile convention
Clear communication	Legible plots, concise tables, context-specific interpretations
Sound conclusions	Flags investigated; descriptive associations kept separate from causation

You are ready to move on when you can explain **why** each tool suits its question, describe what it leaves out, and recognize a conclusion that exceeds the evidence.

This completes the planned descriptive-statistics sequence. Module 3 now provides five lessons, practice, and a review on probability and distributions.

Return to the Module 2 overview.